

Aigis-Sig⁺

Algorithm Specifications and Supporting Documentation (version 1.1)

Jiang Zhang¹, Yu Yu^{2,5}, Shuqin Fan¹, Kang Yang¹, Debiao He³, Runqing Xu³,
Jiangxia Ge⁴, Yuanmi Chen⁵, Weijia Wang⁶, Cong Peng³, Di Yan¹, Hui Guo¹,
Hui Zhu⁷, Ruikang Liu⁷, and Jing Pan⁷

¹ State Key Laboratory of Cryptology, Beijing, China

² Shanghai Jiao Tong University, Shanghai, China

³ Wuhan University, Wuhan, China

⁴ China Telecom Quantum Group Ltd., Hefei, China

⁵ East China Normal University, Shanghai, China

⁶ Shandong University, Qingdao, China

⁷ Xidian University, Xi'an, China

June 28, 2026

Table of Contents

1	Introduction	1
1.1	Feature Statements	1
1.2	Overview of Design Approach	2
2	Preliminaries	7
2.1	Notation	7
2.2	Syntax of Digital Signature	8
2.3	Hardness Assumptions	9
3	Specification of Aigis-Sig ⁺ Signature	10
3.1	High/Low Order Bits and Hints	10
3.2	Description of the Aigis-Sig ⁺ Scheme	12
3.3	Correctness	13
3.4	Number of Repetitions	14
4	Concrete Parameters	14
5	Implementation Details	15
5.1	Polynomial Multiplication	16
5.2	Initialization for XOF and CRH	18
5.3	Encode and Decode	19
5.4	Sampling Algorithms	25
5.5	The Aigis-Sig ⁺ Scheme	28
6	Performance and Comparison	31
6.1	Performance on x86	31
6.2	Performance on ARM	33
7	Provable Security	35
7.1	SUF-CMA Security Proof of Aigis-Sig ⁺ in the ROM	35
7.2	SUF-CMA Security Proof of Aigis-Sig ⁺ in the QROM	35
7.3	Tightness of the Security Bound	36
8	Known Attacks	37
8.1	Attacks against ALWE	37
8.2	Attacks against ASIS	43
8.3	Concrete Security Estimates	45
8.4	Attacks against Symmetric Primitives	46
9	Side-Channel Vulnerabilities and Countermeasures	46
9.1	Security Analysis	46
9.2	A Discussion on Masking Countermeasures	48
A	Proof of Theorem 1	52
A.1	Rewrite Aigis-Sig ⁺ via the FS _W A Transformation	53
A.2	SUF-CMA-to-UF-NMA Reduction of Aigis-Sig ⁺ in the ROM	57
A.3	Prove the UF-NMA Security and CUR Property	62
A.4	Putting Everything Together	64
A.5	Proof of Lemma 4	66
A.6	Proof of Lemma 5	68
A.7	Proof of Lemma 6	74

B	Proof of Theorem 2	80
B.1	New Proof of Eq. (26) and Lemma 5 in the QROM.....	81
B.2	New Proof of Eq. (34) and Lemma 6 in the QROM.....	84
B.3	Putting Everything Together	88

1 Introduction

1.1 Feature Statements

This document presents a digital signature scheme **Aigis-Sig⁺**, an improved version of **Aigis-Sig** from [35] based on the asymmetric MSIS and MLWE problems (namely, AMLWE and AMSIS). The scheme is designed to be easy to implement securely, to offer highly competitive overall performance, and to support flexible parameter choices. The following summarizes several advantages of **Aigis-Sig⁺**:

- **Enhanced rejection conditions and provable security:** **Aigis-Sig⁺** inherits the asymmetric design philosophy of **Aigis-sig** [35] and deliberately introduces several enhanced rejection conditions both to facilitate the security proof and to boost performance. Formal and detailed security proofs are provided under the AMLWE and AMSIS assumptions, in both the random oracle model (ROM) and the quantum random oracle model (QROM).
- **Easy to implement securely:** **Aigis-Sig⁺** prioritizes ease of implementation and side-channel protection as Dilithium [19] by purposely avoiding complex operations—such as Gaussian distribution sampling or floating-/fixed-point computation—that are notoriously difficult to secure. Instead, it relies solely on uniform integer sampling and a fixed NTT-friendly ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ ($n = 512$ and $q = 4171777$) across all parameter sets. This simplicity, grounded in basic integer arithmetic, greatly facilitates protection against timing and power analysis attacks.
- **Conservative security estimates:** Our concrete security analysis adopts a conservative approach, relying primarily on the Core-SVP model introduced in [4]—a standard and widely referenced methodology for estimating the hardness of lattice problems. However, recognizing recent progress in hybrid attacks on LWE, as discussed in [13], we also account for the MATZOV algorithm [30], which represents a significant improvement in attacks against certain LWE parameter regimes. By including this algorithm in our estimates, we ensure that our security claims remain valid under the latest cryptanalytic understanding, particularly for long-term security requirements.
- **Competitive performance:** Even under several key conservative design principles—such as protection against side-channel leakage and the use of conservative security parameters mentioned above—**Aigis-Sig⁺** achieves highly competitive performance. The scheme achieves the smallest public key and signature sizes (e.g., $928 + 2014 = 2942$ bytes at 128-bit classical security) of any lattice-based signature not using (discrete) Gaussian sampling or complex float/fixed point arithmetic at the same security level. The total timing for key generation, signing and verification at 128-bit classical security are less than 0.29 ms for reference implementation and 0.07 ms for AVX optimizations.
- **Patent relevance:** We have filed two Chinese patent applications for **Aigis-Sig⁺**: CN202610440090.9, pertaining to the application of enhanced rejection sampling; and CN201910339823.X, filed in early 2019, pertaining to the application of asymmetric MSIS and MLWE. To ensure freedom to operate, we have submitted the necessary statements of free use as required.

1.2 Overview of Design Approach

The signature scheme **Aigis-Sig⁺** is derived from the Fiat–Shamir heuristic [20] as applied to three-move identification protocols. To assist readers who may not be acquainted with the aforementioned framework, we first present the identification protocol between two parties A and B , which rests on the $\text{MSIS}_{n,q,k,\ell,\beta}^\infty$ problem [19]. For any positive integer n and q , let $R = \mathbb{Z}[x]/(x^n+1)$ (resp., $R_q = \mathbb{Z}_q[x]/(x^n+1)$). Let $\mathbf{H} : \{0, 1\}^* \rightarrow R_2$ be a hash function, k, ℓ, η be positive integers, and $\gamma, \beta > 0$ be reals. Formally, user A owns a pair of public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t} = \mathbf{Ax}) \in R_q^{k \times \ell} \times R_q^k$ and secret key $\mathbf{sk} = \mathbf{x} \in R_q^\ell$. In order to convince another user B (who knows the public key $\mathbf{pk}$) of his ownership of $\mathbf{sk}$, A and B can execute the following protocol: 1) A first chooses a vector $\mathbf{y} \in R^\ell$ from some distribution, and sends $\mathbf{w} = \mathbf{Ay}$ to user B ; 2) B randomly chooses an element $c \in R_q$, and sends it as a challenge to A ; 3) A computes $\mathbf{z} := \mathbf{y} + c\mathbf{x}$ and sends it back to B ; B will accept the response $\mathbf{z}$ by check if $\mathbf{Az} = \mathbf{w} + c\mathbf{t}$.

For the soundness (i.e., user A cannot cheat user B), B also has to make sure that $\gamma = \|\mathbf{z}\|_\infty$ is sufficiently small (to ensure that the $\text{MSIS}_{n,q,k,\ell,\gamma}^\infty$ problem is hard), otherwise anyone can easily complete the proof by solving a linear equation. Moreover, we require that $\|\mathbf{y}\|_\infty \gg \|\mathbf{x}\|_\infty = \beta$ (and thus $\gamma \gg \beta$) holds to prevent user B from recovering the secret $\mathbf{x}$ from the public key $\mathbf{pk}$ and the response $\mathbf{z}$. Typically, we should require $\gamma/\beta > 2^{\omega(\log \kappa)}$, where κ is the security parameter. This means that the identification protocol as well as its derived signature from the Fiat-Shamir heuristic will have a very large parameter size. To solve this problem, Lyubashevsky [28,29] introduced the rejection sampling, which allows A to abort and restart the protocol (by choosing another $\mathbf{y}$) if he thinks $\mathbf{z}$ might leak the information of $\mathbf{x}$. This technique could greatly reduce the size of $\mathbf{z}$ (since it allows to set $\gamma/\beta = \text{poly}(\kappa)$), but the cost is painful for an interactive identification protocol. Fortunately, this technique will only increase the computation time of the signer when we transform the identification protocol into a signature scheme.

For any positive integer η , S_η denotes the set of elements of R that each coefficient is taken from $\{-\eta, -\eta+1, \dots, \eta\}$. By the Fiat-Shamir heuristic, one can construct a signature scheme from the MSIS problem as follows:

- **Key generation:** randomly choose $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$, $\mathbf{x} \xleftarrow{\$} S_\eta^\ell$, and compute $\mathbf{t} = \mathbf{Ax}$. Return the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t})$ and secret key $\mathbf{sk} = (\mathbf{x}, \mathbf{pk})$.
- **Signing:** given the secret key $\mathbf{sk} = (\mathbf{x}, \mathbf{pk})$ and a message $\mu \in \{0, 1\}^*$,
 1. Randomly choose $\mathbf{y} \xleftarrow{\$} S_{\gamma-1}^\ell$;
 2. Compute $\mathbf{w} = \mathbf{Ay}$ and $c = \mathbf{H}(\mathbf{w} \parallel \mu)$;
 3. Compute $\mathbf{z} = \mathbf{y} + c\mathbf{x}$;
 4. If $\|\mathbf{z}\|_\infty \geq \gamma - \beta$, restart the computation from step 1, where β is a bound such that $\|c\mathbf{x}\|_\infty \leq \beta$ for all possible c and $\mathbf{x}$. Otherwise, return the signature $\sigma = (\mathbf{z}, c)$.
- **Verification:** given the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t})$, a message $\mu \in \{0, 1\}^*$ and a signature $\sigma = (\mathbf{z}, c)$, if $\|\mathbf{z}\|_\infty < \gamma - \beta$ and $c = \mathbf{H}(\mathbf{Az} - c\mathbf{t} \parallel \mu)$, return 1, otherwise return 0.

Informally, we require the $\text{MSIS}_{n,q,k,\ell,\eta}^\infty$ problem to be hard for the security of the secret key (i.e., it is computationally infeasible to compute sk from pk). Moreover, we also require the $\text{MSIS}_{n,q,k,\ell,2\gamma}^\infty$ problem to be hard for the unforgeability of signatures (i.e., it is computationally infeasible to forge a valid signature). Since $\|\mathbf{c}\mathbf{x}\|_\infty \leq \beta$, for any $(c, \mathbf{x})$ and $\mathbf{z}$ output by the signing algorithm there always exists a $\mathbf{y} \in S_\gamma^\ell$ such that $\mathbf{z} = \mathbf{y} + \mathbf{c}\mathbf{x}$, which guarantees that the signature will not leak the information of the secret key. In terms of efficiency, the signing algorithm will repeat about $\left(\frac{2(\gamma-\beta)-1}{2\gamma-1}\right)^{-n\cdot\ell}$ times to output a signature with size about $n\ell\lceil\log_2(2(\gamma-\beta)-1)\rceil + n$ bits. Ideally, we wish to set ℓ as small as possible to obtain the most compact signature size, but for the hardness of the underlying MSIS problems we have to choose a relatively large ℓ .

To mediate the above conflict, one can use the MLWE problem, which can be seen as a special MSIS problem, to reduce the size of the key and signature [7]. Formally, we can obtain the following improved signature scheme:

- **Key generation:** randomly choose $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$, and $\mathbf{s}_1 \xleftarrow{\$} S_\eta^\ell, \mathbf{s}_2 \xleftarrow{\$} S_\eta^k$, compute $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Return the public key $\text{pk} = (\mathbf{A}, \mathbf{t})$ and secret key $\text{sk} = (\mathbf{s}_1, \mathbf{s}_2, \text{pk})$.
- **Signing:** given the secret key $\text{sk} = (\mathbf{s}_1, \mathbf{s}_2, \text{pk})$ and a message $\mu \in \{0, 1\}^*$,
 1. Randomly choose $\mathbf{y} \xleftarrow{\$} S_{\gamma-1}^{\ell+k}$;
 2. Compute $\mathbf{w} = (\mathbf{A} \parallel \mathbf{I}_k)\mathbf{y}$ and $c = H(\mathbf{w} \parallel \mu)$;
 3. Compute $\mathbf{z} = \mathbf{y} + c \begin{pmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \end{pmatrix}$;
 4. If $\|\mathbf{z}\|_\infty \geq \gamma - \beta$, restart the computation from step 1, where β is a bound such that $\|\mathbf{c}\mathbf{s}_1\|_\infty, \|\mathbf{c}\mathbf{s}_2\|_\infty \leq \beta$ holds for all possible $c, \mathbf{s}_1, \mathbf{s}_2$. Otherwise, output the signature $\sigma = (\mathbf{z}, c)$.
- **Verification:** given the public key $\text{pk} = (\mathbf{A}, \mathbf{t})$, a message $\mu \in \{0, 1\}^*$ and a signature $\sigma = (\mathbf{z}, c)$, if $\|\mathbf{z}\|_\infty < \gamma - \beta$ and $c = H((\mathbf{A} \parallel \mathbf{I}_k)\mathbf{z} - \mathbf{c}\mathbf{t} \parallel \mu)$, return 1, otherwise return 0.

Furthermore, since $\mathbf{w} = (\mathbf{A} \parallel \mathbf{I}_k)\mathbf{y} = \mathbf{A}\mathbf{y}_1 + \mathbf{y}_2$ where $\mathbf{y} = (\mathbf{y}_1^T, \mathbf{y}_2^T)$ and $\gamma \ll q$, we have that the higher order bits of (each coefficient of) $\mathbf{w}$ is almost determined by higher order bits of (the corresponding coefficient of) $\mathbf{A}\mathbf{y}_1$. This fact has been utilized by [7, 19] to compress the signature size. Formally, let $\text{HighBits}(\mathbf{z}, 2\gamma_2)$ and $\text{LowBits}(\mathbf{z}, 2\gamma_2)$ be polynomial vector defined by the higher order bits and lower order bits of a polynomial vector $\mathbf{z} \in R_q^k$ related to a parameter γ_2 . We can obtain the following signature scheme:

- **Key generation:** randomly choose $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$, and $\mathbf{s}_1 \xleftarrow{\$} S_\eta^\ell, \mathbf{s}_2 \xleftarrow{\$} S_\eta^k$, compute $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Return the public key $\text{pk} = (\mathbf{A}, \mathbf{t})$ and secret key $\text{sk} = (\mathbf{s}_1, \mathbf{s}_2, \text{pk})$.
- **Signing:** given the secret key $\text{sk} = (\mathbf{s}_1, \mathbf{s}_2, \text{pk})$ and a message $\mu \in \{0, 1\}^*$,
 1. Randomly choose $\mathbf{y} \xleftarrow{\$} S_{\gamma_1-1}^\ell$;
 2. Compute $\mathbf{w} = \mathbf{A}\mathbf{y}$ and $c = H(\text{HighBits}(\mathbf{w}, 2\gamma_2) \parallel \mu)$;
 3. Compute $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$;

4. If $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ or $\text{LowBits}(\mathbf{A}\mathbf{y} - \mathbf{c}\mathbf{s}_2, 2\gamma_2) \geq \gamma_2 - \beta$, restart the computation from step 1, where β is a bound such that $\|\mathbf{c}\mathbf{s}_1\|_\infty, \|\mathbf{c}\mathbf{s}_2\|_\infty \leq \beta$ hold for all possible $c, \mathbf{s}_1, \mathbf{s}_2$. Otherwise, output the signature $\sigma = (\mathbf{z}, c)$.
- **Verification:** given the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t})$, a message $\mu \in \{0, 1\}^*$ and a signature $\sigma = (\mathbf{z}, c)$, if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ and $c = \text{H}(\text{HighBits}(\mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}, 2\gamma_2)\|\mu)$, return 1, otherwise return 0.

Essentially, the checks in step 4 are used to ensure that 1) the signature $(\mathbf{z}, c)$ will not leak the information of $\mathbf{s}_1$ and $\mathbf{s}_2$; and 2) $\text{HighBits}(\mathbf{A}\mathbf{z} - \mathbf{c}\mathbf{t}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y} - \mathbf{c}\mathbf{s}_2, 2\gamma_2) = \text{HighBits}(\mathbf{w}, 2\gamma_2)$ (note that $\mathbf{w} = \mathbf{A}\mathbf{y} = \mathbf{A}\mathbf{y} - \mathbf{c}\mathbf{s}_2 + \mathbf{c}\mathbf{s}_2$, $\text{LowBits}(\mathbf{A}\mathbf{y} - \mathbf{c}\mathbf{s}_2, 2\gamma_2) < \gamma_2 - \beta$ and $\|\mathbf{c}\mathbf{s}_2\|_\infty \leq \beta$). By setting $\gamma_2 \leq \gamma_1$, we require the $\text{MLWE}_{n,q,k,\ell,\eta}$ problem and the (variant of) $\text{MSIS}_{n,q,k,(\ell+k+1),2\gamma_1+2}^\infty$ problem to be hard to ensure the security of the secret key and the unforgeability of the signature, respectively.

By a careful examination on the above scheme, one can find that the computational efficiency of the signing algorithm is determined by the expected number of repetitions in step 4:

$$\underbrace{\left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1 - 1}\right)^{-n \cdot \ell}}_{=N_1} \cdot \underbrace{\left(\frac{2(\gamma_2 - \beta) - 1}{2\gamma_2 - 1}\right)^{-n \cdot k}}_{=N_2},$$

where N_1 and N_2 are determined by the first and second checks in step (4), respectively. Clearly, it is possible to modify N_1 and N_2 while keeping the total number of repetitions $N = N_1 \cdot N_2$ unchanged. Note that the size of the signature is related to γ_1 and is irrelevant to γ_2 , which means that a shorter signature can be obtained by using a smaller γ_1 . However, simply using a smaller γ_1 will also lead to a bigger N_1 , and thus a worse computational efficiency. In order to obtain a short signature size without (significantly) affecting the computational efficiency:

- We use the AMLWE problem for the security of the secret key, which allows us to use a smaller γ_1 by reducing $\|\mathbf{s}_1\|_\infty$ (and thus $\beta = \|\mathbf{c}\mathbf{s}_1\|_\infty$ in the expression of N_1);
- We use the AMSIS problem for the unforgeability of the signatures, which further allows us to use a smaller γ_1 by increasing γ_2 to keep $N = N_1 \cdot N_2$ unchanged.

Note that reducing $\|\mathbf{s}_1\|_\infty$ (by simply choosing $\mathbf{s}_1 \xleftarrow{\$} S_{\eta_1}$ with a smaller η_1) may weaken the hardness of the underlying AMLWE problem (if we do not change other parameters). Our idea is to choose $\mathbf{s}_2 \xleftarrow{\$} S_{\eta_2}$ with a larger η_2 (and thus $\|\mathbf{s}_2\|_\infty$) to remain the hardness. Similarly, increasing γ_2 will weaken the hardness of the underlying AMSIS problem, and we choose to reduce γ_1 to remain the hardness. Both strategies crucially rely on the asymmetries of the underlying problems. The strategy of only taking the higher order bits of $\mathbf{w}$ to compute the challenge $\mathbf{c}$ also provides a natural way to compress the public key size by simply cutting off the lower d bits $\mathbf{t}_0$ of $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$. For this, two additional algorithms **MakeHint** and **UseHint** are defined to ensure the correctness of the resulting scheme [19]. Formally, we have the following signature scheme **Aigis-Sig** from AMLWE and AMSIS:

- **Key generation:** randomly choose $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$, and $\mathbf{s}_1 \xleftarrow{\$} S_{\eta_1}^\ell, \mathbf{s}_2 \xleftarrow{\$} S_{\eta_2}^k$, compute $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Let $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$, return the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t}_1)$ and secret key $\mathbf{sk} = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{pk}, \mathbf{t}_0)$.
- **Signing:** given the secret key $\mathbf{sk} = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{pk}, \mathbf{t}_0)$ and a message $\mu \in \{0, 1\}^*$,
 1. Randomly choose $\mathbf{y} \xleftarrow{\$} S_{\gamma_1-1}^\ell$;
 2. Compute $\mathbf{w} = \mathbf{A}\mathbf{y}$, $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$ and $c = H(\mathbf{w}_1 \parallel \mu)$;
 3. Compute $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and $\mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2)$;
 4. If $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta_1$ or $\|\mathbf{r}\|_\infty \geq \gamma_2 - \beta_2$, restart the computation from step 1, where β_1 (resp., β_2) is a bound such that $\|c\mathbf{s}_1\|_\infty \leq \beta_1$ (resp., $\|c\mathbf{s}_2\|_\infty \leq \beta_2$) holds for all possible $c, \mathbf{s}_1$ (resp., $c, \mathbf{s}_2$);
 5. Compute $\mathbf{v} = c\mathbf{t}_0$ and $\mathbf{h} = \text{MakeHint}_q(-\mathbf{v}, \mathbf{A}\mathbf{y} - c\mathbf{s}_2 + \mathbf{v}, 2\gamma_2)$. If $\|\mathbf{v}\|_\infty \geq \gamma_2$ or the number of 1's in $\mathbf{h}$ is greater than ω , restart the computation from step 1;
 6. Finally, output the signature $\sigma = (\mathbf{z}, c, \mathbf{h})$.
- **Verification:** given the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t}_1)$, a message $\mu \in \{0, 1\}^*$ and a signature $\sigma = (\mathbf{z}, c, \mathbf{h})$ as inputs, compute $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$, if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1, c = H(\mathbf{w}'_1 \parallel \mu)$ and the number of 1's in $\mathbf{h}$ is $\leq \omega$, return 1, otherwise return 0.

At first glance, the strategy of using **MakeHint** and **UseHint** [19] to compress the public key seems to be irrelevant to the scheme's security, but it indeed poses new restrictions on the parameters for concrete security. In particular, the condition $2\gamma_2 \leq \gamma_1$ is required for the hardness of the MSIS problem in security proof [19], which means that we have to halve the magnitude of γ_2 to maintain the same security as that without using **MakeHint** and **UseHint**. The loss of a factor of 2 in γ_2 also appears in the security proof of **Aigis-Sig** [35], which requires the $\text{AMLWE}_{n,q,k,\ell,\eta_1,\eta_2}^\infty$ and $\text{AMSIS}_{n,q,k,\ell,k+1,4\gamma_2+2,2\gamma_1}^\infty$ assumptions (note that by setting $\gamma_1 = 2\gamma_2$, we have $2\gamma_1 \approx 4\gamma_2 + 2$).

To solve the above problem, we introduce an enhanced check with another parameter $\gamma_3 \ll 2\gamma_2$ in both the signing and verification algorithms, so that we can use much larger γ_2 to obtain better computational efficiency and smaller public key sizes without affecting the concrete security. Moreover, we optimize the hint sizes by forcing the signing algorithm to only accept an hint $\mathbf{h}$ belongs to some admissible set A_ω . Formally, let $\bar{S}_\alpha \in R$ be the set of polynomials with coefficients belongs to $\{-\alpha + 1, \dots, \alpha\}$, we also replace the symmetric distribution in the signing algorithm with $\bar{S}_\alpha$ for ease of implementation. Our improved signature scheme **Aigis-Sig**⁺ from **AMLWE** and **AMSIS** is described as below:

- **Key generation:** randomly choose $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$, and $\mathbf{s}_1 \xleftarrow{\$} S_{\eta_1}^\ell, \mathbf{s}_2 \xleftarrow{\$} S_{\eta_2}^k$, compute $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Let $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$, return the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t}_1)$ and secret key $\mathbf{sk} = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{pk}, \mathbf{t}_0)$.
- **Signing:** given the secret key $\mathbf{sk} = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{pk}, \mathbf{t}_0)$ and a message $\mu \in \{0, 1\}^*$,
 1. Randomly choose $\mathbf{y} \xleftarrow{\$} \bar{S}_{\gamma_1}^\ell$;
 2. Compute $\mathbf{w} = \mathbf{A}\mathbf{y}$, $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$ and $c = H(\mathbf{w}_1 \parallel \mu)$;
 3. Compute $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and $\mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2)$;

4. If $\mathbf{z} \notin \bar{S}_{\gamma_1 - \beta_1}^\ell$ or $\mathbf{r} \notin \bar{S}_{\gamma_2 - \beta_2}^k$, restart the computation from step 1, where β_1 (resp., β_2) is a bound such that $\|\mathbf{cs}_1\|_\infty \leq \beta_1$ (resp., $\|\mathbf{cs}_2\|_\infty \leq \beta_2$) holds for all possible $c, \mathbf{s}_1$ (resp., $c, \mathbf{s}_2$);
 5. If $\mathbf{v} = \mathbf{r} + \mathbf{ct}_0 \notin \bar{S}_{\gamma_3}^k$, restart the computation from step 1;
 6. Compute $\mathbf{h} = \text{MakeHint}(-\mathbf{ct}_0, \mathbf{A}\mathbf{y} - \mathbf{cs}_2 + \mathbf{ct}_0, 2\gamma_2)$. If $\mathbf{h} \notin A_\omega$ for some admissible set A_ω , restart the computation from step 1;
 7. Finally, output the signature $\sigma = (\mathbf{z}, c, \mathbf{h})$.
- **Verification:** given the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t}_1)$, a message $\mu \in \{0, 1\}^*$ and a signature $\sigma = (\mathbf{z}, c, \mathbf{h})$, compute $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - \mathbf{ct}_1 \cdot 2^d, 2\gamma_2)$, if $\mathbf{z} \in \bar{S}_{\gamma_1 - \beta_1}^\ell$, $\mathbf{h} \in A_\omega$, $\mathbf{A}\mathbf{z} - \mathbf{ct}_1 \cdot 2^d - \mathbf{w}'_1 \cdot 2\gamma_2 \in \bar{S}_{\gamma_3}^k$ and $c = \text{H}(\mathbf{w}'_1 \parallel \mu)$, return 1, otherwise return 0.

For correctness, we will show that for any $\mathbf{h} = \text{MakeHint}(-\mathbf{ct}_0, \mathbf{A}\mathbf{y} - \mathbf{cs}_2 + \mathbf{ct}_0, 2\gamma_2)$ computed in step 6 of the signing algorithm, we always have $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - \mathbf{ct}_1 \cdot 2^d, 2\gamma_2) = \mathbf{w}_1$ as long as $\gamma_3 < 2\gamma_2$. In fact, we will present a pair of modified **MakeHint** and **UseHint** algorithms which are functionally equivalent to the original ones for $\gamma_3 < 2\gamma_2$ but admitting more useful properties for both correctness and security. Given this, the security of the above scheme can be reduced to $\text{AMLWE}_{n,q,k,\ell,\eta_1,\eta_2}$ and $\text{AMSIS}_{n,q,k,\ell,k+1,2\gamma_3,2\gamma_1}^\infty$ (which is much weaker than the previous $\text{AMSIS}_{n,q,k,\ell,k+1,4\gamma_2,2\gamma_1}^\infty$ for Aigis-Sig in [36] when $\gamma_3 \ll 2\gamma_2$, and allows us to choose smaller parameters for the same security). In practice, we will choose parameter γ_3 such that the rejection probability in step 5 of the signing algorithm is less than 1%. Our concrete parameters show that γ_3 can actually be $1.2 \sim 1.7$ times smaller than $2\gamma_2$. Moreover, the check in step 6 is to optimize the sizes of storing $\mathbf{h}$, and the set A_ω is chosen to ensure that $\mathbf{h} \notin A_\omega$ is less than 1%.

The last thing we have to deal with is that $\mathbf{w}_1$ is required to have high min-entropy so that it is hard for an adversary to guess when simulating the random oracle $\text{H}(\mathbf{w}_1 \parallel *)$ in existing security proofs [26, 8, 17, 24], which in turn incurs extra restrictions on parameters such as $\gamma_1 > \gamma_2$ (see [26, Lem.4.7 and C.1]), and thus prevent us from reducing the signature size by setting $\gamma_1 \ll \gamma_2$. We address this problem by employing a slightly stricter rejection strategy on $\mathbf{r}$, which basically allows us to replace $\mathbf{w} = \mathbf{A}\mathbf{y}$ with $\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e} \in R_q^k$ for some $\mathbf{e} \in R_q^k$ in the security proof. Consequently, we can directly prove that guessing the higher bits $\mathbf{w}_1$ of $\mathbf{w}$ is computationally hard for any PPT adversary under the AMLWE assumption. Our final signature is described as below:

- **Key generation:** randomly choose $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$, and $\mathbf{s}_1 \xleftarrow{\$} S_{\eta_1}^\ell, \mathbf{s}_2 \xleftarrow{\$} S_{\eta_2}^k$, compute $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Let $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$, return the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t}_1)$ and secret key $\mathbf{sk} = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{pk}, \mathbf{t}_0)$.
- **Signing:** given the secret key $\mathbf{sk} = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{pk}, \mathbf{t}_0)$ and a message $\mu \in \{0, 1\}^*$,
 1. Randomly choose $\mathbf{y} \xleftarrow{\$} \bar{S}_{\gamma_1}^\ell$;
 2. Compute $\mathbf{w} = \mathbf{A}\mathbf{y}$, $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$ and $c = \text{H}(\mathbf{w}_1 \parallel \mu)$;
 3. Compute $\mathbf{z} = \mathbf{y} + \mathbf{cs}_1$ and $\mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y} - \mathbf{cs}_2, 2\gamma_2)$;
 4. If $\mathbf{z} \notin \bar{S}_{\gamma_1 - \beta_1}^\ell$ or $\mathbf{r} \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, restart the computation from step 1, where β_1 (resp., β_2) is a bound such that $\|\mathbf{cs}_1\|_\infty \leq \beta_1$ (resp., $\|\mathbf{cs}_2\|_\infty \leq \beta_2$) holds for all possible $c, \mathbf{s}_1$ (resp., $c, \mathbf{s}_2$)

5. If $\mathbf{v} = \mathbf{r} + c\mathbf{t}_0 \notin \bar{S}_{\gamma_3}^k$, restart the computation from step 1;
 6. Compute $\mathbf{h} = \text{MakeHint}(-c\mathbf{t}_0, \mathbf{A}\mathbf{y} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$. If $\mathbf{h} \notin A_\omega$ for some admissible set A_ω , restart the computation from step 1;
 7. Finally, output the signature $\sigma = (\mathbf{z}, c, \mathbf{h})$.
- **Verification:** given the public key $\mathbf{pk} = (\mathbf{A}, \mathbf{t}_1)$, a message $\mu \in \{0, 1\}^*$ and a signature $\sigma = (\mathbf{z}, c, \mathbf{h})$, compute $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$, if $\mathbf{z} \in \bar{S}_{\gamma_1 - \beta_1}^\ell$, $\mathbf{h} \in A_\omega$, $\mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}'_1 \cdot 2\gamma_2 \in \bar{S}_{\gamma_3}^k$ and $c = H(\mathbf{w}'_1 \| \mu)$, return 1, otherwise return 0.

Since $\eta_1 \ll \beta_2 \ll \gamma_2$, the new rejection condition $\mathbf{r} \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$ will only slightly increase the number of repetitions (less than 1% for our concrete parameters) to generate a signature. Moreover, for any $\mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2) \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, we always have that

$$\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2) = \text{HighBits}(\mathbf{A}\mathbf{y}) = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e})$$

for all $\mathbf{e} \xleftarrow{\$} S_{\eta_1}^k$. This directly allows us to show that the probability for all PPT adversaries to guess $\mathbf{w}_1$ is negligible under the $\text{AMLWE}_{n,q,k,\ell,\gamma_1,\eta_1}$ assumption (i.e., $\mathbf{A}\mathbf{y} + \mathbf{e}$ is computationally indistinguishable from uniform). Finally, it is worth to note that the $\text{AMLWE}_{n,q,k,\ell,\gamma_1,\eta_1}$ assumption can be essentially used in the security proof for free, because the $\text{AMLWE}_{n,q,k,\ell,\eta_1,\eta_2}$ assumption for $\eta_2 \ll \gamma_1$ is already required to ensure the security of the signing key. In other words, the above strategy does not introduce any additional assumptions.

2 Preliminaries

2.1 Notation

We denote by R the ring $R = \mathbb{Z}[X]/(X^n + 1)$ and by R_q the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, where n is a power of 2 so that $X^n + 1$ is a cyclotomic polynomial. The regular font letters (e.g., a, b) represent elements in R or R_q (including elements in $\mathbb{Z}$ or $\mathbb{Z}_q$), and bold lower-case letters (e.g., $\mathbf{a}, \mathbf{b}$) denote vectors with coefficients in R or R_q . By default, all vectors will be column vectors. Bold upper-case letters (e.g., $\mathbf{A}, \mathbf{B}$) represent matrices.

For any positive integer η , define the integer sets $S_\eta = \{-\eta, \dots, \eta\}$ and $\bar{S}_\eta = \{-\eta + 1, \dots, \eta\}$. We slightly abuse notation by also using S_η (resp., $\bar{S}_\eta$) to denote the set of polynomials in R_q all of whose coefficients lie in $\{-\eta, \dots, \eta\}$ (resp., $\{-\eta + 1, \dots, \eta\}$). Let B_τ denotes the set consists of elements in R which contains τ coefficients of $-1, 1$ and others 0. Obviously, we have $|B_\tau| = 2^\tau \cdot \binom{n}{\tau}$. We denote by $x \xleftarrow{\$} D$ sampling x according to a distribution D and by $x \xleftarrow{\$} S$ denote sampling x from a set S uniformly at random. For two bit-strings s and t , $s \| t$ denotes the concatenation of s and t . We use $\log_b$ to denote the logarithm function in base b (e.g., 2 or natural constant e) and $\log$ to represent $\log_e$. We say that a function $f : \mathbb{N} \rightarrow [0, 1]$ is *negligible*, if for every positive c and all sufficiently large κ it holds that $f(\kappa) < 1/\kappa^c$. We denote by $\text{negl} : \mathbb{N} \rightarrow [0, 1]$ an (unspecified) negligible function. We say that f is *overwhelming* if $1 - f$ is negligible.

Modular reductions. For an even positive integer α , we define $r' = r \bmod^\pm \alpha$ as the unique element in the range $\bar{S}_{\alpha/2} = (-\frac{\alpha}{2}, \frac{\alpha}{2}]$ such that $r' = r \bmod \alpha$. For an odd positive integer α , we define $r' = r \bmod^\pm \alpha$ as the unique element in the range $S_{(\alpha-1)/2} = [-\frac{\alpha-1}{2}, \frac{\alpha-1}{2}]$ such that $r' = r \bmod \alpha$. For any positive integer α , we define $r' = r \bmod^+ \alpha$ as the unique element in the range $[0, \alpha)$ such that $r' = r \bmod \alpha$. When the exact representation is not important, we simply write $r \bmod \alpha$.

Sizes of elements. For an element $w \in \mathbb{Z}_q$, we write $\|w\|_\infty$ to mean $|w \bmod^\pm q|$. The ℓ_∞ and ℓ_2 norms of a ring element $w = w_0 + w_1X + \dots + w_{n-1}X^{n-1} \in R$ are defined as follows:

$$\|w\|_\infty = \max_i \|w_i\|_\infty, \quad \|w\| = \sqrt{\|w_0\|_\infty^2 + \dots + \|w_{n-1}\|_\infty^2}.$$

Similarly, for $\mathbf{w} = (w_1, \dots, w_k) \in R^k$, we define

$$\|\mathbf{w}\|_\infty = \max_i \|w_i\|_\infty, \quad \|\mathbf{w}\| = \sqrt{\|w_1\|^2 + \dots + \|w_k\|^2}.$$

2.2 Syntax of Digital Signature

A digital signature scheme $\Pi_{sig} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ consists of three PPT algorithms:

- **KeyGen**(1^κ): given the security parameter κ as input, output a verification key vk and a secret signing key sk , denoted as $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$.
- **Sign**(sk, M): given the signing key sk and a message $M \in \{0, 1\}^*$ as inputs, output a signature σ on M , denoted as $\sigma \leftarrow \text{Sign}(\text{sk}, M)$.
- **Verify**(vk, C): given the verification key vk , message $M \in \{0, 1\}^*$ and a string $\sigma \in \{0, 1\}^*$ as inputs, output 1 if σ is a valid signature on M , otherwise 0, denoted as $1/0 \leftarrow \text{Verify}(\text{vk}, M, \sigma)$.

For correctness, we require that for any $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$, any message $M \in \{0, 1\}^*$, and any $\sigma \leftarrow \text{Sign}(\text{sk}, M)$, the equation $\text{Verify}(\text{vk}, M, \sigma) = 1$ holds with overwhelming probability, where the probability is taken over the choices of the random coins used in **KeyGen**, **Sign** and **Verify**.

For security, we consider both existential unforgeability against chosen message attacks (UF-CMA) and strong UF-CMA (SUF-CMA), which are modeled by the following security game between a challenger $\mathcal{C}$ and a forger $\mathcal{F}$:

KeyGen. The challenger $\mathcal{C}$ first runs $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\kappa)$ with the security parameter κ . Then, it gives the verification key vk to the forger $\mathcal{F}$, and keeps the signing secret key sk to itself.

Signing. The forger $\mathcal{F}$ is allowed to ask the signature on any message M . The challenger $\mathcal{C}$ computes and sends $\sigma \leftarrow \text{Sign}(\text{sk}, M)$ to the forger $\mathcal{F}$. The forger can repeat this any polynomial number of times.

Forge. $\mathcal{F}$ outputs a message-signature pair (M^*, σ^*) .

Let $\mathcal{Q}_m$ be the set of all messages queried by $\mathcal{F}$ in the signing phase. We say that $\mathcal{F}$ wins the EUF-CMA security game if $M^* \notin \mathcal{Q}_m$ and $\text{Verify}(\text{vk}, M^*, \sigma^*) = 1$. Denote $\text{Adv}_{\Pi_{sig}, \mathcal{F}}^{\text{uf-cma}}(1^\kappa)$ as the advantage of $\mathcal{F}$ wins in the EUF-CMA security game.

Definition 1 (UF-CMA). Let κ be the security parameter. A signature scheme Π_{sig} is said to be existentially unforgeable against chosen message attacks if the advantage $\text{Adv}_{\Pi_{\text{sig}}, \mathcal{F}}^{\text{uf-cma}}(1^\kappa)$ is negligible in κ for any PPT forger $\mathcal{F}$.

Let $\mathcal{Q}_{ms}$ be the set of all message-signature pairs obtained by $\mathcal{F}$ in the signing phase. We say that $\mathcal{F}$ wins the SUF-CMA security game if $(M^*, \sigma^*) \notin \mathcal{Q}_{ms}$ and $\text{Verify}(\text{vk}, M^*, \sigma^*) = 1$. Denote $\text{Adv}_{\Pi_{\text{sig}}, \mathcal{F}}^{\text{suf-cma}}(1^\kappa)$ as the advantage of $\mathcal{F}$ wins in the SUF-CMA security game.

Definition 2 (SUF-CMA). Let κ be the security parameter. A signature scheme Π_{sig} is said to be strongly existentially unforgeable against chosen message attacks if the advantage $\text{Adv}_{\Pi_{\text{sig}}, \mathcal{F}}^{\text{suf-cma}}(1^\kappa)$ is negligible in κ for any PPT forger $\mathcal{F}$.

2.3 Hardness Assumptions

Aigis-Sig⁺ is based on the asymmetric variant of the MLWE and MSIS assumptions formally defined in [36]. Let n, q be positive integers with n being a powers of 2. Let $R = \mathbb{Z}[X]/(X^n + 1)$ (resp., $R_q = \mathbb{Z}_q[X]/(X^n + 1)$) be integer polynomials of degree less than n .

2.3.1 The AMLWE Assumption. Let $k, \ell \geq 1, \alpha_1, \alpha_2$ be positive integers. Let $\chi_{\alpha_1}, \chi_{\alpha_2}$ be two distribution over R . The decisional AMLWE problem $\text{AMLWE}_{n,q,k,\ell,\alpha_1,\alpha_2}$ asks to distinguish $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$ and uniform over $R_q^{k \times \ell} \times R_q^k$, where $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}, \mathbf{s} \xleftarrow{\$} \chi_{\alpha_1}^\ell, \mathbf{e} \xleftarrow{\$} \chi_{\alpha_2}^k$. When $\alpha_1 = \alpha_2$, the AMLWE problem is the standard MLWE problem. Given the current state of known attacks against (M)LWE, we have the following hardness relations:

$$\text{MLWE}_{n,q,k,\ell,\min(\alpha_1,\alpha_2)} \leq \text{AMLWE}_{n,q,k,\ell,\alpha_1,\alpha_2} \leq \text{MLWE}_{n,q,k,\ell,\max(\alpha_1,\alpha_2)}.$$

For our purpose, we are particularly interested in the problem $\text{AMLWE}_{n,q,k,\ell,\eta_1,\eta_2}$ with uniform distributions $\mathbf{s} \xleftarrow{\$} (S_{\eta_1}^n)^\ell, \mathbf{e} \xleftarrow{\$} (S_{\eta_2}^n)^k$. The AMLWE assumption says that it is hard for any (quantum) PPT algorithms to solve the decisional $\text{AMLWE}_{n,q,k,\ell,\eta_1,\eta_2}$ problem with non-negligible advantage over a random guess.

2.3.2 The AMSIS Assumption. Given a uniform matrix $\mathbf{A} \in R^{k \times (\ell_1 + \ell_2 - k)}$, the (Hermite Normal Form) AMSIS problem $\text{AMISIS}_{n,q,k,\ell_1,\ell_2,\beta_1,\beta_2}^\infty$ over ring R_q asks to find a non-zero vector $\mathbf{x} \in R_q^{\ell_1 + \ell_2} \setminus \{\mathbf{0}\}$ such that $(\mathbf{I}_k \| \mathbf{A})\mathbf{x} = \mathbf{0} \bmod q$, $\|\mathbf{x}_1\|_\infty \leq \beta_1$ and $\|\mathbf{x}_2\|_\infty \leq \beta_2$, where $\mathbf{x} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{pmatrix} \in R_q^{\ell_1 + \ell_2}, \mathbf{x}_1 \in R_q^{\ell_1}, \mathbf{x}_2 \in R_q^{\ell_2}$. When $\beta_1 = \beta_2$, the AMSIS problem is the standard MSIS problem. In particular, we are easy to prove the following hardness relation of MSIS and AMSIS problems:

$$\text{MSIS}_{n,q,k,\ell_1 + \ell_2, \max(\beta_1, \beta_2)}^\infty \leq \text{AMISIS}_{n,q,k,\ell_1, \ell_2, \beta_1, \beta_2}^\infty \leq \text{MSIS}_{n,q,k,\ell_1 + \ell_2, \min(\beta_1, \beta_2)}^\infty.$$

The AMSIS assumption says that it is hard for any (quantum) PPT algorithms to solve the $\text{AMISIS}_{n,q,k,\ell_1,\ell_2,\beta_1,\beta_2}^\infty$ problem with non-negligible advantage.

2.3.3 The SelfTargetAM SIS Assumption. Let $H : \mathcal{B}^{48} \times R_q^k \rightarrow B_h$ be a hash function. Given a uniformly random matrix $\mathbf{A} \in R_q^{k \times (\ell_1 + \ell_2 - k)}$ and a uniform ring element $\mathbf{t} \in R_q^k$, the SelfTargetAM SIS problem asks to find a vector

$$\mathbf{x} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ c \end{pmatrix} \text{ and } \mu \in \{0, 1\}^*, \text{ such that } \|\mathbf{x}_1\|_\infty \leq \beta_1, \|\mathbf{x}_2\|_\infty \leq \beta_2, \|c\|_\infty \leq 1$$

and $H(\mu, (\mathbf{I}_k \| \mathbf{A} \| \mathbf{t})\mathbf{x}) = c$ holds. When H is modeled as a classical random oracle, there is a (non-tight) reduction from AM SIS to SelfTargetAM SIS, which is the same as the one from MSIS to SelfTargetMSIS [19]. The SelfTargetAM SIS assumption says that it is hard for any (quantum) PPT algorithms to solve the SelfTargetAM SIS $_{n,q,k,\ell_1,\ell_2,\beta_1,\beta_2}^\infty$ problem with non-negligible advantage.

3 Specification of Aigis-Sig⁺ Signature

Aigis-Sig⁺ will adopt the ideas in Dilithium [19] to extract the “higher-order” bits and “lower-order” bits from elements in $\mathbb{Z}_q$. The goal is that given an arbitrary element $r \in \mathbb{Z}_q$ and another small element $z \in \mathbb{Z}_q$, we would like to recover the higher order bits of $r + z$ without needing to store z . We start by defining several supporting algorithms and then re-establish two technical lemmas that are essential to the correctness and security of our signature scheme.

3.1 High/Low Order Bits and Hints.

Algorithms 1–4 closely follow those of Dilithium [19], except that our MakeHint_q is described in a way to accommodate our enhanced rejection conditions. It is, however, functionally equivalent to Dilithium’s MakeHint_q up to input differences.

Algorithm 1: Power2Round $_q(r, d)$

```

1  $r := r \bmod^+ q$ ;
2  $r_0 := r \bmod^\pm 2^d$ ;
3  $r_1 := (r - r_0)/2^d$ ;
4 return  $(r_1, r_0)$ ;
```

Algorithm 2: Decompose $_q(r, \alpha)$

```

1  $r := r \bmod^+ q$ ;
2  $r_0 := r \bmod^\pm \alpha$ ;
3 if  $r - r_0 = q - 1$  then
4    $r_1 := 0$ ;
5    $r_0 := r_0 - 1$ ;
6 else
7    $r_1 := (r - r_0)/\alpha$ ;
8 end
9 return  $(r_1, r_0)$ ;
```

Algorithm 3: MakeHint_q(z, r_1, α)

```

1  $z' = z \bmod^+ q$ ;
2 if  $z' \leq \alpha/2$  or  $z' > q - \alpha/2$  or ( $z' = q - \alpha/2$  and  $r_1 = 0$ ) then
3    $h := 0$ ;
4 else
5    $h := 1$ ;
6 end
7 return  $h$ ;

```

Algorithm 4: UseHint_q(h, r, α)

```

1  $k := (q - 1)/\alpha$ ;
2  $(r_1, r_0) := \text{Decompose}_q(r, \alpha)$ ;
3 if  $h = 1$  and  $r_0 > 0$  then
4    $r_1 := (r_1 + 1) \bmod^+ k$ ;
5 end
6 if  $h = 1$  and  $r_0 \leq 0$  then
7    $r_1 := (r_1 - 1) \bmod^+ k$ ;
8 end
9 return  $r_1$ ;

```

The above algorithms are described for integers in $\mathbb{Z}_q$, but can be naturally extended to $\mathbb{Z}_q^n$ or R_q by applying them independently to each coefficients. The following two lemmas prove several useful properties of the above algorithms, which are crucial for the correctness and security of our signature scheme.

Lemma 1. *Let q and α be positive integers such that $q > 2\alpha$, $q \bmod \alpha = 1$ and α is even. Suppose that $r \in \mathbb{Z}_q, z \in \mathbb{Z}$ are integers, where $|z| \leq \beta < \alpha/2$. Let $(r_1, r_0) = \text{Decompose}_q(r, \alpha)$ and $(r'_1, r'_0) = \text{Decompose}_q(r + z, \alpha)$. Then,*

1. $r_0 + z \in \bar{S}_{\alpha/2-\beta} \Rightarrow r'_1 = r_1 \wedge r'_0 = r_0 + z$;
2. $r'_0 \in \bar{S}_{\alpha/2-\beta} \Rightarrow r'_1 = r_1 \wedge r'_0 = r_0 + z$.

Proof. By the definition of Decompose_q , we have $r + z = r_1\alpha + r_0 + z \bmod^+ q$ and $-\alpha/2 - \beta \leq r_1\alpha + r_0 + z < q$. We distinguish the following two cases:

- For $0 \leq r_1\alpha + r_0 + z < q$, we have $r + z \bmod^+ q = r_1\alpha + r_0 + z$. Since $r_0 + z \in \bar{S}_{\alpha/2-\beta}$, we have $r'_1 = r_1$ and $r'_0 = (r + z \bmod^+ q) \bmod^\pm \alpha = r_0 + z$;
- For $-\alpha/2 - \beta \leq r_1\alpha + r_0 + z < 0$, we have $r_1 = 0$ and $r + z \bmod^+ q = q + r_0 + z$. Since $r_0 + z \in \bar{S}_{\alpha/2-\beta}$, we have $(r + z \bmod^+ q) \bmod^\pm \alpha = 1 + r_0 + z$, which means that $r'_1 = r_1 = 0$ and $r'_0 = r_0 + z$.

This completes the proof of the first claim. Similarly, we distinguish the following two cases to prove the second claim:

- For $0 \leq r_1\alpha + r_0 + z < q$, we have $r + z \bmod^+ q = r_1\alpha + r_0 + z$ and $-\alpha/2 - \beta < r_0 + z \leq \alpha/2 + \beta$. Now, we claim that $-\alpha/2 < r_0 + z \leq \alpha/2$, which implies

- that $r'_1 = r_1$ and $r'_0 = (r + z \bmod^+ q) \bmod^\pm \alpha = r_0 + z$. Otherwise, we have $(r + z \bmod^+ q) \bmod^\pm \alpha > \alpha/2 - \beta$ or $(r + z \bmod^+ q) \bmod^\pm \alpha \leq -\alpha/2 + \beta$, which is contradictory to $r'_0 \in \bar{S}_{\alpha/2-\beta}$;
- For $-\alpha/2 - \beta \leq r_1\alpha + r_0 + z < 0$, we have $r_1 = 0$ and $r + z \bmod^+ q = q + r_0 + z$. Now, we claim that $-\alpha/2 \leq r_0 + z < 0$ and $(r + z \bmod^+ q) \bmod^\pm \alpha = 1 + r_0 + z$, which in turn implies that $r'_1 = r_1 = 0$ and $r'_0 = r_0 + z$. Otherwise, we have $r'_0 = (r + z \bmod^+ q) \bmod^\pm \alpha = \alpha + 1 + r_0 + z \geq \alpha/2 - \beta + 1$, which is contradictory to $r'_0 \in \bar{S}_{\alpha/2-\beta}$;

This completes the proof of Lemma 1.

Lemma 2. *Let q and $\alpha > \beta > \alpha/2$ be positive integers such that $q > 2\alpha$, $q \bmod \alpha = 1$ and α is even. Let $r \in \mathbb{Z}_q$ and $(r_1, r_0) = \text{Decompose}_q(r, \alpha)$. Let z be an integer satisfying $r_0 + z \in \bar{S}_\beta$ and $(r'_1, r'_0) = \text{Decompose}_q(r + z, \alpha)$. Then*

1. *if $h = \text{MakeHint}_q(r_0 + z, r_1, \alpha)$ and $v_1 = \text{UseHint}_q(h, r + z, \alpha)$, we have that $v_1 = r_1$ and $r + z - v_1 \cdot \alpha \bmod^\pm q \in \bar{S}_\beta$. Furthermore, if $h = 0$, we have that $|r + z - v_1 \cdot \alpha \bmod^\pm q| \leq \alpha/2$.*
2. *For any $h, h' \in \{0, 1\}$, if $\text{UseHint}_q(h, r, \alpha) = \text{UseHint}_q(h', r, \alpha)$, then $h = h'$.*

Proof. By the definition of Decompose_q , we have $r + z = r_1\alpha + r_0 + z \bmod^+ q$ and $-\beta < r_1\alpha + r_0 + z < q$. If $v_1 = r_1$, then we have $r + z - v_1\alpha = r_0 + z \bmod^+ q$. Since $r_0 + z \in \bar{S}_\beta$, we have $r + z - v_1\alpha \bmod^\pm q = r_0 + z$, which implies that $r + z - v_1\alpha \bmod^\pm q \in \bar{S}_\beta$. Furthermore, if $h = 0$, we have $v_1 = r'_1$, which implies that $|r + z - v_1\alpha \bmod^\pm q| = |r'_0| \leq \alpha/2$. Thus, to prove the first claim it suffices to show that $v_1 = r_1$. By the definition of MakeHint_q ,

- For $h = 0$, we have $-\alpha/2 < r_0 + z \leq \alpha/2$ or $r_1 = 0 \wedge r_0 + z = -\alpha/2$. We distinguish the following two cases:
 - For $0 \leq r_1\alpha + r_0 + z < q$, we have $r + z \bmod^+ q = r_1\alpha + r_0 + z$, which implies that $v_1 = r'_1 = r_1$.
 - For $-\beta < r_1\alpha + r_0 + z < 0$, we have $r_1 = 0$ and $r + z \bmod^+ q = q + r_0 + z$. Thus, $(r + z \bmod^+ q) \bmod^\pm \alpha = 1 + r_0 + z$, which means that $v_1 = r'_1 = r_1 = 0$.
- For $h = 1$, we have $-\beta < r_0 + z \leq -\alpha/2$ or $\alpha/2 < r_0 + z \leq \beta$. Let $k = (q-1)/\alpha$. We distinguish the following two cases:
 - For $0 \leq r_1\alpha + r_0 + z < q$, we have $r + z \bmod^+ q = r_1\alpha + r_0 + z$. This implies that either $r'_1 = r_1 + 1 \bmod^+ k$ and $r'_0 = r_0 + z - \alpha < 0$ for $r_0 + z > \alpha/2$ or $r'_1 = r_1 - 1 \bmod^+ k$ and $r'_0 = r_0 + z + \alpha > 0$ for $r_0 + z \leq -\alpha/2$.
 - For $-\beta < r_1\alpha + r_0 + z < 0$, we have $r_1 = 0$, $r_0 + z < -\alpha/2$ and $r + z \bmod^+ q = q + r_0 + z$. Thus, $r'_1 = r_1 - 1 \bmod^+ k = k - 1$ and $r'_0 = (r + z \bmod^+ q) \bmod^\pm \alpha = 1 + r_0 + z + \alpha > 0$.

Clearly, for all cases, we have $v_1 = r_1$. This completes the proof of the first claim. This second claim directly follows from the definition of UseHint_q .

3.2 Description of the Aigis-Sig⁺ Scheme

Let $n, q, k, \ell, d, \eta_1, \eta_2, \beta_1, \beta_2, \gamma_1, \gamma_2, \gamma_3, \tau, \omega$ be integers such that $\beta_1, \beta_2 \ll \gamma_1, \gamma_2 < \gamma_3$. Let B_τ denote the set consists of elements in R which contains τ coefficients

of $-1, 1$ and others 0 . Let $H : \{0, 1\}^* \rightarrow B_\tau$ be a hash function. Let A_ω be some admissible set $A_\omega \subset R_2^k$. We now give our signature scheme Aigis-Sig⁺, which consists of the following three algorithms (KeyGen, Sign, Verify):

- Aigis-Sig⁺.KeyGen(κ): given the security parameter κ as input, first randomly choose $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$, and $\mathbf{s}_1 \xleftarrow{\$} S_{\eta_1}^\ell, \mathbf{s}_2 \xleftarrow{\$} S_{\eta_2}^k$. Then, compute $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ and $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}_q(\mathbf{t}, d)$ such that $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$. Finally, return the public key $\text{vk} = (\mathbf{A}, \mathbf{t}_1)$ and secret key $\text{sk} = (\mathbf{s}_1, \mathbf{s}_2, \text{vk}, \mathbf{t}_0)$.
- Aigis-Sig⁺.Sign(sk, μ): given the secret key $\text{sk} = (\mathbf{s}_1, \mathbf{s}_2, \text{vk}, \mathbf{t}_0)$ and a message $\mu \in \{0, 1\}^*$ as inputs, perform the following computation:
 1. Randomly choose $\mathbf{y} \xleftarrow{\$} \bar{S}_{\gamma_1}^\ell$;
 2. Compute $\mathbf{w} = \mathbf{A}\mathbf{y}$, $(\mathbf{w}_1, \mathbf{w}_0) = \text{Decompose}_q(\mathbf{w}, 2\gamma_2)$ and $c = H(\mathbf{w}_1 \parallel \mu)$;
 3. Compute $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and $\mathbf{r}_0 = \mathbf{w}_0 - c\mathbf{s}_2$;
 4. If $\mathbf{z} \notin \bar{S}_{\gamma_1 - \beta_1}^\ell$ or $\mathbf{r}_0 \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, restart the computation from step 1, where β_1 (resp., β_2) is a bound such that $\|c\mathbf{s}_1\|_\infty \leq \beta_1$ (resp., $\|c\mathbf{s}_2\|_\infty \leq \beta_2$) holds for all possible $c, \mathbf{s}_1$ (resp., $c, \mathbf{s}_2$);
 5. If $\mathbf{v} = \mathbf{r}_0 + c\mathbf{t}_0 \notin \bar{S}_{\gamma_3}^k$, restart the computation from step 1;
 6. Compute $\mathbf{h} = \text{MakeHint}(\mathbf{v}, \mathbf{w}_1, 2\gamma_2)$. If $\mathbf{h} \notin A_\omega$, restart the computation from step 1;
 7. Finally, output the signature $\sigma = (\mathbf{z}, c, \mathbf{h})$.
- Aigis-Sig⁺.Verify(vk, μ, σ): given the public key $\text{vk} = (\mathbf{A}, \mathbf{t}_1)$, a message $\mu \in \{0, 1\}^*$ and a signature $\sigma = (\mathbf{z}, c, \mathbf{h})$ as inputs, first compute $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$. Then, return 1 if $\mathbf{z} \in \bar{S}_{\gamma_1 - \beta_1}^\ell$, $\mathbf{h} \in A_\omega$, $\mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}'_1 \cdot 2\gamma_2 \in \bar{S}_{\gamma_3}^k$ and $c = H(\mathbf{w}'_1 \parallel \mu)$, otherwise 0.

3.3 Correctness

Let HighBits_q (resp., LowBits_q) be a function that takes the inputs as Decompose_q and returns the first (resp., second) output of Decompose_q . Note that $\mathbf{w} = \mathbf{A}\mathbf{y}$, $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ and $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$, we have that

$$\begin{aligned} \mathbf{w} - c\mathbf{s}_2 &= \mathbf{A}\mathbf{y} - c\mathbf{s}_2 = \mathbf{A}(\mathbf{z} - c\mathbf{s}_1) - c\mathbf{s}_2 = \mathbf{A}\mathbf{z} - c\mathbf{t}, \\ \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0 &= \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d. \end{aligned}$$

Since $\mathbf{r}_0 = \mathbf{w}_0 - c\mathbf{s}_2 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$ and $\|c\mathbf{s}_2\|_\infty \leq \beta_2 \ll \gamma_2$, by Lemma 1 we have

$$\text{HighBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2) = \text{HighBits}_q(\mathbf{w}, 2\gamma_2) = \mathbf{w}_1 \text{ and } \mathbf{r}_0 = \text{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2).$$

Moreover, since $\mathbf{v} = \mathbf{r}_0 + c\mathbf{t}_0$ and $\|\mathbf{v}\|_\infty \leq \gamma_3 < 2\gamma_2$, by Lemma 2 we have

$$\mathbf{w}'_1 = \text{UseHint}_q(\mathbf{h}, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2) = \text{HighBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2) = \mathbf{w}_1$$

and $\mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}'_1 \cdot 2\gamma_2 \in \bar{S}_{\gamma_3}^k$. Hence, the value of $\mathbf{w}'_1$ computed by the verification algorithm is the same as $\mathbf{w}_1$ in the signing algorithm, and thus the verification algorithm will always return 1.

3.4 Number of Repetitions

Since our scheme uses the rejection sampling [28,29] to generate a signature $\sigma = (\mathbf{z}, c, \mathbf{h})$, the efficiency of the signing algorithm is determined by the number of repetitions that will be caused by step 4 of the signing algorithm. We first estimate the probability that $\mathbf{z} \in \bar{S}_{\gamma_1 - \beta_1}^\ell$ holds. Assuming that $\|\mathbf{cs}_1\|_\infty \leq \beta_1$ holds, then for any $\mathbf{z} \in \bar{S}_{\gamma_1 - \beta_1}^\ell$ there exists a $\mathbf{y} \in \bar{S}_{\gamma_1}^\ell$. The size of this range is $2(\gamma_1 - \beta_1)$. Note that each coefficient of $\mathbf{y}$ is chosen randomly from $2\gamma_1$ possible values. That is, for a fixed $\mathbf{cs}_1$, each coefficient of vector $\mathbf{z} = \mathbf{y} + \mathbf{cs}_1$ has $2\gamma_1$ possibilities. Therefore, the probability that $\mathbf{z} \in \bar{S}_{\gamma_1 - \beta_1}^\ell$ is

$$\left(\frac{2(\gamma_1 - \beta_1)}{2\gamma_1}\right)^{n \cdot \ell} = \left(1 - \frac{\beta_1}{\gamma_1}\right)^{n \cdot \ell} \approx e^{-n\ell\beta_1/\gamma_1}.$$

Now, we estimate the probability that

$$\mathbf{r}_0 = \mathbf{w}_0 - \mathbf{cs}_2 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^\ell$$

holds. If we (heuristically) assume that each coefficient of $\mathbf{w}_0$ is uniformly distributed over $\bar{S}_{\gamma_2}$, the probability that $\mathbf{r}_0 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^\ell$ is

$$\left(\frac{2(\gamma_2 - \beta_2 - \eta_1)}{2\gamma_2}\right)^{n \cdot k} \approx e^{-nk(\beta_2 + \eta_1)/\gamma_2}.$$

Finally, under our choice of parameters, the probability that steps 5 and 6 of the signing algorithm will cause a repetition is less than 2%. Thus, the expected number of repetitions is roughly $e^{n(\ell\beta_1/\gamma_1 + k(\beta_2 + \eta_1)/\gamma_2)}$.

4 Concrete Parameters

In Table 1, we select three parameter sets based AMLWE and AMSIS, targeted $\geq 128/256/512$ classical (and $\geq 80/128/256$ quantum) bits security. Let $\mathbf{B}_{64,15,\omega} \subset R_q^k$ be the set of polynomial vectors $\mathbf{f} = (f_0, \dots, f_{k-1}) \in R_2^k$ with binary coefficients $f_i = f_{i,0} + f_{i,1}X + \dots + f_{i,n-1}X^{n-1} \in R_2$ such that (1) the number of ones in each consecutive 64 coefficients of f_i is at most 15, namely, $\|(f_{i,64\ell}, \dots, f_{i,64\ell+63})\|_1 \leq 15$ holds for all $i \in \{0, \dots, k-1\}$ and $\ell \in \{0, \dots, 7\}$; and (2) the total non-zero coefficients in $\mathbf{f}$ is at most ω , namely, $\|(f_{0,0}, f_{0,1}, \dots, f_{k-1,n-1})\|_1 \leq \omega$. We fix $n = 512, q = 4171777$ and $A_\omega = \mathbf{B}_{64,15,\omega}$ for all the three parameter sets.

In Table 2, we present the number of repetitions in the signing algorithm, the sizes of the public key and signature, as well as the estimated security of the above three parameter sets. The second column gives the expected number of parameters estimated by using the formula $e^{n(\ell\beta_1/\gamma_1 + k(\beta_2 + \eta_1)/\gamma_2)}$. The third and fourth columns present the corresponding sizes of public key ($|\mathbf{pk}|$) and signature ($|\sigma|$) in bytes. The fifth column gives the estimated security for the underlying AMLWE problem using the core-SVP model [4], where C/Q denotes the lowest classical/quantum bits. Given the recent progress of dual-hybrid LWE attacks, we believe it is prudent to consider the (refined) MATZOV model in concrete security

estimates as also commented in [22]. For this, we also present the estimated security bits in a (refined) MATZOV model (see Section 8) recently shown to have lowered the concrete security estimates of Kyber below the NIST requirements [13]. The last two columns give the estimated security for the underlying AMSIS problem corresponding to UF-CMA and SUF-CMA using the model for asymmetric MSIS problem in [35] (see Section 8), which in turn is adapted from the model [19].

Table 1: Concrete Parameters for Aigis-Sig⁺

Parameter Sets	$(n, k, \ell, q, d, \omega)$	(η_1, η_2)	τ	(β_1, β_2)	$(\gamma_1, \gamma_2, \gamma_3)$	A_ω
PARAMS I	(512, 2, 2, 4171777, 15, 72)	(1, 5)	24	(24, 120)	$(2^{14}, (q-1)/12, 504000)$	$\mathbf{B}_{64,15,\omega}$
PARAMS II	(512, 4, 4, 4171777, 15, 176)	(1, 1)	44	(44, 44)	$(2^{16}, (q-1)/12, 575000)$	$\mathbf{B}_{64,15,\omega}$
PARAMS III	(512, 8, 7, 4171777, 13, 102)	(1, 1)	118	(118, 118)	$(2^{19}, (q-1)/8, 612000)$	$\mathbf{B}_{64,15,\omega}$

Table 2: Repetitions, Sizes and Estimated Security of Aigis-Sig⁺ (where the numbers in parentheses for SIS Hardness are for SUF-CMA security)

Parameter Sets	Repetitions	$ \mathbf{pk} $	$ \sigma $	AMLWE Hardness (C/Q)			AM SIS Hardness (C/Q)
		(Bytes)	(Bytes)	Primal	Dual	MATZOV [†]	
PARAMS I	6.41	928	2015	134/117	133/117	144/133	139/122 (129/113)
PARAMS II	5.16	1824	4533	287/252	286/251	271/228	285/250 (274/241)
PARAMS III	5.71	4672	9134	561/494	559/492	521/431	585/514 (551/484)

5 Implementation Details

We have implemented our schemes on both x86 and ARM platforms. For each platform, we provide two implementations: a reference C version and a version optimized with vector instructions (AVX2 for x86, NEON for ARM). The following details highlight the implementation choices that most significantly affect performance, with a primary focus on the x86 platform.

Before presenting implementation details, we first outline the conventions of the pseudocode used to describe the algorithms.

Arrays. We write $B[i]$ for the entry of an array B at index i . This notation applies to bit strings, byte strings, and integer arrays. All arrays are indexed from 0. For integers $0 \leq k \leq m$, we write

$$B[k : m] = (B[k], B[k+1], \dots, B[m-1])$$

for the subarray of B consisting of the entries with indices from k to $m-1$. An element $f \in R_q$ is a polynomial of the form

$$f = f_0 + f_1X + \dots + f_{511}X^{511}.$$

In the pseudocode, we represent f by the array

$$(f_0, f_1, \dots, f_{511}) \in \mathbb{Z}_q^{512}, \quad (1)$$

whose entries are the coefficients of f . By slight abuse of notation, we also denote this array by f , and the array entry $f[i]$ denotes the coefficient f_i for $0 \leq i < 512$.

Vectors and Matrices. For a vector $\mathbf{v} \in R_q^\ell$, we denote its entries by

$$\mathbf{v}[0], \mathbf{v}[1], \dots, \mathbf{v}[\ell - 1],$$

where each $\mathbf{v}[i] \in R_q$. For a matrix $\mathbf{A} \in R_q^{k \times \ell}$, we denote its entries by $\mathbf{A}[i, j]$, where $0 \leq i < k$ and $0 \leq j < \ell$. Since each entry of $\mathbf{v}$ and $\mathbf{A}$ is itself a polynomial in R_q , we use nested indexing to refer to polynomial coefficients. Accordingly, we write $\mathbf{v}[i][t]$ for the coefficient of degree t of the polynomial $\mathbf{v}[i]$, and $\mathbf{A}[i, j][t]$ for the coefficient of degree t of the polynomial $\mathbf{A}[i, j]$, where $0 \leq t < 512$.

5.1 Polynomial Multiplication

The polynomial multiplications of **Aigis-Sig**⁺ are implemented by two approaches, NTT and Parallel Small Polynomial Multiplication (PSPM). PSPM is used to compute multiplications of small coefficient polynomials, while other multiplications are computed by NTT.

Let $\zeta = 1327 \in \mathbb{Z}_q$ is a primitive 1024-th root of unity modulo q . Define $\text{BitRev}_9(r)$ to be the integer represented by bit-reversing the unsigned 9-bit value that corresponds to the input integer $r \in \{0, \dots, 511\}$. Specifically, if $r = r_0 + 2r_1 + 4r_2 + \dots + 256r_8$ with $r_i \in \{0, 1\}$, then $\text{BitRev}_9(r) = r_8 + 2r_7 + 4r_6 + \dots + 256r_0$. The NTT and NTT^{-1} algorithms are defined in Algorithms 5 and 6. NTT takes an array $f \in \mathbb{Z}_q^{512}$ (representing an element of R_q) as input and outputs another array $\hat{f} \in \mathbb{Z}_q^{512}$ (representing an element of the ring $\prod_{j=0}^{512} \mathbb{Z}_q$).

If the NTT algorithm is invoked on a vector $\mathbf{s} \in (\mathbb{Z}_q^{512})^k$ (representing an element of R_q^k) with the pseudocode statement

$$\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s}), \quad (2)$$

the result is a vector $\hat{\mathbf{s}} \in (\mathbb{Z}_q^{512})^k$ such that $\hat{\mathbf{s}}[i] = \text{NTT}(\mathbf{s}[i])$ for all $i \in \{0, 1, \dots, k-1\}$. The vector $\hat{\mathbf{s}}$ represents an element of $(\mathbb{Z}_q^{512})^k$. The computation of eq. 2 involves running the NTT algorithm k times.

The PSPM algorithm in Algorithm 7 is a slight modification of the method of [37]. In **Aigis-Sig**⁺, PSPM is used to compute products ca where $c \in B_\tau$ is a sparse ternary polynomial and $a \in S_U$ is a small-coefficient polynomial. In particular, PSPM is used for the products cs_1 , cs_2 , ct_0 , and ct_1 . Concretely, c has exactly τ nonzero coefficients in $\{-1, 1\}$, and the coefficients of a satisfy $a_i \in \{-U, -U+1, \dots, U-1, U\}$ for all $0 \leq i < n$.

Algorithm 5: NTT(f)

Input: Integer array $f \in \mathbb{Z}_q^{512}$.**Output:** Integer array $\hat{f} \in \mathbb{Z}_q^{512}$.

```

1  $\hat{f} := f$ ;
2  $r := 0$ ;
3 for  $len := 256$ ;  $len > 1$ ;  $len := len/2$  do
4   for  $start := 0$ ;  $start < 512$ ;  $start := start + len$  do
5      $zeta := \zeta^{\text{BitRev}_9(r)} \bmod q$ ;
6      $r := r + 1$ ;
7     for  $j := start$ ;  $j < start + len$ ;  $j++$  do
8        $t := zeta \cdot \hat{f}[i + j + len]$ ;
9        $\hat{f}[i + j + len] := \hat{f}[i + j] - t$ ;
10       $\hat{f}[i + j] := \hat{f}[i + j] + t$ ;
11    end
12  end
13 end
14 return  $\hat{f}$ ;
```

Algorithm 6: NTT⁻¹($\hat{f}$)

Input: Integer array $\hat{f} \in \mathbb{Z}_q^{512}$.**Output:** Integer array $f \in \mathbb{Z}_q^{512}$.

```

1  $f := \hat{f}$ ;
2  $r := 511$ ;
3 for  $len := 1$ ;  $len \leq 512$ ;  $len := 2 \cdot len$  do
4   for  $start := 0$ ;  $start < 512$ ;  $start := start + len$  do
5      $zeta := \zeta^{\text{BitRev}_9(r)} \bmod q$ ;
6      $r := r - 1$ ;
7     for  $j := start$ ;  $j < start + len$ ;  $j++$  do
8        $t := f[i + j]$ ;
9        $f[i + j] := t + f[i + j + len]$ ;
10       $f[i + j + len] := zeta \cdot (t - f[i + j + len])$ ;
11    end
12  end
13 end
14  $f := f \cdot 4163629 \bmod q$ ; //  $4163629 \equiv 512^{-1} \bmod q$ 
15 return  $f$ ;
```

Algorithm 7: PSPM(ca)

Input: Polynomial $c \in B_\tau$, and polynomial $a \in S_U$.

Output: Polynomial $u = c \cdot a \in R_q$.

```

1 for  $i : 0 \rightarrow n - 1$  do
2    $w[i] := 0$ ;
3    $v[i] := U + a[i]$ ;
4    $v[n + i] := U - a[i]$ ;
5    $v[2n + i] := U + a[i]$ ;
6 end
7 for  $i : 0 \rightarrow n - 1$  do
8   if  $c[i] \neq 0$  then
9      $t := n - i + c[i](c[i] - 1)n/2$ ;
10    for  $j : 0 \rightarrow n - 1$  do
11       $w[j] := w[j] + v[t + j]$ ;
12    end
13  end
14 end
15 for  $i : 0 \rightarrow n - 1$  do
16    $u[i] := w[i] - \tau U \pmod{q}$ 
17 end
18 return  $u$ 

```

5.2 Initialization for XOF and CRH

In our default implementations, we use SHA-3 family as the hash functions and the extendable-output functions (XOFs). Given an input bit string N and an output length d , and following the notation of $\text{KECCAK}[c](N, d)$ in [32], we recall that SHA3-256 and SHA3-512 are specified as follows:

$$\begin{aligned} \text{SHA3-256}(M) &= \text{KECCAK}[512](M\|01, 256) ; \\ \text{SHA3-512}(M) &= \text{KECCAK}[1024](M\|01, 512) . \end{aligned}$$

Then, We define three extendable-output functions as follows, where XOF-128 and XOF-256 coincide with SHAKE128 and SHAKE256, respectively:

$$\begin{aligned} \text{XOF-128}(M, d) &= \text{KECCAK}[256](M\|1111, d) = \text{SHAKE128}(M, d); \\ \text{XOF-256}(M, d) &= \text{KECCAK}[512](M\|1111, d) = \text{SHAKE256}(M, d); \\ \text{XOF-512}(M, d) &= \text{KECCAK}[1024](M\|1111, d). \end{aligned}$$

The instantiating of XOF_1 , XOF_2 , CRH, and H is listed in Table 3.

XOFs can be used to absorb a sequence of arbitrary-length strings and squeeze a sequence of arbitrary-length strings. These functions perform buffering to handle any incomplete data blocks while absorbing or squeezing. For example, for XOF_2 :

- $ctx \leftarrow \text{XOF}_2.\text{Init}()$
Initializes a hash function context.

Table 3: Instantiating of XOF and CRH

	$\text{XOF}_1(M, d)$	$\text{XOF}_2(M, d)$	$\text{CRH}(M)$	$\text{H}(M)$	κ_1	κ_2
PARAMS I	$\text{XOF-128}(M, d)$	$\text{XOF-256}(M, d)$	$\text{XOF-256}(M, 48)$	$\text{SHA3-256}(M)$	32	48
PARAMS II	$\text{XOF-128}(M, d)$	$\text{XOF-256}(M, d)$	$\text{XOF-256}(M, 48)$	$\text{SHA3-256}(M)$	32	48
PARAMS III	$\text{XOF-128}(M, d)$	$\text{XOF-512}(M, d)$	$\text{XOF-512}(M, 96)$	$\text{SHA3-512}(M)$	64	96

- $\text{ctx} \leftarrow \text{XOF}_2.\text{Absorb}(\text{ctx}, \text{str})$
Injects data to be used in the absorbing phase of XOF_2 and updates context ctx .
- $(\text{ctx}, \text{out}) \leftarrow \text{XOF}_2.\text{Squeeze}(\text{ctx}, r)$
Extracts r output bytes produced during the squeezing phase of XOF_2 and updates context ctx .

5.3 Encode and Decode

In Aigis-Sig⁺, we need to encode the polynomials (or integer arrays) into byte arrays and decode the byte arrays into polynomials (or integer arrays).

The Algorithms from 8 to 11 specify the conversion among bit arrays, byte arrays, and integers. Based on these algorithms, let $\alpha \in \mathbb{Z}$ be the length of the byte string B , we can define the folding functions:

- $\text{IntegerToBytes}(x, \alpha) = \text{BitsToBytes}(\text{IntegerToBits}(x, 8\alpha), 8\alpha)$;
- $\text{BytesToInteger} = \text{BitsToInteger}(\text{BytesToBits}(B, \alpha), 8\alpha)$.

Then, we define the encode function **Encode** to encode polynomials into byte arrays, and the decode function **Decode** as the inverse of the **Encode**. The descriptions of **Encode** and **Decode** are presented in the Algorithm 12 and Algorithm 13.

Additionally, we need the function **HintPack** and its inverse **HintUnpack**. The polynomials in $\mathbf{B}_{64,15,\omega}$ can be transformed into byte strings by function **HintPack**. The length of the result byte string is depended on its inputs. The variable-length output of **HintPack** leads to non-fixed signature lengths. The functions **HintPack** and **HintUnpack** are described in the Algorithm 14 and 15.

Algorithms 16 to 21 present the packing/unpacking functions of public keys, secret keys, and signatures, respectively.

Algorithm 8: BitsToBytes(y, α)

Input: Bit string y of length α .

Output: Byte string z of length $\lceil \alpha/8 \rceil$.

```

1  $z \in \mathcal{B}^{\lceil \alpha/8 \rceil} := 0^{\lceil \alpha/8 \rceil}$ ;
2 for  $i : 0 \rightarrow \alpha - 1$  do
3    $z[\lfloor i/8 \rfloor] := z[\lfloor i/8 \rfloor] + y[i] \cdot 2^{i \bmod 8}$ ;
4 end
5 return  $z$ ;
```

Algorithm 9: BytesToBits(z, α)

Input: Byte string z of length α .**Output:** Bit string y of length 8α .

```

1 for  $i : 0 \rightarrow \alpha - 1$  do
2   for  $j : 0 \rightarrow 7$  do
3      $y[8i + j] := z[i] \bmod 2$ ;
4      $z[i] := \lfloor z[i]/2 \rfloor$ ;
5   end
6 end
7 return  $y$ ;

```

Algorithm 10: BitsToInteger(y, α)

Input: Positive integer α and bit string y of length α .**Output:** Nonnegative integer x .

```

1  $x := 0$ ;
2 for  $i : 1 \rightarrow \alpha$  do
3    $x := 2x + y[\alpha - i]$ ;
4 end
5 return  $x$ ;

```

Algorithm 11: IntegerToBits(x, α)

Input: Nonnegative integer x and positive integer α .**Output:** Bit string y of length α .

```

1 for  $i : 0 \rightarrow \alpha - 1$  do
2    $y[i] := x \bmod 2$ ;
3    $x := \lfloor x/2 \rfloor$ ;
4 end
5 return  $y$ ;

```

Algorithm 12: Encode $_r(f, u, t)$

Input : Integer array $f \in \mathbb{Z}_q^t$ and non-negative integer $u \in \mathbb{Z}$.**Output:** Byte string of length $\lceil rt/8 \rceil$.

```

1 for  $i : 0 \rightarrow t$  do
2   if  $u \neq 0$  then
3      $f[i] := u - f[i]$ ;
4   end
5    $b := b || \text{IntegerToBits}(f[i], r)$ ;
6 end
7 return BitsToBytes( $b$ );

```

Algorithm 13: $\text{Decode}_r(B, u, t)$

Input : Byte string B , non-negative integer $u \in \mathbb{Z}$, and non-negative integer $t \in \mathbb{Z}$.

Output: Integer array $f \in \mathbb{Z}_q^t$.

```

1  $b := \text{BytesToBits}(B, \lceil rt/8 \rceil);$ 
2 for  $i : 0 \rightarrow t$  do
3    $f[i] := \text{BitsToInteger}(b[ir : ir + r], r);$ 
4   if  $u \neq 0$  then
5      $f[i] := u - f[i];$ 
6   end
7 end
8 return  $f;$ 

```

Algorithm 14: $\text{HintPack}(\mathbf{h})$

Input : Polynomial vector $\mathbf{h} \in \mathbf{B}_{64,15,\omega}$

Output: Byte array b and its length len .

```

1  $max := 0;$ 
2  $index := 0;$ 
3  $u := n/64;$ 
4  $y \in \mathbb{Z}_{64}^\omega := (0, \dots, 0);$ 
5  $t \in \mathbb{Z}_{16}^{ku} := (0, \dots, 0);$ 
6 for  $i : 0 \rightarrow k - 1$  do
7   for  $j : 0 \rightarrow u - 1$  do
8      $m := index;$ 
9     for  $r : 0 \rightarrow 63$  do
10      if  $\mathbf{h}[i][64j + r] = 1$  then
11         $y[index] := r;$ 
12         $index := index + 1;$ 
13      end
14    end
15     $t[iu + j] = index - m;$ 
16    if  $t[iu + j] \neq 0$  then
17       $max := iu + j + 1;$ 
18    end
19  end
20 end
21  $b[0] := \text{IntegerToBytes}(max, 1);$ 
22  $b := b \parallel \text{Encode}_4(t, 0, max);$ 
23  $b := b \parallel \text{Encode}_6(y, 0, index);$ 
24  $len := \lceil max/2 \rceil + \lceil 6index/8 \rceil + 1;$ 
25 return  $(b, len);$ 

```

Algorithm 15: HintUnpack(b)

Input : Byte array b .
Output: Polynomial vector $\mathbf{h} \in \mathbf{B}_{64,15,\omega}$

```

1  $index := 0$ ;
2  $u := n/64$ ;
3 for  $i : 0 \rightarrow k - 1$  do
4   for  $j : 0 \rightarrow n - 1$  do
5      $\mathbf{h}[i][j] = 0$ ;
6   end
7 end
8  $max := \text{BytesToInteger}(b[0], 1)$ ;
9  $t := \text{Decode}_4(b[1 : max + 1], 0, max)$ ;
10 for  $i : 0 \rightarrow max - 1$  do
11    $index := index + t[i]$ ;
12 end
13  $y := \text{Decode}_6(b[max + 2 : max + 2 + index], 0, index)$ ;
14  $r := 0$ ;
15 for  $z : 0 \rightarrow max - 1$  do
16    $i = z/u$ ;
17    $j = z \bmod u$ ;
18   for  $m : 0 \rightarrow t[z] - 1$  do
19      $\mathbf{h}[i][64j + y[r]] = 1$ ;
20      $r := r + 1$ ;
21   end
22 end
23 return  $\mathbf{h}$ ;

```

Algorithm 16: pkPack($\rho, \mathbf{t}_1$)

Input : Byte string $\rho \in \mathcal{B}^{\kappa_1}$ and polynomial vector $\mathbf{t}_1 \in R_q^k$.
Output: Public key $pk \in \mathcal{B}^{\kappa_1 + kn(22-d)/8}$.

```

1 for  $i : 0 \rightarrow k - 1$  do
2    $pk := \rho || \text{Encode}_{22-d}(\mathbf{t}_1[i], 0, n)$ ;
3 end
4 return  $pk$ ;

```

Algorithm 17: pkUnpack(pk)

Input : Public key $pk \in \mathcal{B}^{\kappa_1+kn(22-d)/8}$.**Output:** Byte string $\rho \in \mathcal{B}^{\kappa_1}$ and polynomial vector $\mathbf{t}_1 \in R_q^k$.

```

1  $\rho := pk[0 : \kappa_1]$ ;
2  $z := \kappa_1$ ;
3 for  $i : 0 \rightarrow k - 1$  do
4    $\mathbf{t}_1[i] := \text{Decode}_{22-d}(pk[z : z + n(22 - d)/8], 0, n)$ ;
5    $z := z + n(22 - d)/8$ ;
6 end
7 return  $(\rho, \mathbf{t}_1)$ ;
```

Algorithm 18: skPack($\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0$)

Input : Byte string $\rho \in \mathcal{B}^{\kappa_1}$, $K \in \mathcal{B}^{\kappa_1}$, $tr \in \mathcal{B}^{\kappa_2}$ and polynomial vector $\mathbf{s}_1 \in R_q^\ell$, $\mathbf{s}_2 \in R_q^k$, $\mathbf{t}_0 \in R_q^k$.**Output:** Secret key $sk \in \mathcal{B}^{\kappa_1+\kappa_1+\kappa_2+(2n\ell+kn \cdot \text{bitlen}(2\eta_2)+knd)/8}$.

```

1  $sk := \rho \| K \| tr$ ;
2 for  $i : 0 \rightarrow \ell - 1$  do
3    $sk := sk \| \text{Encode}_2(\mathbf{s}_1[i], \eta_1, n)$ ;
4 end
5 for  $i : 0 \rightarrow k - 1$  do
6    $sk := sk \| \text{Encode}_{\text{bitlen}(2\eta_2)}(\mathbf{s}_2[i], \eta_2, n)$ ;
7 end
8 for  $i : 0 \rightarrow k - 1$  do
9    $sk := sk \| \text{Encode}_d(\mathbf{t}_0[i], 2^{d-1}, n)$ ;
10 end
11 return  $sk$ ;
```

Algorithm 19: skUnpack(sk)

Input : Secret key $sk \in \mathcal{B}^{\kappa_1 + \kappa_1 + \kappa_2 + (2n\ell + kn \cdot \text{bitlen}(2\eta_2) + knd)/8}$.

Output: Byte string $\rho \in \mathcal{B}^{\kappa_1}$, $K \in \mathcal{B}^{\kappa_1}$, $tr \in \mathcal{B}^{\kappa_2}$ and polynomial vector $\mathbf{s}_1 \in R_q^\ell, \mathbf{s}_2 \in R_q^k, \mathbf{t}_0 \in R_q^k$.

```

1  $\rho := sk[0 : \kappa_1]$ ;
2  $K := sk[\kappa_1 : 2\kappa_1]$ ;
3  $tr := sk[2\kappa_1 : 2\kappa_1 + \kappa_2]$ ;
4  $z := 2\kappa_1 + \kappa_2$ ;
5 for  $i : 0 \rightarrow \ell - 1$  do
6    $\mathbf{s}_1[i] := \text{Decode}_2(sk[z : z + n/4], \eta_1, n)$ ;
7    $z := z + n/4$ ;
8 end
9 for  $i : 0 \rightarrow k - 1$  do
10   $\mathbf{s}_2[i] := \text{Decode}_{\text{bitlen}(2\eta_2)}(sk[z : z + n \cdot \text{bitlen}(2\eta_2)/8], \eta_2, n)$ ;
11   $z := z + n \cdot \text{bitlen}(2\eta_2)/8$ ;
12 end
13 for  $i : 0 \rightarrow k - 1$  do
14   $\mathbf{t}_0[i] := \text{Decode}_d(sk[z : z + nd/8], 2^{d-1}, n)$ ;
15   $z := z + nd/8$ ;
16 end
17 return  $(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$ ;
```

Algorithm 20: sigPack($\mathbf{z}, \tilde{c}, \mathbf{h}$)

Input : Polynomial $\mathbf{z} \in R_q^\ell$, byte string $\tilde{c} \in \mathcal{B}^{\kappa_1}$, polynomial $\mathbf{h} \in \mathbf{B}_{64,15,\omega}$.

Output: Signature $\sigma \in \mathcal{B}^{n\ell \cdot \text{bitlen}(2\gamma_1)/8 + \kappa_1 + hlen}$, and integer $siglen$.

```

1 for  $i : 0 \rightarrow \ell - 1$  do
2    $\sigma := \sigma \parallel \text{Encode}_{\text{bitlen}(2\gamma_1)}(\mathbf{z}[i], \gamma_1, n)$ ;
3 end
4  $\sigma := \sigma \parallel \tilde{c}$ ;
5  $(b, hlen) := \text{HintPack}(\mathbf{h})$ ;
6  $\sigma := \sigma \parallel b$ ;
7  $siglen := n\ell \cdot \text{bitlen}(2\gamma_1)/8 + \kappa_1 + hlen$ ;
8 return  $\sigma$ ;
```

Algorithm 21: sigUnpack(σ)

Input : Signature $\sigma \in \mathcal{B}^{n\ell \cdot \text{bitlen}(2\gamma_1)/8 + \kappa_1 + h\ell n}$.
Output: Polynomial $\mathbf{z} \in R_q^\ell$, byte string $\tilde{c} \in \mathcal{B}^{\kappa_1}$, polynomial $\mathbf{h} \in \mathbf{B}_{64,15,\omega}$.

```

1  $r := 0$ ;
2 for  $i : 0 \rightarrow \ell - 1$  do
3    $\mathbf{z}[i] := \text{Decode}_{\text{bitlen}(2\gamma_1)}(\sigma[r : r + n \log 2\gamma_1/8], \gamma_1, n)$ ;
4    $r := r + n \log 2\gamma_1/8$ ;
5 end
6  $\tilde{c} := \sigma[r : r + \kappa_1]$ ;
7  $r := r + \kappa_1$ ;
8  $\mathbf{h} := \text{HintUnpack}(\sigma[r : r + h\ell n])$ ;
9 return  $(\mathbf{z}, \tilde{c}, \mathbf{h})$ ;
```

5.4 Sampling Algorithms

Uniformly sample polynomials in R_q . We will use the function `Parse` in Algorithm 22 to do deterministic sample uniform elements in R_q . Since the NTT transformation maps uniformly distributed elements in R_q to uniformly distributed vector in $\mathbb{Z}_q^n$, we can assume that the output of `Parse` is the result of the NTT transformation of a random element. The function `ExpandA` in Algorithm 23 uses `Parse` to generate the polynomial matrix $\mathbf{A}$.

Algorithm 22: Parse($\tilde{\rho}$)

Input : Byte string $\tilde{\rho} \in \mathcal{B}^{\kappa_1+1}$
Output: Polynomial $\hat{a} \in R_q$

```

1  $j := 0$ ;
2  $\text{ctx} := \text{XOF}_1.\text{Init}()$ ;
3  $\text{ctx} := \text{XOF}_1.\text{Absorb}(\text{ctx}, \tilde{\rho})$ ;
4 while  $j < n$  do
5    $(\text{ctx}, z) := \text{XOF}_1.\text{Squeeze}(\text{ctx}, 11)$ ;
6    $t := \text{Decode}_{22}(z, 0, 4)$ ;
7   for  $i : 0 \rightarrow 3$  do
8     if  $t[i] < q$  and  $j < n$  then
9        $\hat{a}[j] := t[i]$ ;
10       $j := j + 1$ ;
11    end
12  end
13 end
14 return  $\hat{a}$ ;
```

Algorithm 23: ExpandA(ρ)

Input : Byte string $\rho \in \mathcal{B}^{\kappa_1}$
Output: Polynomial $\mathbf{A} \in R_q^{k \times l}$
1 **for** $i : 0 \rightarrow k - 1$ **do**
2 **for** $j : 0 \rightarrow \ell - 1$ **do**
3 $b := \text{IntegerToBits}(i, 4) \parallel \text{IntegerToBits}(j, 4);$
4 $\hat{\mathbf{A}}[i, j] := \text{Parse}(\rho \parallel \text{BitsToBytes}(b, 8));$
5 **end**
6 **end**
7 **return** $\mathbf{A};$

Sample from B_h . Let B_h denotes the set consists of elements in R which contains h coefficients of $-1, 1$ and others 0. Obviously, we have $|B_h| = 2^h \cdot \binom{n}{h}$. In Aigis-Sig^+ , a deterministic function can be sampled from B_h for several different h , where $n = 512$. The **SampleInBall** is described in algorithm 24, which uses Fisher-Yates Shuffle algorithm [21] to map uniform byte strings to the elements in B_h .

Algorithm 24: SampleInBall($\tilde{c}$)

Input : Byte string $\tilde{c} \in \mathcal{B}^{\kappa_1}$.
Output: Polynomial $c \in B_h$.
1 $k := 0;$
2 **for** $i : 0 \rightarrow n - 1$ **do**
3 $c[i] := 0;$
4 **end**
5 $ctx := \text{XOF}_2.\text{Init}();$
6 $ctx := \text{XOF}_2.\text{Absorb}(ctx, \tilde{c});$
7 $(ctx, b) := \text{XOF}_1.\text{Squeeze}(ctx, \lceil h/8 \rceil);$
8 $s := \text{BytesToBits}(b, \lceil h/8 \rceil);$
9 $i := n - h;$
10 **while** $(i < n)$ **do**
11 $(ctx, b) := \text{XOF}_1.\text{Squeeze}(ctx, 9);$
12 $t := \text{Decode}_9(b, 0, 8);$
13 **for** $j : 0 \rightarrow 8$ **do**
14 $d := t[j];$
15 **if** $d \leq i$ **and** $i < n$ **then**
16 $c[i] := c[d];$
17 $c[d] := (-1)^{s[i+h-n]};$
18 $i := i + 1;$
19 **end**
20 **end**
21 **end**
22 **return** $c;$

Randomly sample from S_η and $\bar{S}_\eta$. For any positive integer η , we define the set S_η (resp., $\bar{S}_\eta$) consists of polynomials in R_q whose every coefficient belongs to $\{-\eta, -\eta+1, \dots, \eta\}$ (resp., $\{-\eta+1, -\eta, \dots, \eta\}$). In Aigis-Sig⁺, we use the function `ExpandMask` in Algorithm 25 to sample a vector $\mathbf{y} \in R_q^\ell$, which guarantees that every coefficients of $\mathbf{y}$ is uniformly sampled from $\bar{S}_{\gamma_1} = \{-\gamma_1+1, \dots, \gamma_1\}$ as the input is uniformly random.

Algorithm 25: `ExpandMask(K, μ, N)

---`

Input : Byte string $K \in \mathcal{B}^{\kappa_1}$, $\mu \in \mathcal{B}^{\kappa_2}$, and integer N .
Output: Polynomial $\mathbf{y} \in \bar{\mathbf{S}}_{\gamma_1}^\ell$

```

1 for  $i : 0 \rightarrow \ell - 1$  do
2    $b := \text{XOF}_2(K \parallel \mu \parallel \text{IntegerToBytes}(N + i, 2), n \log 2\gamma_1)$ ;
3    $\mathbf{y}[i] := \text{Decode}_{\text{bitlen}(2\gamma_1)}(b, \gamma_1, n)$ ;
4 end
5 return  $\mathbf{y}$ ;
```

Similarly, for any positive integer $\eta \in \mathbb{Z}$, we can sample uniformly random elements from the set $S_\eta = \{-\eta, \dots, \eta\}$ defined by integer η by using rejected sampling. Algorithms 26–28 present the sampling functions to generate secret keys.

Algorithm 26: `RejEta5(ρ)

---`

Input : A seed $\rho \in \mathcal{B}^{\kappa_1+1}$.
Output: A polynomial $a \in R$.

```

1  $j := 0$ ;
2  $\text{ctx} := \text{XOF}_2.\text{Init}()$ ;
3  $\text{ctx} := \text{XOF}_2.\text{Absorb}(\text{ctx}, \rho)$ ;
4 while  $j < n$  do
5    $(\text{ctx}, b) := \text{XOF}_2.\text{Squeeze}(\text{ctx}, 1)$ ;
6    $z = \text{BytesToInteger}(b, 1)$ ;
7    $z_0 := z \bmod 16$ ;
8    $z_1 := \lfloor z/16 \rfloor$ ;
9   if  $z_0 < 11$  then
10     $a[j] := 5 - z_0$ ;
11     $j := j + 1$ ;
12  end
13  if  $z_1 < 11$  and  $j < n$  then
14     $a[j] := 5 - z_1$ ;
15     $j := j + 1$ ;
16  end
17 end
18 return  $a$ ;
```

Algorithm 27: $\text{RejEta1}(\rho)$

Input : A seed $\rho \in \mathcal{B}^{\kappa_1+1}$.
Output: A polynomial $a \in R$.

```

1  $j := 0$ ;
2  $ctx := \text{XOF}_2.\text{Init}()$ ;
3  $ctx := \text{XOF}_2.\text{Absorb}(ctx, \rho)$ ;
4 while  $j < n$  do
5    $(ctx, b) := \text{XOF}_2.\text{Squeeze}(ctx, 1)$ ;
6    $z = \text{BytesToInteger}(b, 1)$ ;
7    $z_0 := z \bmod 4$ ;
8    $z_1 := \lfloor z/4 \rfloor \bmod 4$ ;
9    $z_2 := \lfloor z/16 \rfloor \bmod 4$ ;
10   $z_3 := \lfloor z/64 \rfloor$ ;
11  for  $i : 0 \rightarrow 3$  do
12    if  $z_i \neq 3$  and  $j < n$  then
13       $a[j] := 1 - z_i$ ;
14       $j := j + 1$ ;
15    end
16  end
17 end
18 return  $a$ ;
```

Algorithm 28: $\text{ExpandS}(\rho)$

Input : Byte string $\rho \in \mathcal{B}^{\kappa_1}$.
Output: Polynomials vectors $\mathbf{s}_1 \in R_q^l, \mathbf{s}_2 \in R_q^k$.

```

1 for  $r : 0 \rightarrow \ell - 1$  do
2    $\mathbf{s}_1[r] := \text{RejEta1}(\rho \parallel \text{IntegerToBytes}(r, 1))$ ;
3 end
4 if  $\eta_2 = 1$  then
5   for  $r : 0 \rightarrow k - 1$  do
6      $\mathbf{s}_2[r] := \text{RejEta1}(\rho \parallel \text{IntegerToBytes}(r + \ell, 1))$ ;
7   end
8 end
9 else if  $\eta_2 = 5$  then
10  for  $r : 0 \rightarrow k - 1$  do
11     $\mathbf{s}_2[r] := \text{RejEta5}(\rho \parallel \text{IntegerToBytes}(r + \ell, 1))$ ;
12  end
13 end
14 return  $(\mathbf{s}_1, \mathbf{s}_2)$ ;
```

5.5 The Aigis-Sig⁺ Scheme

Now, we are ready to give the pseudocode of the key generation, signing and verification algorithms of Aigis-Sig⁺.

Algorithm 29: Aigis-Sig⁺.KeyGen()

Output: Public key $pk \in \mathcal{B}^{\kappa_1 + kn(22-d)/8}$

Output: Secret key $sk \in \mathcal{B}^{2\kappa_1 + \kappa_2 + (2n\ell + kn \cdot \text{bitlen}(2\eta_2) + knd)/8}$

```

1  $\xi \xleftarrow{\$} \mathcal{B}^{\kappa_1}$ ;
2  $(rnd, \rho, K) \in \mathcal{B}^{\kappa_1} \times \mathcal{B}^{\kappa_1} \times \mathcal{B}^{\kappa_1} := \text{XOF}_2(\xi, 3\kappa_1)$ ;
3  $(s_1, s_2) \in R_q^\ell \times R_q^k := \text{ExpandS}(rnd)$ ;
4  $\hat{A} \in R_q^{k \times \ell} := \text{ExpandA}(\rho)$ ;
5  $\hat{s}_1 := \text{NTT}(s_1)$ ;
6  $\mathbf{t} := \text{NTT}^{-1}(\hat{A} \circ \hat{s}_1) + s_2$ ; /*  $\mathbf{t} := \mathbf{A}s_1 + s_2$  */
7  $(\mathbf{t}_1, \mathbf{t}_0) := \text{Power2Round}_q(\mathbf{t}, d)$ ;
8  $pk := \text{pkPack}(\rho, \mathbf{t}_1)$ ;
9  $tr \in \mathcal{B}^{\kappa_2} := \text{CRH}(pk)$ ;
10  $sk := \text{skPack}(\rho, K, tr, s_1, s_2, \mathbf{t}_0)$ ;
11 return  $(pk, sk)$ ;
```

Algorithm 30: Aigis-Sig⁺.Sign(sk, M)

Input : Secret key $sk \in \mathcal{B}^{2\kappa_1+\kappa_2+(2n\ell+kn \cdot \text{bitlen}(2\eta_2)+knd)/8}$ and Message $M \in \mathcal{B}^*$.

Output: Signature $\sigma \in \mathcal{B}^{n\ell \cdot \text{bitlen}(2\gamma_1)/8+\kappa_1+hlen}$ and its length $siglen \in \mathbb{Z}$.

```

1   $(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0) := \text{skUnpack}(sk);$ 
2   $\hat{\mathbf{A}} \in R_q^{k \times \ell} := \text{ExpandA}(\rho);$ 
3   $\mu \in \mathcal{B}^{\kappa_2} := \text{CRH}(tr \| M);$ 
4   $N := 0;$ 
5   $(\mathbf{z}, \mathbf{h}) := \perp;$ 
6  while  $(\mathbf{z}, \mathbf{h}) = \perp$  do                                     /* do RejSampling */
7       $\mathbf{y} \in \bar{S}_{\gamma_1}^\ell := \text{ExpandMask}(K, \mu, N);$ 
8       $N := N + \ell;$ 
9       $\mathbf{w} := \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{y}));$                                /*  $\mathbf{w} := \mathbf{A}\mathbf{y}$  */
10      $(\mathbf{w}_1, \mathbf{w}_0) := \text{Decompose}_q(\mathbf{w}, 2\gamma_2);$ 
11      $\tilde{c} := \text{H}(\mu, \mathbf{w}_1);$ 
12      $c \in B_\tau := \text{SampleInBall}(\tilde{c});$ 
13      $\mathbf{z} := \mathbf{y} + c\mathbf{s}_1;$ 
14      $\mathbf{r}_0 := \mathbf{w}_0 - c\mathbf{s}_2;$ 
15     if  $\mathbf{z} \notin \bar{S}_{\gamma_1-\beta_1}^\ell$  or  $\mathbf{r}_0 \notin \bar{S}_{\gamma_2-\beta_2-\eta_1}^k$  then
16          $(\mathbf{z}, \mathbf{h}) := \perp;$ 
17     else
18          $\mathbf{v} := \mathbf{r}_0 + c\mathbf{t}_0;$ 
19          $\mathbf{h} := \text{MakeHint}_q(\mathbf{v}, \mathbf{w}_1, 2\gamma_2);$ 
20         if  $\mathbf{v} \notin \bar{S}_{\gamma_3}^k$  or  $\mathbf{h} \notin \mathbf{B}_{64,15,\omega}$  then
21              $(\mathbf{z}, \mathbf{h}) := \perp;$ 
22         end
23     end
24 end
25  $(\sigma, siglen) := \text{sigPack}(\mathbf{z}, \tilde{c}, \mathbf{h});$ 
26 return  $(\sigma, siglen);$ 

```

Algorithm 31: Aigis-Sig⁺.Verify(pk, M, σ)

Input : Public key $pk \in \mathcal{B}^{\kappa_1 + kn(22-d)/8}$ and Message $M \in \mathcal{B}^*$.
Input : Signature $\sigma \in \mathcal{B}^{n\ell \cdot \text{bitlen}(2\gamma_1)/8 + \kappa_1 + hlen}$.
Output: $b \in \{0, 1\}$

```

1  $(\rho, \mathbf{t}_1) := \text{pkUnpack}(pk);$ 
2  $(\mathbf{z}, \tilde{c}, \mathbf{h}) := \text{sigUnpack}(\sigma);$ 
3  $c \in B_\tau := \text{SampleInBall}(\tilde{c});$ 
4  $\hat{\mathbf{A}} \in R_q^{k \times \ell} := \text{ExpandA}(\rho);$ 
5  $\mu \in \mathcal{B}^{\kappa_2} := \text{CRH}(\text{CRH}(pk) \| M);$ 
6  $\mathbf{u} := \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{z})) - c\mathbf{t}_1 \cdot 2^d; \quad /* \mathbf{u} := \mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d */$ 
7  $\mathbf{w}'_1 := \text{UseHints}_q(\mathbf{h}, \mathbf{u}, 2\gamma_2);$ 
8  $\tilde{c}' := \text{H}(\mu, \mathbf{w}'_1);$ 
9 if  $\mathbf{z} \in \tilde{S}_{\gamma_1 - \beta_1}^\ell$  and  $\tilde{c} = \tilde{c}'$  and  $\mathbf{u} - \mathbf{w}'_1 \cdot 2\gamma_2 \in \tilde{S}_{\gamma_3}^k$  and  $\mathbf{h} \in \mathbf{B}_{r, \tau, \omega}$  then
10    $b := 1; \quad /* \text{The Signature is Valid} */$ 
11 else
12    $b := 0; \quad /* \text{The Signature is Invalid} */$ 
13 end
14 return  $b;$ 

```

6 Performance and Comparison

We evaluate the performance of Aigis-Sig⁺ and compare it with Dilithium on the following two platforms:

x86 AMD Ryzen 9 7950X3D processor based on the Zen 4 microarchitecture. The CPU frequency was fixed at 4.20 GHz, with Core Performance Boost and Simultaneous Multithreading (SMT) disabled. The RAM is 64 GiB. All benchmarks were executed in a single-threaded configuration. The software environment consisted of Ubuntu 26.04 LTS and GCC 15.2.0. The implementations were compiled using the flags: `-O3 -fomit-frame-pointer -march=native`.

ARM HUAWEI Kirin 9000C Processor based on the AArch64 architecture. The CPU frequency was fixed at 2.487 GHz. The RAM is 8 GiB. The software environment consisted of Kylin Linux Desktop V10 SP1 and GCC 9.3.0. The implementations were compiled using the flags: `-O3 -fomit-frame-pointer -march=native`.

Section 6.1 reports the x86 results, while Section 6.2 reports the ARM results. The performance is measured in CPU cycles and reported as the average over 30,000 runs of each function, except that the median signing time is also reported because signing uses rejection sampling. The performance of Dilithium is profiled using the NIST round-3 submission package.

6.1 Performance on x86

Table 4 compares Aigis-Sig⁺ and Dilithium under both REF and AVX2 implementations. In the REF setting, Aigis-Sig⁺ substantially reduces the CPU cycles for

Table 4: The Performance of Aigis-Sig⁺ (CPU cycles)

Scheme	Parameter Set	REF				AVX2			
		KeyGen (average)	Sign (median)	Sign (average)	Verify (average)	KeyGen (average)	Sign (median)	Sign (average)	Verify (average)
Aigis-Sig ⁺	PARAMS I	190,197	700,098	890,700	172,117	44,824	169,344	216,463	45,660
	PARAMS II	477,200	1,339,842	1,691,626	468,837	102,497	269,304	333,016	117,584
	PARAMS III	1,343,786	3,689,133	4,949,919	1,367,985	315,544	880,488	1,139,947	400,649
Dilithium	Dilithium2	262,797	951,341	1,252,005	289,759	70,599	159,305	195,695	76,773
	Dilithium3	476,055	1,530,647	1,918,884	453,735	121,478	264,683	323,294	119,440
	Dilithium5	724,275	2,046,575	2,496,284	754,167	189,212	334,319	383,928	187,429

key generation, signing, and verification compared with Dilithium at the 128- and 256-bit security levels. Although Aigis-Sig⁺ does not have a parameter set directly matching Dilithium3, its PARAMS II configuration still outperforms Dilithium3 in every measured operation under both implementations.

Figure 1 presents the computational performance on x86-64, with security levels (CoreSVP, MATZOV) on the x-axis, grouped by security strength. Figure 2 shows the total costs when transmission overhead (estimated at 2000 cycles/byte) is included. Notably, while the Dilithium parameter sets fall short of 128- and 256-bit classical security levels, Aigis-Sig⁺ not only meets or exceeds these targets but also delivers substantially faster execution and more compact public-key and signature sizes.

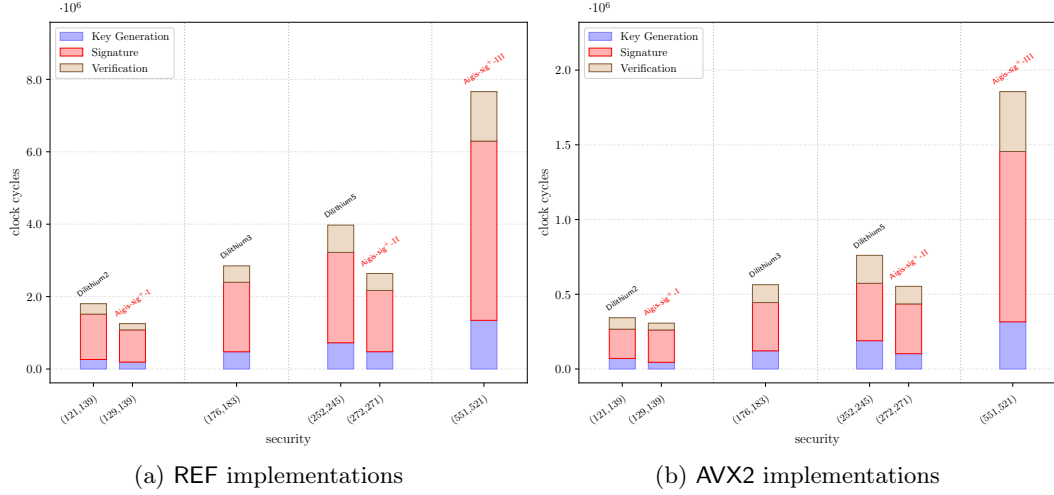


Fig. 1: Benchmarks on x86 platform

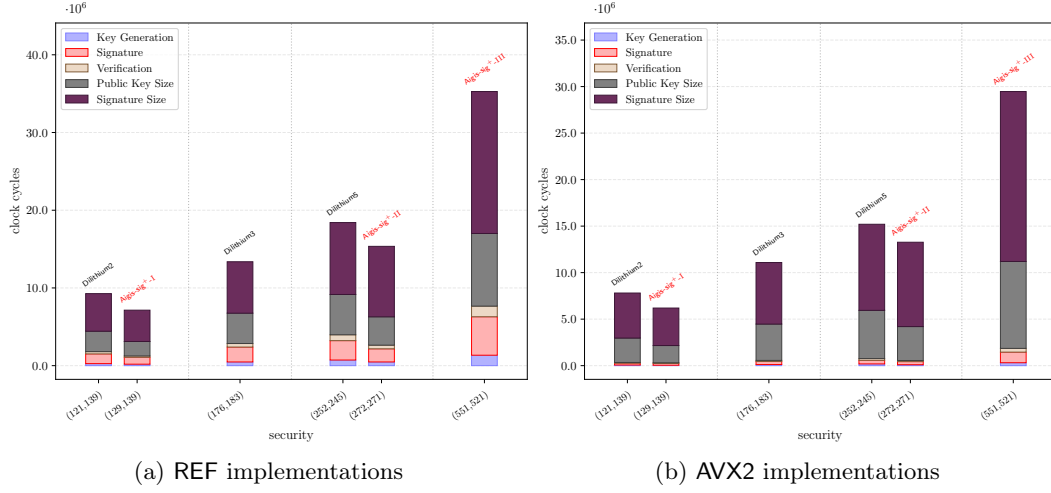


Fig. 2: Benchmarks on x86 platform with 2000 cycles/byte transmission costs

6.2 Performance on ARM

Table 5: The Performance of Aigis-Sig⁺ on ARM (CPU cycles)

Scheme	Parameter Set	KeyGen (average)	Sign (median)	Sign (average)	Verify (average)
AArch64					
Aigis-Sig ⁺	PARAMS I	139,443	531,903	674,685	136,676
	PARAMS II	353,781	1,036,986	1,291,619	377,972
	PARAMS III	1,007,961	3,017,467	3,994,920	1,167,113
Dilithium	Dilithium2	195,001	662,143	875,570	213,049
	Dilithium3	363,340	1,182,398	1,449,427	340,371
	Dilithium5	532,358	1,410,880	1,730,766	552,107
NEON					
Aigis-Sig ⁺	PARAMS I	131,554	505,252	642,773	123,509
	PARAMS II	333,158	848,681	1,049,918	349,715
	PARAMS III	996,973	2,612,753	3,351,211	1,089,873

On the AArch64 platform, Aigis-Sig⁺ consistently outperforms Dilithium, requiring significantly fewer CPU cycles for key generation, signing, and verification—all while providing a stronger concrete security margin. Relative to the portable AArch64 baseline, the NEON-optimized implementation further reduces cycle counts across all operations, with the most notable gains in signing and verification.

Figure 3 presents the computational performance of Aigis-Sig⁺ and Dilithium on the ARM platform, with security levels (CoreSVP, MATZOV) on the x-axis as in the x86 comparison. Figure 4 shows the total costs when transmission overhead

(estimated at 2000 cycles/byte) is included. These results confirm that Aigis-Sig⁺ maintains its performance advantage on ARM.

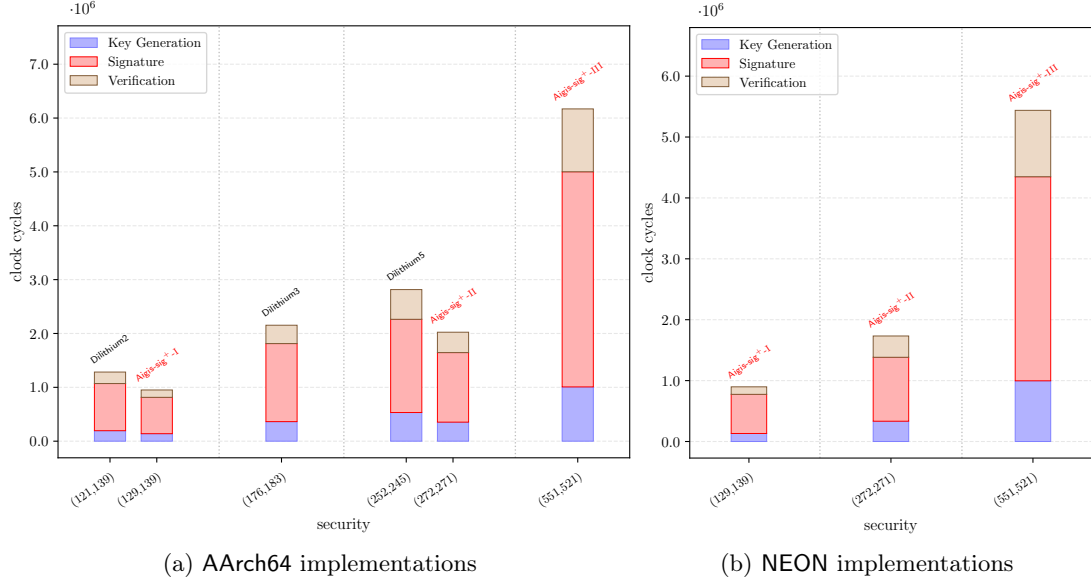


Fig. 3: Benchmarks on ARM platform

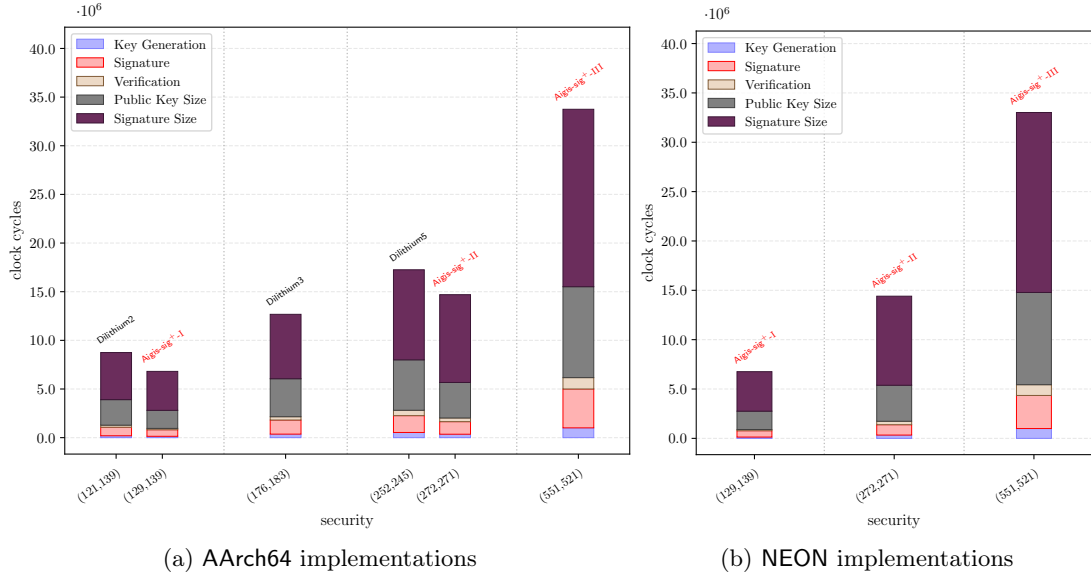


Fig. 4: Benchmarks on ARM platform with 2000 cycles/byte transmission costs

7 Provable Security

In this section, based on the AMLWE, AMSIS, and SelfTargetAMISIS assumptions, we prove the SUF-CMA security of Aigis-Sig⁺ in the ROM and QROM. The UF-CMA security is directly implied by the SUF-CMA security.

7.1 SUF-CMA Security Proof of Aigis-Sig⁺ in the ROM

Theorem 1. *Let $\bar{p}$ and Γ_p be such that $0 < \bar{p} < 1$ and $p_{sk} \leq \bar{p}$ for all $sk \in \Gamma_p$, where p_{sk} is defined in Eq. (7). Let $\mathcal{A}$ be a SUF-CMA adversary against Aigis-Sig⁺, making q_S classical queries to the signing oracle and q_H classical queries to the random oracle H . Then we can construct two adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ against the AMLWE assumption, an adversary $\mathcal{C}$ against the SelfTargetAMISIS assumption, and an adversary $\mathcal{D}$ against the AMSIS assumption such that*

$$\begin{aligned} \text{Adv}_{\text{Aigis-Sig}^+}^{\text{SUF-CMA}}(\mathcal{A}) &\leq \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1) + \text{Adv}_{n,q,k,\ell,\gamma_3+\tau \cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMISIS}}(\mathcal{C}) \\ &\quad + \text{Adv}_{n,q,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMISIS}}(\mathcal{D}) + \frac{4q_S}{(1-\bar{p})^2} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2) \\ &\quad + \frac{q_S(2q_H + q_S)}{(1-\bar{p})^2} \cdot \epsilon + \frac{3q_S}{1-\bar{p}} \cdot \Pr[sk \notin \Gamma_p]. \end{aligned}$$

Here $\mathcal{C}$ makes at most q_H classical queries to its oracle, ϵ is defined in Eq. (7), the probability $\Pr[sk \notin \Gamma_p]$ is taken over $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ (Fig. 10), and the running times of $\mathcal{B}_1$, $\mathcal{B}_2$, $\mathcal{C}$, and $\mathcal{D}$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A. $\square$

7.2 SUF-CMA Security Proof of Aigis-Sig⁺ in the QROM

Theorem 2. *Let $\bar{p}$ and Γ_p be such that $0 < \bar{p} < 1$ and $p_{sk} \leq \bar{p}$ for all $sk \in \Gamma_p$, where p_{sk} is defined in Eq. (7). Let $\mathcal{A}$ be a SUF-CMA adversary against Aigis-Sig⁺, making q_S classical queries to the signing oracle and q_H quantum queries to the random oracle H . Then we can construct two adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ against the AMLWE assumption, an adversary $\mathcal{C}$ against the SelfTargetAMISIS assumption, and an adversary $\mathcal{D}$ against the AMSIS assumption such that*

$$\begin{aligned} \text{Adv}_{\text{Aigis-Sig}^+}^{\text{SUF-CMA}}(\mathcal{A}) &\leq \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1) + \text{Adv}_{n,q,k,\ell,\gamma_3+\tau \cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMISIS}}(\mathcal{C}) \\ &\quad + \text{Adv}_{n,q,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMISIS}}(\mathcal{D}) \\ &\quad + \frac{32q_S(q_H + 2)}{(1-\bar{p})^2} \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2)} \\ &\quad + \frac{2q_S(4q_H + q_S + 5)}{(1-\bar{p})^2} \cdot \sqrt{\epsilon} + \frac{4q_S}{1-\bar{p}} \cdot \Pr[sk \notin \Gamma_p]. \end{aligned}$$

Here $\mathcal{C}$ makes at most q_H quantum queries to its oracle, ϵ is defined in Eq. (7), the probability $\Pr[sk \notin \Gamma_p]$ is taken over $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ (Fig. 10), and the running times of $\mathcal{B}_1$, $\mathcal{B}_2$, $\mathcal{C}$, and $\mathcal{D}$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix B. $\square$

7.3 Tightness of the Security Bound

In this section, we analyze the tightness of the security bounds for Theorems 1 and 2. We first estimate the sizes of $\bar{p}$, $\Pr[sk \notin \Gamma_p]$, ϵ , and $\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2)$ under our parameter setting.

- $\bar{p}$ and $\Pr[sk \notin \Gamma_p]$: Note that the parameter ϵ defined in Eq. (7) is independent of the public and secret keys. Therefore, unlike [8], we do not need to additionally rely on Γ_p to control the size of ϵ . This enables us to give a concise size estimation for $\bar{p}$ and $\Pr[sk \notin \Gamma_p]$. Specifically, we directly set Γ_p as the universal set so that $\Pr[sk \notin \Gamma_p] = 0$. We then follow the heuristic from [26]—as we do in Section 3.4—to obtain an expected upper bound $\bar{p} := 1 - e^{-n\ell\beta_1/\gamma_1} + 1 - e^{-nk(\beta_2+\eta_1)/\gamma_2}$ for all p_{sk} , where $\bar{p}$ is a reasonably sized constant under our parameter setting.
- ϵ : The parameter ϵ defined in Eq. (7) is independent of the public and secret keys, and under our parameter setting, it can be derived that $\epsilon \leq 2^{-2641}$.
- $\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2)$: In our parameter setting, γ_1 is much larger than η_1 with $\gamma_1 \geq 2^{14}$ and $\eta_1 = 1$, which makes the corresponding AMLWE problem extremely hard. Therefore, we conclude that this advantage is negligible and much smaller than $\text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1)$, $\text{Adv}_{n,q,k,k,\ell,\gamma_3+\tau\cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMSIS}}(\mathcal{C})$, and $\text{Adv}_{n,q,k,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMSIS}}(\mathcal{D})$.

Then, we consider the security loss of our security bounds related to $\Pr[sk \notin \Gamma_p]$, ϵ , and $\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2)$, which can be described as follows:

$$\begin{aligned} \mathbf{L}_{\text{ROM}} &:= q_S \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2) + q_S q_H \cdot \epsilon + q_S \cdot \Pr[sk \notin \Gamma_p], \\ \mathbf{L}_{\text{QROM}} &:= q_S q_H \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2)} + q_S q_H \cdot \sqrt{\epsilon} + q_S \cdot \Pr[sk \notin \Gamma_p]. \end{aligned}$$

Here $\mathbf{L}_{\text{ROM}}$ (resp. $\mathbf{L}_{\text{QROM}}$) denotes the security loss in the ROM (resp. QROM), and we omit constant terms for simplicity. Since all terms constituting $\mathbf{L}_{\text{QROM}}$, apart from $q_S \cdot \Pr[sk \notin \Gamma_p]$, are significantly larger than their counterparts in $\mathbf{L}_{\text{ROM}}$, we focus on analyzing the size of $\mathbf{L}_{\text{QROM}}$ in what follows. In this document, we assume that the adversary's query times q_S and q_H are both at most 2^{80} . Under this assumption and based on the estimations given earlier, the term $q_S q_H \cdot \sqrt{\epsilon} + q_S \cdot \Pr[sk \notin \Gamma_p]$ remains negligible and at most 2^{-1160} . As for

$$q_S q_H \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2)},$$

since there is a substantial gap between the sizes of γ_1 and η_1 , we also believe that it remains much smaller than $\text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1)$ even in the presence of an $q_S q_H$ factor and a square-root blow-up. This implies that the dominant part of our security bounds (in both the ROM and QROM) consists of the advantages $\text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1)$, $\text{Adv}_{n,q,k,k,\ell,\gamma_3+\tau\cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMSIS}}(\mathcal{C})$, and $\text{Adv}_{n,q,k,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMSIS}}(\mathcal{D})$, and can thus be rewritten as

$$\begin{aligned} \text{Adv}_{\text{Aigis-Sig}^+, \mathcal{A}}^{\text{SUF-CMA}} &\leq 2 \cdot \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1) + \text{Adv}_{n,q,k,k,\ell,\gamma_3+\tau\cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMSIS}}(\mathcal{C}) \\ &\quad + \text{Adv}_{n,q,k,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMSIS}}(\mathcal{D}) + 2^{-1160}. \end{aligned}$$

8 Known Attacks

As shown in Sec. 7, Aigis-Sig⁺ is provably secure under the hardness of the AMLWE and AMSIS problems in both the classical and quantum random oracle models. Essentially, the AMLWE problem is employed to guarantee the security of the public key, while the AMSIS problem is used to ensure the unforgeability of the signature. Since the best known attacks against the underlying AMLWE and AMSIS problems do not make use of the structure in the corresponding lattices, we therefore analyze the concrete security of the public keys and the signatures by considering the best known attacks against ALWE and ASIS problems.

8.1 Attacks against ALWE

Solvers for ALWE mainly include primal attacks, dual attacks (against the underlying lattice problems) and direct solving algorithms such as BKW and Arora-Ge [3]. BKW and Arora-Ge attacks need sub-exponentially (or even exponentially) many samples, and thus they are not relevant to the public-key cryptography scenario where only a restricted amount of samples are available. Therefore, for analyzing and evaluating practical lattice-based crypto-systems, we typically consider primal attacks and dual attacks.

Since any bounded centrally symmetric distribution can be regarded as subgaussian for a certain parameter, for simplicity and without loss of generality, we consider the case that secret vector and error vector in $\text{ALWE}_{n,q,m,\alpha_1,\alpha_2}$ are sampled from subgaussians with parameters α_1 and α_2 respectively. Formally, the problem is to recover $\mathbf{s}$ from samples

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m ,$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. In the following, we first consider the primal/dual attacks adapted to the asymmetric setting in [35], and then consider a variant of the MATZOV attack in [13] also adapted to the asymmetric setting.

8.1.1 Primal Attack. The basic intuition behind primal attack is to convert ALWE problem to the corresponding bounded distance decoding (BDD) problem or shortest vector problem by embedding. First, define a m -dimensional lattice

$$\Lambda = \{\mathbf{y} \in \mathbb{Z}^m \mid \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} ,$$

For $m > n$ the $m \times n$ matrix $\mathbf{A}$ has full rank with high probability. Assume that the first n rows of $\mathbf{A}$ are independent (otherwise swap the rows to make it), and denote by $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ the submatrix by keeping the first n rows of $\mathbf{A}$, i.e., $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \end{pmatrix}$. Denote $\mathbf{A}_1$'s inverse by $\mathbf{A}_1^{-1} \in \mathbb{Z}_q^{n \times n}$, and let $\mathbf{A}' = \begin{pmatrix} \mathbf{I}_n \\ \mathbf{A}_2 \mathbf{A}_1^{-1} \end{pmatrix}$. Then, we have

$$\begin{aligned} \Lambda &= \{\mathbf{y} \mid \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \\ &= \{\mathbf{y} \mid \mathbf{y} = \mathbf{A}'\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \subset \mathbb{Z}^m , \end{aligned}$$

and the columns of the following matrix

$$\mathbf{B} = \begin{pmatrix} \mathbf{I}_n & 0 \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q \mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitutes a basis of Λ . It is easy to see $\det(\Lambda) = q^{m-n}$. We have by $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$ that the distance between vector $\mathbf{b}$ and lattice Λ is $\|\mathbf{e}\|$. If we could find the closest lattice point $\mathbf{u} \in \Lambda$ to $\mathbf{b}$, then we have $\mathbf{e} = \mathbf{b} - \mathbf{u}$ with high probability. In other words, the problem can be reduced to bounded distance decoding (BDD) problem on Λ , and then one applies the nearest planes algorithm [27] to solve. Furthermore, using Kannan's embedding [25,1] one can reduce BDD to unique shortest vector problem (uSVP). In detail, consider lattice Λ' spanned by the column vectors of the following matrix:

$$\mathbf{B}' = \begin{pmatrix} \mathbf{I}_n & 0 & \mathbf{b} \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q \mathbf{I}_{m-n} & t \\ 0 & 0 & t \end{pmatrix} \in \mathbb{Z}^{(m+1) \times (m+1)},$$

where $t \in \mathbb{Z}$ is an adjustable parameter. Theoretically, the algorithm works best when $t = \|\mathbf{e}\|$. However, empirical experiments suggest that best performance is achieved when $t = 1$, which is the typical value we choose. In this case, we have that $\mathbf{v} = (\mathbf{e}^T, 1)^T \in \mathbb{Z}^{m+1}$ is the unique shortest vector in Λ' with high probability. To summarize, we work on uSVP problem in Λ' to find out the error vector $\mathbf{e} \in \mathbb{Z}^m$, which is in turn used to solve the system of linear congruences to recover $\mathbf{s} \in \mathbb{Z}_q^n$, i.e., $\mathbf{A}\mathbf{s} = \mathbf{b} - \mathbf{e}$. Besides, since every element in vector $\mathbf{e}$ is sampled from subgaussian χ_{α_2} , we have $\|\mathbf{v}\| \approx \|\mathbf{e}\| \approx \alpha_2 \sqrt{m}$, namely, $\mathbf{v}$ is of small norm.

One can also make use of the fact that $\mathbf{s} \xleftarrow{\$} \chi_{\alpha_1}^n$ has small norm in practical lattice-based cryptosystems by considering a variant of the LWE embedding [6,4]. Formally, let $c = \alpha_2/\alpha_1$ and consider lattice

$$\Lambda = \left\{ \mathbf{v} = \begin{pmatrix} c\mathbf{x} \\ \mathbf{y} \\ \alpha_2 z \end{pmatrix} \in \mathbb{R}^{n+m+1} \left| (\mathbf{A}\|\mathbf{I}_m\| - \mathbf{b})\mathbf{u} = 0 \bmod q, \mathbf{u} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ z \end{pmatrix} \in \mathbb{Z}^{n+m+1} \right. \right\},$$

Then the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_n & 0 & 0 \\ -\mathbf{A} & q\mathbf{I}_m & \mathbf{b} \\ 0 & 0 & \alpha_2 \end{pmatrix} \in \mathbb{R}^{(n+m+1) \times (n+m+1)},$$

constitutes a basis of Λ . We observe that Λ is of dimension $m+n+1$, $\det(\Lambda) = \alpha_2 c^n q^m$, and that $\mathbf{v} = (c\mathbf{s}^T, \mathbf{e}^T, \alpha_2)^T \in \mathbb{R}^{m+n+1}$ is a short vector in Λ (and $\|\mathbf{v}\| \approx \alpha_2 \sqrt{n+m+1}$). Therefore, we recover $\mathbf{s}$ by solving uSVP on lattice Λ .

The Cost of Primal Attack. As shown above, the computational costs of primal attack is mainly dominated by the cost of solving uSVP problem. The best known SVP solver is the BKZ- b lattice basis reduction and its variants. Given a d -dimensional basis as input, the algorithm translates the lattice basis reduction

problem to the SVP problem on b -dimensional sublattice ($b < d$). BKZ- b repeats the SVP solving procedure $O(d)$ times (the actual number may depend on the specific algorithm used) on different b -dimensional sublattices to reduce the norm of the basis (of the original d -dimensional basis). Therefore, the cost of BKZ- b basis reduction is mainly decided by SVP solving algorithm for b -dimensional lattice.

Assume that the reduced basis obtained from running BKZ- b on d -dimensional lattice basis $\mathbf{B} \in \mathbb{Z}^{d \times d}$ satisfies the Geometry Series Assumption (GSA), which is the best scenario for adversary. Let $\hat{\mathbf{B}} = (\hat{\mathbf{b}}_1, \dots, \hat{\mathbf{b}}_d)$ be a reduced basis, then we have

$$\|\hat{\mathbf{b}}_1\| = \delta^d \det(\Lambda)^{1/d}, \quad \|\hat{\mathbf{b}}_i^*\| = \delta^{-2d(i-1)/(d-1)} \|\hat{\mathbf{b}}_1\|,$$

where $\delta = ((\pi b)^{1/b} \cdot b / 2\pi e)^{\frac{1}{2(b-1)}} [15]$, $\hat{\mathbf{B}}^* = (\hat{\mathbf{b}}_1^*, \dots, \hat{\mathbf{b}}_d^*)$ is the orthogonalized matrix of $\hat{\mathbf{B}}$ (i.e., $\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_1^*$). Specifically, the literatures [4,1] show that when the norm of the projection of the unique shortest vector $\mathbf{v}$ into the space spanned by the last b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ is less than $\|\hat{\mathbf{b}}_{d-b+1}^*\|$, then the BKZ- b lattice basis reduction recovers $\mathbf{v}$.

The norm of $\mathbf{v}$'s projection into the space spanned by b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ approximately equals $\ell \sqrt{b/d}$, where $\ell = \|\mathbf{v}\|$. That is, we assume every component of vector $\mathbf{v}$'s projection is of roughly the same size. In this case, the computational cost of primal attack is mainly decided by the minimal value of b that satisfies the constraint

$$\ell \sqrt{b/d} \leq \delta^{(-d^2+2db-d)/(d-1)} \det(\Lambda)^{1/d}. \quad (3)$$

Following [4], our estimation of the complexity of solving uSVP on d -dimensional lattice is based on that of solving SVP on b -dimensional lattice, where b satisfy inequality (3) (namely, the Core-SVP model). Furthermore, we use $\text{cost}_b = 2^{0.292b}$ [4,12] and $\text{cost}_b = 2^{0.257b}$ [14] to estimate the complexities of classical and quantum algorithms (for solving SVP on b -dimensional lattice) respectively.

8.1.2 Dual Attack. Dual attack translates the LWE problem to a ℓ_2 -norm SIS problem, whose solution is used to solve the decisional LWE by distinguishing between subgaussians (for certain parameters) and uniform distribution over $\mathbb{Z}_q^n$. The first dual attack was given in [31], which only cares about the error distribution of the LWE-like problem, and thus does not distinguish between LWE and ALWE. In other words, the original dual attack solves ALWE the same way as it does to LWE, and it appears more efficient when $\alpha_1 \gg \alpha_2$. Formally, let

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\},$$

be a lattice of dimension $d = m \gg n$. Note that $\mathbf{A}$ has full rank with high probability and we assume the submatrix $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ made up by the first n columns of $\mathbf{A}^T$ has rank n and $\mathbf{A}^T = (\mathbf{A}_1 \parallel \mathbf{A}_2) \in \mathbb{Z}_q^{n \times m}$. Let $\mathbf{A}' = (\mathbf{I}_n \parallel \mathbf{A}_1^{-1} \mathbf{A}_2)$. Then, we have that

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\} = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}' \mathbf{x} = \mathbf{0} \bmod q\},$$

and the column vectors of

$$\mathbf{B} = \begin{pmatrix} q\mathbf{I}_n & -\mathbf{A}_1^{-1}\mathbf{A}_2 \\ 0 & \mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitute a basis of lattice Λ and $\det(\Lambda) = q^n$. The dual attack first solves the SIS problem by finding out a short vector $\mathbf{v} \in \mathbb{Z}^m$ with $\mathbf{A}^T \mathbf{v} = \mathbf{0} \bmod q$ in lattice Λ with basis $\mathbf{B}$. It then computes $u = \langle \mathbf{v}, \mathbf{b} \rangle = \langle \mathbf{v}, \mathbf{e} \rangle \bmod q$ in order to distinguish between subgaussian and uniform distributions. Specifically, if the norm $\ell = \|\mathbf{v}\|$ is relatively small, then the value of $\langle \mathbf{v}, \mathbf{e} \rangle$ is relatively small too and u can be seen as random variable drawn from subgaussian distribution with standard variance $\ell\alpha_2$, which can be distinguished from uniform distribution on $\mathbb{Z}_q$ with probability $4 \exp(-2\pi^2\tau^2)$, where $\tau = \ell\alpha_2/q$.

Similar as for primal attacks, we can also make use of the fact that $\mathbf{s} \stackrel{\$}{\leftarrow} \chi_{\alpha_1}^n$ has small norm [4] to optimize the dual attack. Formally, define lattice

$$\Lambda = \{(\mathbf{c}\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n} \mid \mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q, \mathbf{x} \in \mathbb{Z}^m, \mathbf{y} \in \mathbb{Z}^n\},$$

where Λ has dimension $d = m + n$ and $c = \frac{\alpha_2}{\alpha_1}$. Then, the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_m & 0 \\ \mathbf{A}^T & q\mathbf{I}_n \end{pmatrix} \in \mathbb{R}^{(m+n) \times (m+n)}$$

constitutes a basis of lattice Λ with $\det(\Lambda) = c^m q^n$. Likewise, we find out a short vector $\mathbf{v} = (\mathbf{c}\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n}$ with $\mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q$ from $(m+n)$ -dimensional lattice Λ with basis $\mathbf{B}$, and then compute

$$u = \langle \mathbf{x}, \mathbf{b} \rangle = \langle \mathbf{y}, \mathbf{s} \rangle + \langle \mathbf{c}\mathbf{x}, \mathbf{e}/c \rangle \bmod q,$$

and therefore distinguish the above u from uniform. Specifically, the values of $\langle \mathbf{y}, \mathbf{s} \rangle$ and $\langle \mathbf{x}, \mathbf{e} \rangle$ are relatively small for small $\ell = \|\mathbf{v}\|$. Thus, $u = \langle \mathbf{y}, \mathbf{s} \rangle + \langle \mathbf{c}\mathbf{x}, \mathbf{e}/c \rangle$ can be almost regarded as following subgaussian distribution with parameter $\ell\alpha_1$. This enable efficient algorithms that distinguish the subgaussian-distributed u from uniform with probability $4 \exp(-2\pi^2\tau^2)$, where $\tau = \ell\alpha_1/q$.

The Cost of Dual Attack. As shown above, the computational costs of dual attack is mainly dominated by the cost of solving SIS problem and transforming distinguishing attacks into secret recovery attacks (with success probability more than half) for the ALWE problem. We use the model of primal attack to estimate the cost of solving SIS problem. That is, applying BKZ- b to reduce the basis of d -dimensional lattice yields a short vector $\mathbf{v}$ of norm $\ell = \|\mathbf{v}\| = \delta^d \det(\Lambda)^{1/d}$. Substituting this into the previous analysis, we distinguish ALWE problem with advantage $\epsilon = 4 \exp(-2\pi^2\tau^2)$, where τ is fully decided by ℓ and the actual dual attack algorithm. To obtain the final secret recovery attack with probability greater than $1/2$, we need $1/\epsilon^2$ short vectors. Taking into account that each invocation of the sieving yields $2^{0.2075b}$ short vectors, we need to repeat it $R = 1/\max(1, 1/(2^{0.2075b}\epsilon^2))$ times.

Following the cost estimation framework for primal attacks, we adopt the Core-SVP model, which uses the complexity of solving SVP on a b -dimensional lattice as a (conservative) estimate for solving SVP on a d -dimensional lattice. Specifically, we set $\text{cost}_b = 2^{0.292b}$ for classical algorithms and $\text{cost}_b = 2^{0.257b}$ for quantum algorithms when solving SVP on b -dimensional lattices. Under this model, the complexity of dual attack is estimated as

$$1/\max(1, 1/(2^{0.2075b} \cdot 16 \exp(-4\pi^2 \tau^2))) \cdot \text{cost}_b, \quad (4)$$

where τ is decided by b and other parameters of the actual dual attack.

8.1.3 Refined MATZOV Attack (MATZOV[†]). The MATZOV attack given in [30] is basically a variant of the hybrid-dual attack which splits the attack into three parts: a BKZ lattice reduction stage, a guessing stage and a FFT-based distinguishing stage. Formally, given a LWE problem

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m,$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. Let $n_{\text{lat}}, n_{\text{enu}}, n_{\text{fft}}$ be integers satisfying $n = n_{\text{lat}} + n_{\text{enu}} + n_{\text{fft}}$. The matrix $\mathbf{A}$ and secret $\mathbf{s}$ can be divided accordingly:

$$\mathbf{A} = [\mathbf{A}_{\text{enu}}, \mathbf{A}_{\text{fft}}, \mathbf{A}_{\text{lat}}] \in \mathbb{Z}_q^{m \times n_{\text{enu}}} \times \mathbb{Z}_q^{m \times n_{\text{fft}}} \times \mathbb{Z}_q^{m \times n_{\text{lat}}},$$

$$\mathbf{s} = \begin{pmatrix} \mathbf{s}_{\text{enu}} \\ \mathbf{s}_{\text{fft}} \\ \mathbf{s}_{\text{lat}} \end{pmatrix} \in \mathbb{Z}_q^{n_{\text{enu}}} \times \mathbb{Z}_q^{n_{\text{fft}}} \times \mathbb{Z}_q^{n_{\text{lat}}}.$$

The attack works by first performing BKZ lattice reduction on the lattice generated by

$$\mathbf{B}_{\text{lat}} = \begin{pmatrix} \mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{Z}_q^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}$$

to obtain a set $\mathcal{S}$ of N vectors $(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}_q^{m+n_{\text{lat}}}$ with small Euclidean norm such that $\mathbf{y} = \mathbf{A}_{\text{lat}}^T \mathbf{x}$. Then, for each vector $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$, by the equation $\mathbf{b} = \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} + \mathbf{A}_{\text{fft}} \mathbf{s}_{\text{fft}} + \mathbf{A}_{\text{lat}} \mathbf{s}_{\text{lat}} + \mathbf{e}$ we have that

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{x}, \mathbf{A}_{\text{fft}} \mathbf{s}_{\text{fft}} \rangle}_{= \langle \hat{\mathbf{a}}, \mathbf{s}_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle}_{= \hat{e}}, \quad (5)$$

where $\hat{\mathbf{a}} = \mathbf{A}_{\text{fft}}^T \mathbf{x}$, $\hat{b} = \mathbf{x}^T (\mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}})$ and $\hat{e} = \mathbf{x}^T \mathbf{e} + \mathbf{y}^T \mathbf{s}_{\text{lat}}$. Clearly, if $(\mathbf{e}, \mathbf{s}_{\text{lat}})$ has small Euclidean norm, then $\hat{e}$ is small. Thus, by correctly guessing $\mathbf{s}_{\text{enu}}$, we can obtain a new LWE sample $\hat{b} = \hat{\mathbf{a}}^T \mathbf{s}_{\text{fft}} + \hat{e}$ in much smaller dimension $n_{\text{fft}} \ll n$ for each $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$. Finally, the attack ends by running a FFT-based distinguishing algorithm over $\mathbb{Z}_q^{n_{\text{fft}}}$ which can determine whether a guessed value for $\mathbf{s}_{\text{enu}}$ is correct or not. The core improvement of the MATZOV attack over previous hybrid-dual attacks such as [23] is the introduction of modulus-switching to speedup the FFT-based distinguishing stage, which greatly reduces the FFT cost from previous

$O(n_{\text{fft}}q^{n_{\text{fft}}})$ to $O(n_{\text{fft}}p^{n_{\text{fft}}})$ for some much smaller $p \ll q$ by switching the obtained LWE samples in Equation (5) from $\mathbb{Z}_q^{n_{\text{fft}}}$ to $\mathbb{Z}_p^{n_{\text{fft}}}$. The main problem of the MAZOV attack is the requirement of some controversial assumptions in the complexity analysis [18,33].

Recently, Carrier et al. [13] replaced the modulus switching part of the MATZOV attack with a lossy source/quantizing approach, and obtained a variant that is supported by experimentally validated assumptions. Specifically, given integers $k_{\text{fft}} \ll n_{\text{fft}}$ and a linear code generated by some $\mathbf{G} \in \mathbb{Z}_q^{n_{\text{fft}} \times k_{\text{fft}}}$ such that for each $\mathbf{A}_{\text{fft}}^T \mathbf{x}$ there exists a vector $\mathbf{u} \in \mathbb{Z}_q^{k_{\text{fft}}}$ and a small vector $\mathbf{e}_{\mathbf{x}} \in \mathbb{Z}_q^{n_{\text{fft}}}$ satisfying $\mathbf{A}_{\text{fft}}^T \mathbf{x} = \mathbf{G}\mathbf{u} + \mathbf{e}_{\mathbf{x}}$, we can rewrite Equation (5) as follows:

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{u}, \mathbf{G}^T \mathbf{s}_{\text{fft}} \rangle}_{= \langle \mathbf{u}, \mathbf{s}'_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle + \langle \mathbf{e}_{\mathbf{x}}, \mathbf{s}_{\text{fft}} \rangle}_{= \hat{e}'} . \quad (6)$$

Now, the attack can be finished by running the FFT-based distinguishing algorithm over $\mathbb{Z}_q^{k_{\text{fft}}}$, and the FFT cost is reduced from previous $O(n_{\text{fft}}q^{n_{\text{fft}}})$ to $O(k_{\text{fft}}q^{k_{\text{fft}}})$. In [13], Carrier et al. demonstrated that their variant of the MATZOV attack is not only supported by experimental evidence but also outperforms the original MATZOV attack [30]. Thus, for more conservative concrete security estimates, one should consider the MATZOV attack in [13], as also remarked in [22].

Since Carrier et al.'s attack [13] only considers symmetric distributions (namely, $\alpha_1 = \alpha_2$), we slightly adapt their attack to handle asymmetric distributions. Formally, let $c = \alpha_2/\alpha_1$, we scale the lattice used in the BKZ lattice reduction stage as follows:

$$\mathbf{B}'_{\text{lat}} = \begin{pmatrix} c\mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{R}^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}.$$

By doing this, we can model the distribution of $\hat{e}'$ in Equation (6) as a Gaussian distribution with parameter $\ell\alpha_1$, where $\ell = \|(c\mathbf{x}, \mathbf{y}, \mathbf{e}_{\mathbf{x}})\|_2$ for each $\mathbf{y} = \mathbf{A}_{n_{\text{lat}}}^T \mathbf{x}$. For ease of presentation, we refer to our above refined MATZOV attack as MATZOV[†].

The Cost of MATZOV[†] Attack. As shown in [13], the total cost of the MATZOV[†] attack depends on the cost T_{sample} for obtaining the set $\mathcal{S}$ of N short vectors, R for the expected number of times to guess $\mathbf{s}_{\text{enu}}$ correctly, T_{dec} for decoding $\mathbf{e}_{\mathbf{x}}$ from $\mathbf{A}_{\text{fft}}^T \mathbf{x}$, and T_{fft} for running the FFT-based distinguishing algorithm, namely,

$$T_{\text{cost}} = T_{\text{sample}} + R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}}),$$

where $T_{\text{sample}} = 2^{0.292b}$ and $N = 2^{0.2075b}$ for performing SVP sieving algorithm in b -dimensional sublattice. For quantum security estimates, in addition to considering quantum SVP sieving cost $T'_{\text{sample}} = 2^{0.257b}$, we additionally assume that the guessing-then-FFT part can achieve a quantum quadratic speedup [2], namely,

$$T'_{\text{cost}} = T'_{\text{sample}} + \sqrt{R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}})}.$$

We note that this will only make our concrete estimates more conservative. Because T_{dec} and T_{fft} depend strongly on the modulus size, and R on the sparsity of the secret vector, the above analysis immediately imply that the MATZOV[†] attack may be particularly advantageous for sparse secrets and small modulus.

8.2 Attacks against ASIS

Given a uniform matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times (m_1+m_2-n)}$, the ASIS problem $\text{ASIS}_{n,q,m_1,m_2,\beta_1,\beta_2}^\infty$ asks to find a non-zero vector $\mathbf{x} \in \mathbb{Z}_q^{m_1+m_2} \setminus \{\mathbf{0}\}$ such that $(\mathbf{I}_n \parallel \mathbf{A})\mathbf{x} = \mathbf{0} \bmod q$, $\|\mathbf{x}_1\|_\infty \leq \beta_1$ and $\|\mathbf{x}_2\|_\infty \leq \beta_2$, where $\mathbf{x} = (\mathbf{x}_1^T, \mathbf{x}_2^T)^T \in \mathbb{Z}^{m_1+m_2}$, $\mathbf{x}_1 \in \mathbb{Z}^{m_1}$, $\mathbf{x}_2 \in \mathbb{Z}^{m_2}$. The general BKZ lattice basis reduction attack and its variants are more useful for solving the ℓ_2 -norm ASIS problem than the ℓ_∞ -norm counterpart. We therefore consider the BKZ basis reduction attack specialized to $\text{ASIS}_{n,q,m_1,m_2,\beta_1,\beta_2}^\infty$ in [35], which are basically an adaption of the ℓ_∞ -norm SIS attack in [19] to the asymmetric setting under several heuristics that are favorable to the attackers.

8.2.1 BKZ Lattice Reduction Attack.

Define lattice

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^{m_1+m_2} \mid (\mathbf{I}_n \parallel \mathbf{A})\mathbf{x} = \mathbf{0} \bmod q\}.$$

Then, column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} q\mathbf{I}_n & -\mathbf{A} \\ 0 & \mathbf{I}_{m_1+m_2} \end{pmatrix} \in \mathbb{Z}^{(m_1+m_2) \times (m_1+m_2)}$$

constitute a basis of lattice Λ of dimension $d = m_1 + m_2$ and $\det(\Lambda) = q^n$. Applying the BKZ- b basis reduction to $\mathbf{B}$ yields a reduced basis $\hat{\mathbf{B}} \in \mathbb{Z}^{d \times d}$. Let $\hat{\mathbf{B}}^* = (\hat{\mathbf{b}}_1^*, \dots, \hat{\mathbf{b}}_d^*)$ be the orthogonalized matrix of $\hat{\mathbf{B}}$ by means of the Gram-Schmidt process, and $\ell_i = \log_2(\|\hat{\mathbf{b}}_i^*\|)$, then there exist integers $1 \leq i \leq j \leq d$ such that

- For the first i vectors, we have $\ell_1 = \dots = \ell_i = \log_2 q$;
- For the middle $j - i$ vectors, we have $\ell_{j'} = \log_2 q + s \cdot (j' - i)$ for each $j' \in \{i+1, \dots, j\}$;
- For the last $d - j$ vectors, we have $\ell_{j+1} = \ell_{j+2} = \dots = \ell_d = 0$.

where $s = \frac{1}{b-1} \log_2(\frac{b}{2\pi e}(\pi b)^{1/b}) < 0$ and $(j - i)(i + j + 1)s = -2(n - i) \log_2 q$.

The BKZ- b algorithm does not guarantee finding a valid solution to the ASIS problem. However, at the cost of solving an SVP problem on a b -dimensional lattice, one can produce $(\sqrt{4/3})^b$ vectors. For each such vector, the ℓ_2 -norm of its orthogonal projection onto the space spanned by the first i vectors of $\hat{\mathbf{B}}^*$ is approximately $2^{\ell_{i+1}}$. As in [19], we also assume that the $(\sqrt{4/3})^b$ vectors follow the distribution below: the coefficients for the first i dimensions follow the uniform distribution on $\mathbb{Z}_q$, those for the $(i+1)$ -th to j -th dimension follow the Gaussian distribution with standard variance $2^{\ell_{i+1}}/\sqrt{j-i}$, and the last $d-j$ one are zero. Furthermore, we assume the attack succeeds in solving the $\text{ASIS}_{n,q,m_1,m_2,\beta_1,\beta_2}$ problem whenever there exists a vector $\mathbf{x} = (\mathbf{x}_1^T, \mathbf{x}_2^T)^T \in \mathbb{Z}_q^{m_1+m_2}$ with $|\mathbf{x}_1|_\infty \leq \max(\beta_1, \beta_2)$ and $|\mathbf{x}_2|_\infty \leq \min(\beta_1, \beta_2)$. Here, the dimension of $\mathbf{x}_1$ is m_1 if $\beta_1 > \beta_2$, and m_2 otherwise. That is, we always assume that the target vector can be arranged in a descending order that is compatible with the BKZ basis (note that making such an assumption can only make the estimation conservative). Under the above assumptions, we finally estimate the success probability p of solving the ASIS problem as the probability that, among those $(\sqrt{4/3})^b$ vectors, there is a vector $\mathbf{x} = (\mathbf{x}_1^T, \mathbf{x}_2^T)^T$ satisfying $\|\mathbf{x}_1\|_\infty \leq \max(\beta_1, \beta_2)$ and $\|\mathbf{x}_2\|_\infty \leq \min(\beta_1, \beta_2)$. The total cost is then given by cost_b/p , where cost_b is the cost of obtaining these $(\sqrt{4/3})^b$ vectors.

8.2.2 Scaling and Randomizing. We can also optimize the above attack in [19] by balancing the components of the target vector [35]. Formally, let $c = \beta_2/\beta_1$ and define lattice

$$\Lambda = \left\{ \mathbf{x} = \begin{pmatrix} c\mathbf{x}_1 \\ \mathbf{x}_2 \end{pmatrix} \in \mathbb{R}^{m_1+m_2} \mid (\mathbf{I}_n \parallel \mathbf{A})\mathbf{v} = 0 \bmod q, \mathbf{v} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{pmatrix} \in \mathbb{Z}^{m_1+m_2} \right\}.$$

Then, the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_{m_1} & 0 \\ 0 & \mathbf{I}_{m_2} \end{pmatrix} \begin{pmatrix} q\mathbf{I}_n & -\mathbf{A} \\ 0 & \mathbf{I}_{m_1+m_2-n} \end{pmatrix} \in \mathbb{R}^{(m_1+m_2) \times (m_1+m_2)}$$

constitute a basis of lattice Λ of dimension $d = m_1 + m_2$ and $\det(\Lambda) = c^{m_1}q^n$. Clearly, we can get the solution $(\mathbf{x}_1^T, \mathbf{x}_2^T)^T \in \mathbb{Z}^{m_1+m_2}$ to the ASIS problem by finding out $\mathbf{x} = (c\mathbf{x}_1^T, \mathbf{x}_2^T)^T \in \mathbb{R}^{m_1+m_2}$ with $\|\mathbf{x}\|_\infty \leq \beta_2$ on lattice Λ . Note that when $c \neq 1$ and $m_1 \geq n$, the basis $\mathbf{B}$ does not contain the trivial vectors $q\mathbf{e}_i$, where $\mathbf{e}_i$ denotes the i -th unit vector. We therefore forget the possible q -vectors and randomize the matrix $\mathbf{B}$ to get $\mathbf{B}'$ before running BKZ- b to get a reduced basis $\hat{\mathbf{B}} \in \mathbb{Z}^{d \times d}$ as in [19]. Let $\hat{\mathbf{B}}^* = (\hat{\mathbf{b}}_1^*, \dots, \hat{\mathbf{b}}_d^*)$ be the Gram-Schmidt orthogonalized matrix of $\hat{\mathbf{B}}$. Denote $\ell_i = \log_2(\|\hat{\mathbf{b}}_i^*\|)$. We have that there exists $j \in \{1, \dots, d\}$ such that:

- For the first j vectors, we have $\ell_{j'} = -s \cdot (j - j' + 1)$ for each $j' \in \{1, \dots, j\}$;
- For the last $d - j$ vectors, we have $\ell_{j+1} = \ell_{j+2} = \dots = \ell_d = 0$,

where $s = \frac{1}{b-1} \log_2(\frac{b}{2\pi e}(\pi b)^{1/b}) < 0$ and $j(j+1) = -\frac{2n \log_2 q}{s}$. Similarly, by paying the cost of solving SVP problem on b -dimensional lattice, we obtain from $\hat{\mathbf{B}} \in \mathbb{Z}^{d \times d}$ a sequence of $(\sqrt{4/3})^b$ vectors with ℓ_2 -norm roughly equal to 2^{ℓ_1} . Similarly, we assume that the $(\sqrt{4/3})^b$ vectors follow the distribution below [19]: the coefficients for the 1-th to j -th dimension follow the Gaussian distribution with standard variance $2^{\ell_1}/\sqrt{j}$, and the last $d - j$ one are zero. Under the above assumptions, we estimate the success probability p of solving the ASIS problem as the probability that, among those $(\sqrt{4/3})^b$ vectors, there is a vector $\mathbf{x} \in \mathbb{Z}^{m_1+m_2}$ satisfying $\|\mathbf{x}\|_\infty \leq \beta_2$. Finally, the total cost is then given by cost_b/p as before, where cost_b is the cost of obtaining these $(\sqrt{4/3})^b$ vectors.

Remark 1. In general, the BKZ lattice reduction attack is weaker than the scaling and randomizing attack. But when $\max(\beta_1, \beta_2)$ is close to the q , keeping the q -vectors in the former attack will be more useful than intentionally forgetting them. We thus always consider the two attacks in our concrete estimates.

8.2.3 Cost Model for Solving SVP. As discussed above, the attacks will first perform BKZ- b basis reduction and then solve the b -dimensional SVP problem to sufficient number of candidate vectors. As the former has to invoke the latter as a sub-routine polynomial times, we consider the conservative Core-SVP model [19], which simply use the cost cost_b of a single call to the b -dimensional SVP-solver as an estimate of the cost for getting the $(\sqrt{4/3})^b$ vectors for the above attacks. As before, we take $2^{0.292b}$ (resp. $2^{0.257b}$) as the time cost of the best known classical (resp. quantum) algorithm for solving the b -dimensional SVP, as reported in [10] (resp. [14]).

8.3 Concrete Security Estimates

The complexity of the attacks depends on the attack type, the number of samples m , and the block size $b \in \mathbb{Z}$ used in the BKZ- b algorithm. Therefore, to obtain a conservative overall security estimation for Aigis-Sig⁺ under the three recommended parameter sets, we enumerate all possible values of m and b . In details, Table 6 estimate the complexities of the three parameter sets of the underlying AMLWE problem against the aforementioned attacks, where the concrete security estimates against MATZOV[†] are derived from a code that modifies the publicly available implementation by Carrier et al. [13], which can be found at <https://github.com/kevin-carrier/CodedDualAttack>. For comparison, we also give the figures against the original MATZOV attack [30] given by the LWE estimator [3]. As shown in 6, the MATZOV[†] attack almost dominates the concrete estimates of LWE hardness for all parameters. Table 7 estimate the complexities of the three parameter sets of the underlying SelfTargetAM SIS (resp., AMSIS) problem for the UF-CMA (resp., SUF-CMA) security.

Table 6: The Concrete Security Estimates for the underlying AMLWE problem of Aigis-Sig⁺ (where C and Q denotes classical and quantum security, respectively)

	Primal	Dual	MATZOV	MATZOV [†]
PARAMS I	C: (921,459,134) Q: (921,459,117)	C: (921,456,133) Q: (921,456,117)	C: (1024,447,130) Q: (1024,449,115)	C: (1020,454,144) Q: (1020,509,133)
PARAMS II	C: (1821, 984, 287) Q: (1821, 984, 252)	C: (1751,980,286) Q: (1751,980,251)	C: (2048,941,275) Q: (2048,948,243)	C: (1826,927,271) Q: (1728,879,228)
PARAMS III	C: (2971,1924,561) Q: (2971,1924,494)	C: (2971,1916,559) Q: (2971,1916,492)	C: (3566,1821,531) Q: (3566,1837,472)	C: (2984,1782,521) Q: (2829,1674,431)

Table 7: The Concrete Security Estimates for the underlying AMSIS problem of Aigis-Sig⁺ (where C and Q denotes classical and quantum security, respectively)

Parameter Set	SelfTargetAM SIS (m, b, sec)		AM SIS hardness (m, b, sec)	
	BKZ	BKZ-SR	BKZ	BKZ-SR
PARAMS I	C: (2370,684,200) Q: (2370,684,175)	C: (1790,477,139) Q: (1790,477,122)	C: (2520,596,174) Q: (2520,596,153)	C: (1780,442,129) Q: (1780,442,113)
PARAMS II	C: (4154,1241,362) Q: (4154,1241,318)	C: (3484,976,285) Q: (3484,976,250)	C: (3939,1061,309) Q: (3939,1061,272)	C: (3524,939,274) Q: (3524,939,241)
PARAMS III	C: (7587,2308,675) Q: (7587,2308,593)	C: (7157,2001,585) Q: (7157,2001,514)	C: (7502,2237,653) Q: (7502,2237,574)	C: (7102,1891,552) Q: (7102,1891,485)

8.4 Attacks against Symmetric Primitives

The overall security of **Aigis-Sig**⁺ relies not only on the hardness of the underlying mathematical problems, but also closely on the security of several symmetric primitives it invokes, such as collision-resistant hash functions and pseudorandom number generators. In Sec. 5.2, we instantiate these primitives with SHA3. This instantiation rests on a clear security assumption: SHA3 provides the required collision resistance, preimage resistance, and pseudorandomness at the level matching our concrete security parameter in both the classical and quantum setting.

Crucially, **Aigis-Sig**⁺ is designed and proven secure by treating these symmetric primitives as idealised black boxes throughout the security reduction. This means that the proof does not depend on the internal details or specific algebraic structure of SHA3, but only on its external input-output behaviour and the expected security metrics. Such a highly modular abstraction offers strong cryptographic agility: in the future, should any major cryptanalytic breakthrough against SHA3 emerge—for instance, collision or preimage attacks significantly more efficient than anticipated—that would undermine the prescribed security strength in either the classical or quantum setting, we would not need to redesign the core public-key framework of **Aigis-Sig**⁺ nor invalidate its existing security reduction. In practice, one would simply replace SHA3 with another well-analysed, standardised algorithm that provides an equivalent interface, and the overall scheme architecture and core logic would remain unchanged, thereby swiftly restoring and maintaining the actual security margin of the scheme.

9 Side-Channel Vulnerabilities and Countermeasures

To the best of our knowledge, existing design and implementation of **Aigis-Sig**⁺ follow a constant-time design paradigm, where the execution time is independent of secret-dependent values. Consequently, timing-based side-channel leakage is not expected to be exploitable in practice.

Regarding physical side channels such as power consumption and electromagnetic leakage, although there exist constructions that aim to achieve leakage resilience, they often incur prohibitive overhead and are therefore impractical for real-world deployments. Instead, we argue that having a clear and systematic understanding of the inherent side-channel vulnerabilities of the algorithm, together with targeted and efficient countermeasures, is of primary importance for achieving practical security.

9.1 Security Analysis

Figures 5 and 6 illustrate the **Aigis-Sig**⁺ signature procedure and its side-channel vulnerabilities. Non-sensitive components are highlighted in blue, indicating that no protection is required, while sensitive components are marked in red.

We analyze the side-channel leakage of the **Aigis-Sig**⁺ signature algorithm by identifying all sensitive variables and operations during key generation and signing.

Public values can be assumed to be fully exposed, since they are deterministically derived from the seed ρ . Similarly, tr is also deterministically generated

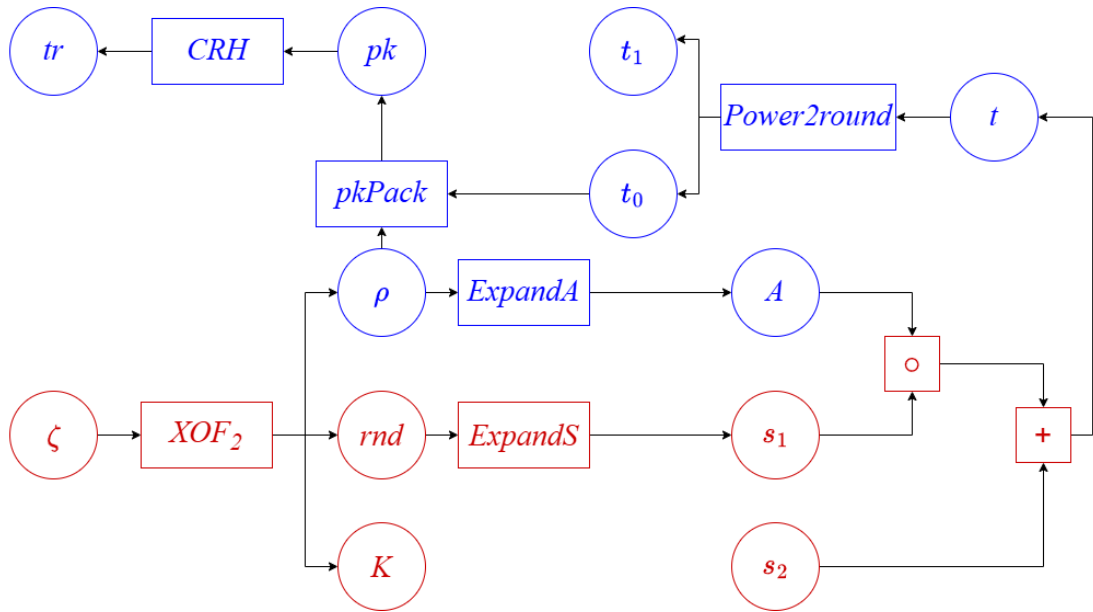


Fig. 5: Aigis-Sig⁺ key-generation procedure and side-channel vulnerabilities.

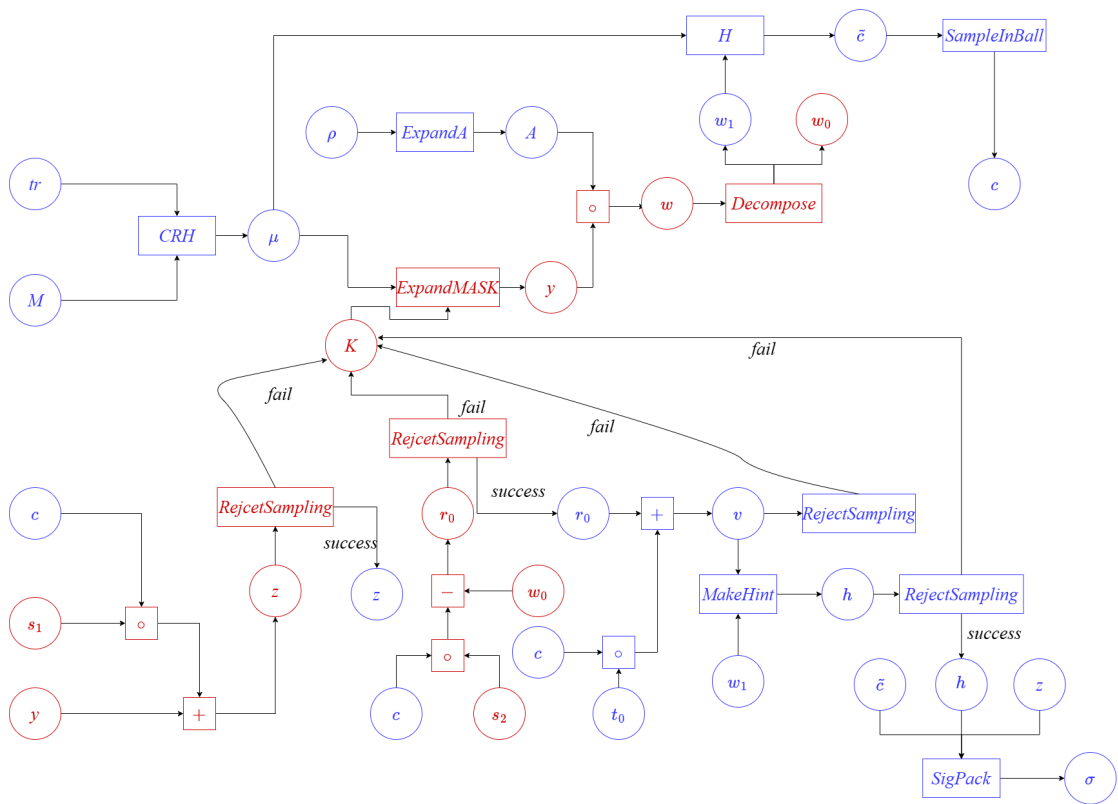


Fig. 6: Aigis-Sig⁺ signing procedure and side-channel vulnerabilities.

from ρ and does not require protection. The polynomial vector $\mathbf{t}_0$ is obtained via `Power2Round` from $\mathbf{t}$, where $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Although $\mathbf{t}$ depends on secret values $\mathbf{s}_1$ and $\mathbf{s}_2$, recovering them from $\mathbf{t}$ is infeasible under the hardness of AMLWE. Therefore, $\mathbf{t}_0$ does not require protection.

Key Generation. During key generation, the randomness ξ must be masked, since it serves as the seed for all subsequent values (e.g., K, ρ). Similarly, the seed rnd used to generate $\mathbf{s}_1$ and $\mathbf{s}_2$ must be masked. All operations involving secret values must be implemented using secure masked addition and multiplication.

Signature Generation. During signing, secret values $\mathbf{s}_1$ and $\mathbf{s}_2$ must always remain masked. In addition, the polynomial vector $\mathbf{y}$ must also be masked, since from a valid signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ one obtains $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, which allows the adversary to extract information about $\mathbf{s}_1$ from leakage on $\mathbf{y}$ [34].

Furthermore, the `Decompose` procedure must be redesigned. First, since from $\mathbf{w} = \mathbf{A}\mathbf{y}$, the adversary can recover $\mathbf{y}$ given $\mathbf{w}$ and $\mathbf{A}$. Second, if $\mathbf{w}_0$ is unprotected, then from $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$, $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$, one obtains $\mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d = \mathbf{w}_1 \cdot 2\gamma_2 + \mathbf{w}_0 + c(\mathbf{t}_0 - \mathbf{s}_2)$. When $(\mathbf{w}_0)_{i,j} = 0$, the value $(\mathbf{t}_0 - \mathbf{s}_2)_{i,j}$ can be directly recovered. By collecting many signatures and performing template attacks to determine whether $(\mathbf{w}_0)_{i,j} = 0$, the adversary can recover the coefficients of $\mathbf{s}_2$ [11].

Finally, rejection sampling involving

$$\mathbf{z} = \mathbf{y} + c\mathbf{s}_1, \quad \mathbf{r}_0 = \mathbf{w}_0 - c\mathbf{s}_2$$

must be performed securely, and all intermediate values must remain masked before acceptance.

9.2 A Discussion on Masking Countermeasures

Linear operations (e.g., polynomial addition, NTT, point-wise multiplication) can be computed in a share-wise manner. We focus on nonlinear components. **Aigis-Sig⁺** shares substantial structural and implementation-level commonalities with ML-DSA (e.g., Dilithium), especially in its lattice-based construction, polynomial arithmetic, and rejection sampling workflow. This close correspondence allows existing masking techniques and design methodologies for ML-DSA to be systematically transferred and adapted to the **Aigis-Sig⁺** setting. In the rest of this subsection, we denote d as the masked order.

Masked Sampling of $\mathbf{y}$. In the unmasked setting, sampling a coefficient $\mathbf{y}$ is straightforward: one simply samples an integer uniformly from the interval $[-\gamma_1, \gamma_1]$. Since γ_1 is typically a power of two, this can be efficiently implemented by generating $\log_2(\gamma_1) + 1$ random bits and applying a simple mapping.

However, this approach does not directly extend to the masked setting. When using arithmetic masking, each coefficient is represented as the sum of d shares modulo q . Even if each share is independently sampled within $[-\gamma_1, \gamma_1]$, their modular sum may fall outside the desired interval due to wrap-around effects. As a result, the distribution of the reconstructed value is no longer guaranteed to be uniform over $[-\gamma_1, \gamma_1]$.

It conversys that, masked sampling requires additional mechanisms to ensure that the final reconstructed value lies within the correct range while preserving both correctness and security, which significantly complicates the design compared to the unmasked case. In this respect, masked sampling of $\mathbf{y}$ consists of the following two parts:

- Masking Keccak costs $O(td^2)$, where t is the complexity of unmasked Keccak;
- Converting bit vectors into randomness over $\mathbb{Z}_q$ requires a change of masking domain, i.e., a Boolean-to-arithmetic conversion. Typical construction incurs a computational complexity of $O(d^2 \log q)$ [9].

Masked Decompose. The Decompose procedure splits an element $r \in \mathbb{Z}_q$ into a high part and a low part,

$$r = r_1\alpha + r_0,$$

where r_1 represents the most significant bits and r_0 the least significant part. This decomposition is used in both signing and verification, in particular for norm checking and hint generation. From a side-channel perspective, Decompose constitutes a major challenge. The original algorithm involves conditional operations and special-case handling (e.g., centered reduction and boundary adjustments), which introduce non-linear control flow and data-dependent branches. Typical construction (e.g., [16]) of masking such operation incurs a computational complexity of $O(d^2(\log q + \log d))$.

Masked Rejection Sampling. Masked rejection sampling is particularly challenging due to the need to securely evaluate acceptance conditions on masked values. In Aigis-Sig⁺, the quantities involved in rejection sampling, such as $\mathbf{z} = \mathbf{y} + \mathbf{c}\mathbf{s}_1$ and $\mathbf{r} = \mathbf{w}_0 - \mathbf{c}\mathbf{s}_2$, are represented in arithmetic sharing. However, the rejection conditions inherently involve comparisons and threshold checks, which are not directly supported in the arithmetic masking domain.

As a result, it is necessary to convert arithmetic shares into Boolean shares via a secure arithmetic-to-Boolean (A2B) conversion, enabling bitwise comparison and conditional evaluation. This conversion constitutes the main computational bottleneck. In particular, typical A2B construction (e.g., [16]) incurs a complexity of $O(kd^2)$, where k is the bit-length of $\mathbf{z}$ or $\mathbf{r}$.

References

1. Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
2. Martin R. Albrecht and Yixin Shen. Quantum augmented dual attack. Cryptology ePrint Archive, Paper 2022/656, 2022.
3. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. In *Journal of Mathematical Cryptology*, 9:169–203, oct 2015.
4. Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange-a new hope. In *USENIX Security Symposium*, volume 2016, 2016.

5. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II*, pages 269–295. Springer, 2019.
6. Shi Bai and Steven D. Galbraith. Lattice decoding attacks on binary lwe. In Willy Susilo and Yi Mu, editors, *Information Security and Privacy*, pages 322–337, Cham, 2014. Springer International Publishing.
7. Shi Bai and Steven D. Galbraith. An improved compression technique for signatures based on learning with errors. In Josh Benaloh, editor, *Topics in Cryptology - CT-RSA 2014*, volume 8366 of *Lecture Notes in Computer Science*, pages 28–47. Springer International Publishing, 2014.
8. Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023*, pages 358–389, Cham, 2023. Springer Nature Switzerland.
9. Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Benjamin Grégoire, Mélissa Rossi, and Mehdi Tibouchi. Masking the GLP lattice-based signature scheme at any order. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II*, Lecture Notes in Computer Science, pages 354–384. Springer, 2018.
10. Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '16, page 10–24, USA, 2016. Society for Industrial and Applied Mathematics.
11. Alexandre Berzati, Andersson Calle Viera, Maya Chartouni, Steven Madec, Damien Vergnaud, and David Vigilant. A practical template attack on crystals-dilithium. *IACR Cryptol. ePrint Arch.*, 2023:50, 2023.
12. J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle. Crystals - kyber: A cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS P)*, pages 353–367, April 2018.
13. Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In *Annual International Cryptology Conference*, pages 444–476. Springer, 2025.
14. André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021*, pages 63–91, Cham, 2021. Springer International Publishing.
15. Yuanmi Chen. Lattice reduction and concrete security of fully homomorphic encryption. *Dept. Informatique, ENS, Paris, France, PhD thesis*, 2013.
16. Jean-Sebastien Coron, Francois Gerard, Simon Montoya, and Rina Zeitoun. High-order polynomial comparison and masking lattice-based encryption. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(1):153–192, 2023.
17. Julien Devevey, Pouria Fallahpour, Alain Passelègue, and Damien Stehlé. A detailed analysis of fiat-shamir with aborts. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023*, pages 327–357, Cham, 2023. Springer Nature Switzerland.
18. Léo Ducas and Ludo N. Pulles. Does the dual-sieve attack on learning with errors even work? In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023*, pages 37–69, Cham, 2023. Springer Nature Switzerland.
19. Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, Feb. 2018.
20. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer Berlin Heidelberg, 1987.
21. Ronald Aylmer Fisher and Frank Yates. *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver & Boyd, 1938.
22. Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. FrodoKEM: A CCA-secure learning with errors key encapsulation mechanism. *IACR Communications in Cryptology*, 2(3), 2025.
23. Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving lwe with applications to crystals. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021*, pages 33–62, Cham, 2021. Springer International Publishing.

24. Kelsey A. Jackson, Carl A. Miller, and Daochen Wang. Evaluating the security of crystals-dilithium in the quantum random oracle model. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 418–446, Cham, 2024. Springer Nature Switzerland.
25. Ravi Kannan. Minkowski’s convex body theorem and integer programming. *Mathematics of Operations Research*, 12(3):415–440, 1987.
26. Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of fiat-shamir signatures in the quantum random-oracle model. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, volume 10822 of *LNCS*, pages 552–586. Springer International Publishing, 2018.
27. Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In Aggelos Kiayias, editor, *Topics in Cryptology – CT-RSA 2011*, volume 6558 of *Lecture Notes in Computer Science*, pages 319–339. Springer Berlin Heidelberg, 2011.
28. Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *Advances in Cryptology – ASIACRYPT 2009*, volume 5912 of *Lecture Notes in Computer Science*, pages 598–616. Springer Berlin / Heidelberg, 2009.
29. Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755. Springer Berlin Heidelberg, 2012.
30. IDF MATZOV. Report on the security of lwe: improved dual lattice attack, 2022. *URL: <https://zenodo.org/record/6412487>*, 2022.
31. Daniele Micciancio and Oded Regev. Lattice-based cryptography. In DanielJ. Bernstein, Johannes Buchmann, and Erik Dahmen, editors, *Post-Quantum Cryptography*, pages 147–191. Springer Berlin Heidelberg, 2009.
32. NIST. FIPS PUB 202: SHA-3 Standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication 202, U.S. Department of Commerce, 2015.
33. Amaury Pouly and Yixin Shen. Provable dual attacks on learning with errors. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 256–285, Cham, 2024. Springer Nature Switzerland.
34. Vincent Quentin Ulitzsch, Soundes Marzougui, Mehdi Tibouchi, and Jean-Pierre Seifert. Profiling side-channel attacks on dilithium - A small bit-fiddling leak breaks it all. In Benjamin Smith and Huapeng Wu, editors, *Selected Areas in Cryptography - 29th International Conference, SAC 2022, Windsor, ON, Canada, August 24-26, 2022, Revised Selected Papers*, Lecture Notes in Computer Science, pages 3–32. Springer, 2022.
35. Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020*, pages 37–65. Springer, 2020.
36. Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In *IACR International Conference on Public-Key Cryptography*, pages 37–65. Springer, 2020.
37. Jieyu Zheng, Feng He, Shiyu Shen, Chenxi Xue, and Yunlei Zhao. Parallel small polynomial multiplication for dilithium: A faster design and implementation. In *Annual Computer Security Applications Conference, ACSAC 2022, Austin, TX, USA, December 5-9, 2022*, pages 304–317. ACM, 2022.

Appendix A Proof of Theorem 1

In this appendix, we prove Theorem 1 according to the following three steps:

1. Recall that Aigis-Sig^+ is obtained by applying the Fiat-Shamir with Aborts (FSwA) transformation to a three-move identification scheme. In Appendix A.1, we first abstract the three-move identification scheme internally used by Aigis-Sig^+ , which we denote by Aigis-id . Then, we define two parameters and a zero-knowledge simulator of Aigis-id needed by the subsequent proof.
2. In Appendix A.2, we prove that in the ROM the SUF-CMA security of Aigis-Sig^+ can be reduced to the UF-NMA security of Aigis-Sig^+ and the computational unique response property of Aigis-id .
3. In Appendix A.3, we prove the UF-NMA security of Aigis-Sig^+ in the ROM and prove that Aigis-id has the computational unique response property.

Finally, in Appendix A.4, by combining the results of Appendix A.2 and A.3, we prove the SUF-CMA security of Aigis-Sig^+ in the ROM, i.e., Theorem 1. Before giving the proof, we first present some necessary definitions as follows.

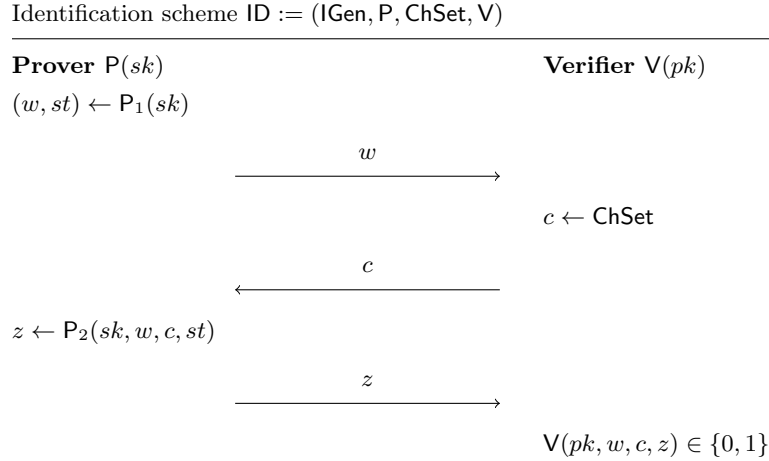


Fig. 7: A canonical identification scheme $\text{ID} := (\text{IGen}, \text{P}, \text{ChSet}, \text{V})$.

Definition 3 (Canonical Identification Scheme). A canonical identification scheme ID is defined as a tuple of algorithms $\text{ID} := (\text{IGen}, \text{P}, \text{ChSet}, \text{V})$.

- The key generation algorithm IGen takes system parameters par as input and returns public and secret key (pk, sk) . We assume that pk defines ChSet (the set of challenges), WSet (the set of commitments), and ZSet (the set of responses).
- The prover algorithm $\text{P} = (\text{P}_1, \text{P}_2)$ is split into two algorithms. P_1 takes as input the secret key sk and returns a commitment $w \in \text{WSet}$ and a state st ; P_2 takes as input the secret key sk , a commitment w , a uniform challenge c , and a state st and returns a response $z \in \text{ZSet} \cup \perp$, where $\perp \notin \text{ZSet}$ is a special symbol indicating failure.

- The verifier algorithm V takes the public key pk and the conversation transcript as input and outputs a deterministic decision, 1 (acceptance) or 0 (rejection).

A more concise description of ID is also given in Fig. 7. We say ID is correct if for any possible transcripts (w, c, z) satisfying $z \neq \perp$, $V(pk, w, c, z) = 1$.

Definition 4 (Commitment-Recoverable). We say $ID := (IGen, P, ChSet, V)$ is commitment-recoverable, if for any (pk, sk) generated by $IGen$, $c \in ChSet$, and $z \in ZSet$ such that there exists some $w \in WSet$ make $V(pk, w, c, z) = 1$, then this w must be unique and can be publicly computed using a commitment recovery algorithm as $w := Rec(pk, c, z)$.

Definition 5 (Accepting Honest-Verifier Zero-Knowledge). We say $ID := (IGen, P, ChSet, V)$ is an accepting Honest-Verifier Zero-Knowledge (acHVZK) with simulation error ζ_{sk} if there exists a polynomial time algorithm $ZKSim$ that, when given the public key pk , outputs (w, c, z) with a distribution that has statistical distance at most ζ_{sk} from the distribution of a transcript (w, c, z) produced by an honest execution of the protocol conditioned on $z \neq \perp$.

Definition 6 (Computational Unique Response). We say ID has the computational unique response (CUR) property if for any polynomial time adversary $\mathcal{A}$, the following advantage function is negligible.

$$\text{Adv}_{ID}^{\text{CUR}}(\mathcal{A}) := \Pr \left[\begin{array}{l} V(pk, w, c, z) = 1, \\ V(pk, w, c, z') = 1 \wedge z \neq z' \end{array} : \begin{array}{l} (pk, sk) \leftarrow IGen, \\ (w, c, z, z') \leftarrow \mathcal{A}(pk) \end{array} \right].$$

FSwA for Commitment-Recoverable Identification Scheme. Let $ID = (IGen, P, ChSet, V)$ be a commitment-recoverable identification scheme with commitment recovery algorithm Rec . Let $H : WSet \times \mathcal{M} \rightarrow ChSet$ be a hash function. The FSwA transformation converts ID into a signature scheme $FSwA[ID, H]$ (with message space $\mathcal{M}$) as shown in Fig. 8.

$FSwA[ID, H].KeyGen$	$FSwA[ID, H].Sign(sk, m)$	$FSwA[ID, H].V(pk, m, (c, z))$
1 : $(pk, sk) \leftarrow IGen$	1 : while $z = \perp$ do	1 : $w := Rec(pk, c, z)$
2 : return (pk, sk)	2 : $(w, st) \leftarrow P_1(sk)$	2 : return $[V(pk, w, c, z)] \wedge [c = H(w, m)]$
	3 : $c = H(w, m)$	
	4 : $z \leftarrow P_2(sk, w, c, st)$	
	5 : end	
	6 : return (c, z)	

Fig. 8: The constituting algorithms of $FSwA[ID, H]$ for commitment-recoverable ID .

A.1 Rewrite Aigis-Sig⁺ via the FSwA Transformation

In this section, we first recall the description of Aigis-Sig⁺, which is given in Fig. 9. Then, we define an identification scheme Aigis.id as shown in Fig. 10, where the

set of challenges ChSet is defined as $\text{ChSet} = \{c \in R_q \mid \|c\|_\infty = 1 \wedge \|c\|_1 = \tau\}$, the set of commitments WSet is defined as $\text{WSet} = R_q^{k8}$, and $\text{ZSet} = \bar{S}_{\gamma_1 - \beta_1}^\ell \times \mathbf{B}_{64,15,\omega}$.

Aigis.sig.KeyGen	Aigis.sig.Sign($sk, m \in \mathcal{M}$)
1 : $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$	1 : $sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$
2 : $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_{\eta_1}^\ell \times S_{\eta_2}^k$	2 : while $(\mathbf{z}, \mathbf{h}) = \perp$ do
3 : $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$	3 : $\mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell$
4 : $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}, d)$	4 : $\mathbf{w} = \mathbf{A}\mathbf{y}$
5 : $pk = (\mathbf{A}, \mathbf{t}_1)$	5 : $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$
6 : $sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$	6 : $c = \text{H}(\mathbf{w}_1, m)$
7 : return (pk, sk)	7 : $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1, \mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c\mathbf{s}_2$
Aigis.sig.Verify($pk, m; c, (\mathbf{z}, \mathbf{h})$)	8 : if $\mathbf{z} \notin \bar{S}_{\gamma_1 - \beta_1}^\ell \vee \mathbf{r} \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$
1 : $pk = (\mathbf{A}, \mathbf{t}_1)$	9 : $(\mathbf{z}, \mathbf{h}) = \perp$
2 : $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$	10 : $\mathbf{v} = \mathbf{r} + c\mathbf{t}_0, \mathbf{h} = \text{MakeHint}(\mathbf{v}, \mathbf{w}_1, 2\gamma_2)$
3 : return $\llbracket \mathbf{z} \in \bar{S}_{\gamma_1 - \beta}^\ell \rrbracket \wedge \llbracket \mathbf{h} \in \mathbf{B}_{64,15,\omega} \rrbracket \wedge$	11 : if $\mathbf{v} \notin \bar{S}_{\gamma_3}^k \vee \mathbf{h} \notin \mathbf{B}_{64,15,\omega}$
4 : $\llbracket c = \text{H}(\mathbf{w}'_1, m) \rrbracket \wedge$	12 : $(\mathbf{z}, \mathbf{h}) = \perp$
5 : $\llbracket \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}'_1 \cdot 2\gamma_2 \in \bar{S}_{\gamma_3}^k \rrbracket$	13 : end
	14 : return $(c, (\mathbf{z}, \mathbf{h}))$

Fig. 9: The constituting algorithms of Aigis-Sig⁺. Here $\mathcal{M}$ denotes the message space and H is a hash function with input sapce $R_q^k \times \mathcal{M}$ and output space $\{c \in R_q \mid \|c\|_\infty = 1 \wedge \|c\|_1 = \tau\}$.

One can observe that the correctness of Aigis.id follows from the correctness of Aigis-Sig⁺. In addition, for any (pk, sk) generated by Aigis.id.Gen, $c \in \text{ChSet}$, and $(\mathbf{z}, \mathbf{h}) \in \text{ZSet}$, if there exists some $w \in \text{WSet}$ make $\text{Aigis.id.V}(pk, w, c, (\mathbf{z}, \mathbf{h})) = 1$, such w must be unique and equal to $\text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$. Thus, by defining

$$\text{Rec}(pk, c, (\mathbf{z}, \mathbf{h})) := \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2),$$

the identification scheme Aigis.id is actually commitment-recoverable and Rec is the corrspeonding commitment recovery algorithm.

Based on the descriptions of Aigis-Sig⁺ and Aigis.id given in Figs. 9 and 10, we have $\text{Aigis-Sig}^+ = \text{FSwA}[\text{Aigis.id}, \text{H}]$, where H is a hash function with input space $\text{WSet} \times \mathcal{M}$ and output space $\text{ChSet} = \{c \in R_q \mid \|c\|_\infty = 1 \wedge \|c\|_1 = \tau\}$. That is, Aigis-Sig⁺ is a signature scheme obtained by applying the FSwA transformation on the (Commitment-Recoverable) identification scheme Aigis.id. In the following, we define two parameters related to Aigis.id:

$$p_{sk} := \Pr \left[z = \perp : \begin{array}{l} (w, st) \leftarrow \text{Aigis.id.P}_1(sk), c \leftarrow \text{ChSet} \\ z \leftarrow \text{Aigis.id.P}_2(sk, w, c, st) \end{array} \right], \quad (7)$$

$$\epsilon := \max_{\mathbf{x} \in \text{WSet}} \Pr [\text{HighBits}(\mathbf{w}, 2\gamma_2) = \mathbf{x} : \mathbf{w} \leftarrow R_q^k].$$

⁸ This is reasonable since $\text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$ for any $\mathbf{A} \in R_q^{k \times \ell}$ and $\mathbf{y} \in \bar{S}_{\gamma_1}^\ell$ must belong to the set R_q^k .

Here p_{sk} denotes the probability that **Aigis.id** generates a transcript (w, c, z) with $z = \perp$, and ϵ denotes the guessing probability of the higher bits of $\text{HighBits}(\mathbf{w}, 2\gamma_2)$, where $\mathbf{w}$ is uniformly sampled from R_q^k . These two parameters will be used by later proofs, and we point out that p_{sk} is defined for a fixed (pk, sk) pair sampled by **Aigis.id.Gen** while ϵ is independent of (pk, sk) .

Aigis.id.Gen	Aigis.id.P ₁ (sk)
1 : $\mathbf{A} \xleftarrow{\$} R_q^{k \times \ell}$	1 : $sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$
2 : $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_{\eta_1}^\ell \times S_{\eta_2}^k$	2 : $\mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell$
3 : $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$	3 : $\mathbf{w} = \mathbf{A}\mathbf{y}$
4 : $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}, d)$	4 : $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$
5 : $pk = (\mathbf{A}, \mathbf{t}_1)$	5 : return $(w = \mathbf{w}_1, st = (\mathbf{A}\mathbf{y}, \mathbf{y}))$
6 : $sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$	
7 : return (pk, sk)	
Aigis.id.V(pk, w = w ₁ , c ∈ ChSet, z = (z, h))	Aigis.id.P ₂ (sk, w = w ₁ , c ∈ ChSet, st = (A _y , y))
1 : $pk = (\mathbf{A}, \mathbf{t}_1)$	1 : $sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$
2 : $\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$	2 : $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1, \mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c\mathbf{s}_2$
3 : return $[\mathbf{z} \in \bar{S}_{\gamma_1-\beta}^\ell] \wedge [\mathbf{h} \in \mathbf{B}_{64,15,\omega}] \wedge$	3 : if $\mathbf{z} \notin \bar{S}_{\gamma_1-\beta_1}^\ell \vee \mathbf{r} \notin \bar{S}_{\gamma_2-\beta_2-\eta_1}^k$
4 : $[\mathbf{w}_1 = \mathbf{w}'_1] \wedge$	4 : $z = \perp$
5 : $[\mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2 \in \bar{S}_{\gamma_3}^k]$	5 : $\mathbf{v} = \mathbf{r} + c\mathbf{t}_0, \mathbf{h} = \text{MakeHint}(\mathbf{v}, \mathbf{w}_1, 2\gamma_2)$
	6 : if $\mathbf{v} \notin \bar{S}_{\gamma_3}^k \vee \mathbf{h} \notin \mathbf{B}_{64,15,\omega}, z = \perp$
	7 : return $z = (\mathbf{z}, \mathbf{h})$

Fig. 10: The constituting algorithms of **Aigis.id**.

Next, we introduce Lemma 3, which shows that **Aigis.id** is a perfect acHVZK. The algorithm **ZKSim** constructed in this lemma will also be used by later proofs.

Lemma 3. *Aigis.id is a perfect acHVZK, i.e., the transcripts (w, c, z) generated by **RealTrans** and **ZKSim** as shown in Fig. 11 have the same distribution.*

Proof. For any $c \in \text{ChSet}$ and $\mathbf{s}_1 \in \bar{S}_{\eta_1}^\ell$, since $c \in \text{ChSet}$ implies $\|c\|_1 = \tau$, we have $\|c\mathbf{s}_1\|_\infty \leq \|c\|_1 \cdot \|\mathbf{s}_1\|_\infty \leq \tau \cdot \eta_1 = \beta_1$, where the last equation is guaranteed by our parameter selection. Hence, for any $\mathbf{z}' \in \bar{S}_{\gamma_1-\beta_1}^\ell$, we have $\mathbf{z}' - c\mathbf{s}_1 \in \bar{S}_{\gamma_1}^\ell$, and thus

$$\Pr_{y \leftarrow \bar{S}_{\gamma_1}^\ell} [\mathbf{y} + c\mathbf{s}_1 = \mathbf{z}'] = \Pr_{y \leftarrow \bar{S}_{\gamma_1}^\ell} [\mathbf{y} = \mathbf{z}' - c\mathbf{s}_1] = 1/|\bar{S}_{\gamma_1}^\ell|.$$

This means that, for any $\mathbf{z}' \in \bar{S}_{\gamma_1-\beta_1}^\ell$, there exists exactly one $\mathbf{y}$ (i.e., $\mathbf{y} = \mathbf{z}' - c\mathbf{s}_1$) in the set $\bar{S}_{\gamma_1}^\ell$ such that $\mathbf{y} + c\mathbf{s}_1 = \mathbf{z}'$ holds. Thus, for the uniformly sampled $\mathbf{y}$ from $\bar{S}_{\gamma_1}^\ell$, the probability that $\mathbf{y} + c\mathbf{s}_1$ takes each value in $\bar{S}_{\gamma_1-\beta_1}^\ell$ is identical, equal to $1/|\bar{S}_{\gamma_1}^\ell|$. Furthermore, the probability that $\mathbf{y} + c\mathbf{s}_1$ lies in $\bar{S}_{\gamma_1-\beta_1}^\ell$ is $|\bar{S}_{\gamma_1-\beta_1}^\ell|/|\bar{S}_{\gamma_1}^\ell|$. Therefore, instead of repeatedly sampling $y \leftarrow \bar{S}_{\gamma_1}^\ell$ and $c \leftarrow \text{ChSet}$ to ensure that $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ lies in $\bar{S}_{\gamma_1-\beta_1}^\ell$, we can perform the following equivalent operation:

- With probability $1 - |\bar{S}_{\gamma_1-\beta_1}^\ell|/|\bar{S}_{\gamma_1}^\ell|$, restart; with probability $|\bar{S}_{\gamma_1-\beta_1}^\ell|/|\bar{S}_{\gamma_1}^\ell|$, sample $\mathbf{z} \leftarrow \bar{S}_{\gamma_1-\beta_1}^\ell$ and $c \leftarrow \text{ChSet}$, and then set $\mathbf{y} = \mathbf{z} - c\mathbf{s}_1$.

RealTrans($sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$)	ZKSim($\mathbf{A}, \mathbf{t}$)
1 : while $z = \perp$ do	1 : while $z = \perp$ do
2 : $\mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell$	2 : with probability $1 - \bar{S}_{\gamma_1 - \beta_1}^\ell / \bar{S}_{\gamma_1}^\ell $, $z = \perp$
3 : $\mathbf{w} = \mathbf{A}\mathbf{y}$	3 : with probability $ \bar{S}_{\gamma_1 - \beta_1}^\ell / \bar{S}_{\gamma_1}^\ell $, do
4 : $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$	4 : $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}, d)$
5 : $c \leftarrow \text{ChSet}$	5 : $\mathbf{z} \leftarrow \bar{S}_{\gamma_1 - \beta_1}^\ell$
6 : $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$	6 : $c \leftarrow \text{ChSet}$
7 : $\mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c\mathbf{s}_2$	7 : $\mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{z} - c\mathbf{t}, 2\gamma_2)$
8 : if $\mathbf{z} \notin \bar{S}_{\gamma_1 - \beta_1}^\ell \vee \mathbf{r} \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$	8 : $\mathbf{v} = \mathbf{r} + c\mathbf{t}_0$
9 : $z = \perp$	9 : if $\mathbf{r} \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k \vee \mathbf{v} \notin \bar{S}_{\gamma_3}^k$, $z = \perp$
10 : $\mathbf{v} = \mathbf{r} + c\mathbf{t}_0, \mathbf{h} = \text{MakeHint}(\mathbf{v}, \mathbf{w}_1, 2\gamma_2)$	10 : $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{z} - c\mathbf{t}, 2\gamma_2)$
11 : if $\mathbf{v} \notin \bar{S}_{\gamma_3}^k \vee \mathbf{h} \notin \mathbf{B}_{64,15,\omega}$	11 : $\mathbf{h} = \text{MakeHint}(\mathbf{v}, \mathbf{w}_1, 2\gamma_2)$
12 : $z = \perp$	12 : if $\mathbf{h} \notin \mathbf{B}_{64,15,\omega}$, $z = \perp$
13 : end	13 : end
14 : return ($w = \mathbf{w}_1, c, z = (\mathbf{z}, \mathbf{h})$)	14 : return ($w = \mathbf{w}_1, c, z = (\mathbf{z}, \mathbf{h})$)

Fig. 11: The two algorithms RealTrans and ZKSim. It is noted that ZKSim makes use of $(\mathbf{A}, \mathbf{t})$, instead of the public key $pk = (\mathbf{A}, \mathbf{t}_1)$. Nevertheless, we can still state that ZKSim employs pk without loss of generality. This is because, ZKSim is actually an artificial algorithm designed to serve the reduction algorithms in the security proof, and such reduction algorithms (e.g., adversaries attacking the AMLWE problem) are able to obtain both $\mathbf{A}$ and $\mathbf{t}$.

The above operation is exactly what ZKSim performs.

In addition to checking $\mathbf{z}$, RealTrans also needs to check the $\mathbf{r}$, $\mathbf{v}$, and $\mathbf{h}$. Here we first present the following computations:

$$\mathbf{A}\mathbf{y} - c\mathbf{s}_2 = \mathbf{A}\mathbf{z} - c\mathbf{A}\mathbf{s}_1 - c\mathbf{s}_2 = \mathbf{A}\mathbf{z} - c\mathbf{t}.$$

Based on the above equation, we have

$$\begin{aligned} \mathbf{r} &= \text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c\mathbf{s}_2 \stackrel{(a)}{=} \text{LowBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2) = \text{LowBits}(\mathbf{A}\mathbf{z} - c\mathbf{t}, 2\gamma_2), \\ \mathbf{v} &= \mathbf{r} + c\mathbf{t}_0 = \text{LowBits}(\mathbf{A}\mathbf{z} - c\mathbf{t}, 2\gamma_2) + c\mathbf{t}_0, \\ \mathbf{w}_1 &= \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) \stackrel{(b)}{=} \text{HighBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{z} - c\mathbf{t}, 2\gamma_2). \end{aligned}$$

Here (a) and (b) hold because, under the condition that $\|c\mathbf{s}_2\|_\infty \leq \beta_2^9$ and $\text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c\mathbf{s}_2 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k \subseteq \bar{S}_{\gamma_2 - \beta_2}^k$, by using Lemma 1 we can derive that $\text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c\mathbf{s}_2 = \text{LowBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2)$ and $\text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2, 2\gamma_2)$. Hence, the checks performed by Trans and ZKSim on $\mathbf{r}$, $\mathbf{v}$, $\mathbf{h}(= \text{MakeHint}(\mathbf{v}, \mathbf{w}_1, 2\gamma_2))$ are identical, and the $\mathbf{w}_1$ generated by them are also identical. This means that the outputs ($w = \mathbf{w}_1, c, z = (\mathbf{z}, \mathbf{h})$) of RealTrans and ZKSim have the same distribution. This completes the proof of Lemma 3. $\square$

A.2 SUF-CMA-to-UF-NMA Reduction of Aigis-Sig⁺ in the ROM

In this section, we prove the following theorem, which shows that in the ROM the SUF-CMA security of Aigis-Sig⁺ (= FSWA[Aigis.id, H]) can be reduced to the UF-NMA security of Aigis-Sig⁺ and the CUR property of Aigis.id.

Theorem 3. *Let $\bar{p}$ and Γ_p be such that $0 < \bar{p} < 1$ and $p_{sk} \leq \bar{p}$ for all $sk \in \Gamma_p$, where p_{sk} is defined in Eq. (7). Let $\mathcal{A}$ be a SUF-CMA adversary against Aigis-Sig⁺, making q_S classical queries to the signing oracle and q_H classical queries to the random oracle H. Then we can construct an adversary $\mathcal{B}$ against the UF-NMA security of Aigis-Sig⁺, an adversary $\mathcal{C}$ against the CUR property of Aigis.id, and an adversary $\mathcal{D}$ against the AMLWE assumption such that*

$$\begin{aligned} \text{Adv}_{\text{Aigis-Sig}^+}^{\text{SUF-CMA}}(\mathcal{A}) &\leq \text{Adv}_{\text{Aigis-Sig}^+}^{\text{UF-NMA}}(\mathcal{B}) + \text{Adv}_{\text{Aigis.id}}^{\text{CUR}}(\mathcal{C}) + \frac{4q_S}{(1 - \bar{p})^2} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{D}) \\ &\quad + \frac{q_S \cdot (2q_H + q_S)}{(1 - \bar{p})^2} \cdot \epsilon + \frac{3q_S}{1 - \bar{p}} \cdot \Pr[sk \notin \Gamma_p]. \end{aligned}$$

Here $\mathcal{B}$ makes at most q_H classical queries to H, ϵ is defined in Eq. (7), the probability $\Pr[sk \notin \Gamma_p]$ is taken over $(pk, sk) \leftarrow \text{Aigis.id.Gen}$, and the running times of $\mathcal{B}$, $\mathcal{C}$ and $\mathcal{D}$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 12 to prove this theorem.

⁹ Similar to $\|c\mathbf{s}_1\|_\infty \leq \beta_1$, we have $\|c\mathbf{s}_2\|_\infty \leq \|c\|_1 \cdot \|\mathbf{s}_2\|_\infty \leq \tau \cdot \eta_2 = \beta_2$, where the last equation is guaranteed by our parameter selection.

Game $\mathbf{G}_0$: This game is the SUF-CMA security game of Aigis-Sig^+ with the adversary $\mathcal{A}$ in the ROM. Obviously, we have

$$\text{Adv}_{\text{Aigis-Sig}^+}^{\text{SUF-CMA}}(\mathcal{A}) = \Pr[1 \leftarrow \mathbf{G}_0]. \quad (8)$$

Game $\mathbf{G}_1$: This game is the same as $\mathbf{G}_0$, except that in the loop body **Query** of the signing oracle, the $\mathbf{w}$ is generated by $\mathbf{A}\mathbf{y} + \mathbf{e}$, where $\mathbf{e} \leftarrow S_{\eta_1}^k$.

To analyze the game hop $\mathbf{G}_0$ to $\mathbf{G}_1$, we introduce Lemma 4, which shows that the two signing oracles of games $\mathbf{G}_0$ and $\mathbf{G}_1$ are equivalent. Thus,

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \mathbf{G}_1]. \quad (9)$$

The proof of Lemma 4 relies on Lemma 1, which guarantees that $\mathbf{A}\mathbf{y}$ and $\mathbf{A}\mathbf{y} + \mathbf{e}$ have the same high-order bits, provided the output $(\mathbf{z}, \mathbf{h})$ of Aigis.P_2 is not $\perp$.

Lemma 4. *The signing oracles of games $\mathbf{G}_0$ and $\mathbf{G}_1$ are equivalent, i.e., for any message m , the signatures generated by them have the same distribution.*

Proof. The full proof of this lemma is given in Appendix A.5.

Game $\mathbf{G}_2$: This game is the same as $\mathbf{G}_1$, except that the loop body **Query** of the signing oracle is replaced with **Prog**. Compared with **Query**, **Prog** no longer queries $\mathbf{H}$ to get the challenge c ; instead, it directly samples $c \leftarrow \text{ChSet}$ and reprograms $\mathbf{H}$ into $\mathbf{H}[(\mathbf{w}_1, m) \rightarrow c]$ before giving its return, where $\mathbf{H}[(\mathbf{w}_1, m) \rightarrow c]$ is a new oracle identical to $\mathbf{H}$ but the return for $(\mathbf{w}_1, m)$ is c .

We analyze the game hop $\mathbf{G}_1$ to $\mathbf{G}_2$ via a standard hybrid argument. Denote the signing oracle queried in game $\mathbf{G}_2$ as Sign_2 . Then, we define a new game for $i \in \{0, 1, \dots, q_S\}$ as:

- $\mathbf{G}_2^i$: same as $\mathbf{G}_1$ except that the first i signing queries are answered by Sign_2 .

Thus, the game $\mathbf{G}_2^0$ (resp. $\mathbf{G}_2^{q_S}$) is exactly the game $\mathbf{G}_1$ (resp. $\mathbf{G}_2$). Next, for two adjacent games $\mathbf{G}_2^i$ and $\mathbf{G}_2^{i+1}$, we can prove the following lemma.

Lemma 5. *For any $i \in \{0, 1, \dots, q_S - 1\}$, we can construct an AMLWE adversary $\mathcal{A}_i$ such that $|\Pr[1 \leftarrow \mathbf{G}_2^i] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1}]| \leq \frac{2}{(1-\bar{p})^2} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i) + \frac{q_H + q_S}{(1-\bar{p})^2} \cdot \epsilon + \frac{2}{1-\bar{p}} \cdot \Pr[sk \notin \Gamma_p]$, and the running time of $\mathcal{A}_i$ is roughly the same as that of $\mathcal{A}$.*

Proof. The full proof of this lemma is given in Appendix A.6. $\square$

Define $\mathcal{D}_1$ as an AMLWE adversary that first samples $i \leftarrow \{0, 1, \dots, q_S - 1\}$ and then runs the $\mathcal{A}_i$ of Lemma 5. Obviously, we have $\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{D}_1) = 1/q_S \cdot \sum_{i=0}^{q_S-1} \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i)$. Thus, by using Lemma 5, we have

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| \\ & \leq \sum_{i=0}^{q_S-1} |\Pr[1 \leftarrow \mathbf{G}_2^i] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1}]| \\ & \leq \frac{2q_S}{(1-\bar{p})^2} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{D}_1) + \frac{q_S \cdot (q_H + q_S)}{(1-\bar{p})^2} \cdot \epsilon + \frac{2q_S}{1-\bar{p}} \cdot \Pr[sk \notin \Gamma_p]. \end{aligned} \quad (10)$$

GAMES G_0 - G_7		$H(w, m)$
1: $(pk, sk) \leftarrow \text{Aigis.id.Gen}$	// G_0 - G_7	1: if $\exists c$ s.t. $(w_1, m, c) \in D$, return c // G_6 - G_7
2: $Q = \emptyset$	// G_0 - G_7	2: return $H'(w, m)$ // G_0 - G_5
3: $D = \emptyset$	// G_6 - G_7	
4: $(m^*, c^*, (z^*, h^*)) \leftarrow \mathcal{A}^{\text{H,Sign}}(pk)$	// G_0 - G_7	Query(sk, m)
5: if $(m^*, c^*, (z^*, h^*)) \in Q$, return 0	// G_0 - G_7	1: $y \leftarrow \bar{S}_{\gamma_1}^\ell, e \leftarrow S_{\eta_1}^k$
6: if $H(w^*, m^*) \neq H'(w^*, m^*)$, return 0	// G_7	2: $w = Ay$ // G_0
7: $w^* = \text{UseHint}(h^*, Az^* - c^*t_1 \cdot 2^d, 2\gamma_2)$	// G_0 - G_7	3: $w = Ay + e$ // G_1
8: return $\llbracket c^* = H(w^*, m^*) \rrbracket$	// G_0 - G_7	4: $w_1 = \text{HighBits}(w, 2\gamma_2), c = H(w_1, m)$
9: $\wedge [\text{Aigis.id.V}(pk, w^*, c^*, (z^*, h^*)) = 1]$		5: $(z, h) \leftarrow \text{Aigis.id.P}_2(sk, w_1, c, (Ay, y))$
		6: return $(w_1, c, (z, h))$
Sign(sk, m)		Prog(sk, m)
1: while $(z, h) = \perp$ do		1: $y \leftarrow \bar{S}_{\gamma_1}^\ell, e \leftarrow S_{\eta_1}^k, w = Ay + e$
2: $(w_1, c, (z, h)) \leftarrow \text{Query}(sk, m)$	// G_0 - G_1	2: $w_1 = \text{HighBits}(w, 2\gamma_2), c \leftarrow \text{ChSet}$
3: $(w_1, c, (z, h)) \leftarrow \text{Prog}(sk, m)$	// G_2	3: $(z, h) \leftarrow \text{Aigis.id.P}_2(sk, w_1, c, (Ay, y))$
4: $(w_1, c, (z, h)) \leftarrow \text{Trans}(sk, m)$	// G_3 - G_4	4: $H \leftarrow H[(w_1, m) \rightarrow c]$
5: end		5: return $(w_1, c, (z, h))$
6: $Q = Q \cup (m, c, (z, h))$		
7: return $(c, (z, h))$		
Sign(pk, m)		Trans(sk)
1: $(w_1, c, (z, h)) \leftarrow \text{ZKSim}(pk)$	// G_5 - G_7	1: $y \leftarrow \bar{S}_{\gamma_1}^\ell, e \leftarrow S_{\eta_1}^k$
2: $H \leftarrow H[(w_1, m) \rightarrow c]$	// G_5	2: $w = Ay + e$ // G_3
3: if $\exists c'$ such that $(w_1, m, c') \in D$	// G_6 - G_7	3: $w = Ay$ // G_4
4: $D = D \setminus (w_1, m, c')$		4: $w_1 = \text{HighBits}(w, 2\gamma_2), c \leftarrow \text{ChSet}$
5: $D = D \cup (w_1, m, c)$		5: $(z, h) \leftarrow \text{Aigis.id.P}_2(sk, w_1, c, (Ay, y))$
6: $Q = Q \cup (m, c, (z, h))$		6: if $(z, h) = \perp$, do nothing
7: return $(c, (z, h))$		7: else $H \leftarrow H[(w_1, m) \rightarrow c]$
		8: return $(w_1, c, (z, h))$

Fig. 12: The games G_0 to G_7 used in the proof of Theorem 3. In these games, the random oracle H' is simulated on-the-fly by using the record list L_H .

The running time of $\mathcal{D}_1$ is roughly the same as that of $\mathcal{A}$.

Game G_3 : This game is the same as G_2 , except that the loop body **Prog** of the signing oracle is replaced with **Trans**. Compared with **Prog**, **Trans** no longer reprograms H when $(z, h) = \perp$.

Similar to the game hop G_1 to G_2 , we analyze the game hop G_2 to G_3 via a standard hybrid argument. Denote the signing oracle queried in game G_2 (resp. G_3) as Sign_2 (resp. Sign_3). Then, we define a new game for $i \in \{0, 1, \dots, q_S\}$ as:

- G_3^i : same as G_2 except that the first i signing queries are answered by Sign_3 .

Thus, the game G_3^0 (resp. $G_3^{q_S}$) is exactly the game G_2 (resp. G_3). Next, for two adjacent games G_3^i and G_3^{i+1} , we can prove the following lemma.

Lemma 6. For any $i \in \{0, 1, \dots, q_S - 1\}$, we can construct an AMLWE adversary $\mathcal{A}_i$ such that $|\Pr[1 \leftarrow \mathbf{G}_3^i] - \Pr[1 \leftarrow \mathbf{G}_3^{i+1}]| \leq \frac{1}{1-\bar{p}} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i) + \frac{q_H}{1-\bar{p}} \cdot \epsilon + \Pr[sk \notin \Gamma_p]$, and the running time of $\mathcal{A}_i$ is roughly the same as that of $\mathcal{A}$.

Proof. The full proof of this lemma is given in Appendix A.7. $\square$

Define $\mathcal{D}_2$ as an AMLWE adversary that first samples $i \leftarrow \{0, 1, \dots, q_S - 1\}$ and then runs the $\mathcal{A}_i$ of Lemma 6. Obviously, we have $\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{D}_2) = 1/q_S \cdot \sum_{i=0}^{q_S-1} \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i)$. Thus, by using Lemma 6, we get

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{G}_2] - \Pr[1 \leftarrow \mathbf{G}_3]| \\ & \leq \sum_{i=0}^{q_S-1} |\Pr[1 \leftarrow \mathbf{G}_3^i] - \Pr[1 \leftarrow \mathbf{G}_3^{i+1}]| \\ & \leq \frac{q_S}{1-\bar{p}} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{D}_2) + \frac{q_S \cdot q_H}{1-\bar{p}} \cdot \epsilon + q_S \cdot \Pr[sk \notin \Gamma_p]. \end{aligned} \quad (11)$$

The running time of $\mathcal{D}_2$ is roughly the same as that of $\mathcal{A}$.

Game $\mathbf{G}_4$: This game is the same as $\mathbf{G}_3$, except that in the loop body **Trans** of the signing oracle, the $\mathbf{w}$ is generated as $\mathbf{A}\mathbf{y}$ rather than $\mathbf{A}\mathbf{y} + \mathbf{e}$. Similar to the analysis of the game hop $\mathbf{G}_0$ to $\mathbf{G}_1$, we can prove that the two signing oracles of games $\mathbf{G}_3$ and $\mathbf{G}_4$ are equivalent. Thus,

$$\Pr[1 \leftarrow \mathbf{G}_3] = \Pr[1 \leftarrow \mathbf{G}_4]. \quad (12)$$

Game $\mathbf{G}_5$: In this game, the $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$ used by the signing oracle is directly generated by the algorithm **ZKSim** defined in Fig. 11, and then the $\mathbf{H}$ is reprogrammed to $\mathbf{H}[(\mathbf{w}_1, m) \rightarrow c]$.

Since **Trans** only reprograms $\mathbf{H}$ when $(\mathbf{z}, \mathbf{h}) \neq \perp$, the signing oracle in game $\mathbf{G}_4$ is equivalent to running the **RealTrans** defined in Fig. 11, and reprograms $\mathbf{H}$ to $\mathbf{H}[(\mathbf{w}_1, m) \rightarrow c]$ after obtaining the output $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$. By Lemma 3, the output of **RealTrans** has the same distribution as the output of **ZKSim** used in game $\mathbf{G}_5$. Hence, game $\mathbf{G}_5$ is an equivalent rewrite of game $\mathbf{G}_4$. It follows that

$$\Pr[1 \leftarrow \mathbf{G}_4] = \Pr[1 \leftarrow \mathbf{G}_5]. \quad (13)$$

Game $\mathbf{G}_6$: In this game, a new set D is introduced, which is initially empty. After the signing oracle computes **ZKSim** and obtains $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$, the set D is updated according to $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$. Furthermore, the simulation of $\mathbf{H}$ is modified. For an input $(\mathbf{w}, m)$, it first checks whether there exists some c such that $(\mathbf{w}, m; c) \in D$. If such a c exists, $\mathbf{H}$ directly returns c .

Obviously, game $\mathbf{G}_6$ only implements the reprogramming of $\mathbf{H}$ in game $\mathbf{G}_5$ in an equivalent manner using the set D , and thus we have

$$\Pr[1 \leftarrow \mathbf{G}_5] = \Pr[1 \leftarrow \mathbf{G}_6]. \quad (14)$$

Game G₇: This game is the same as **G₆**, except that in the final verification it also checks whether $H(\mathbf{w}^*, m^*) \neq H'(\mathbf{w}^*, m^*)$, and returns 0 if the check succeeds.

Based on the definitions of games **G₆** and **G₇**, these two games are equivalent if the following event does not occur:

$$\begin{aligned} \text{Bad: } & H(\mathbf{w}^*, m^*) \neq H'(\mathbf{w}^*, m^*), (m^*, (c^*, (\mathbf{z}^*, \mathbf{h}^*))) \notin Q, \\ & \text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}^*, \mathbf{h}^*)) = 1, \text{ and } c^* = H(\mathbf{w}^*, m^*). \end{aligned}$$

Thus, we have

$$|\Pr[1 \leftarrow \mathbf{G}_6] - \Pr[1 \leftarrow \mathbf{G}_7]| \leq \Pr[\text{Bad}]. \quad (15)$$

Next, we upper bound the probability $\Pr[\text{Bad}]$. First, $H(\mathbf{w}^*, m^*) \neq H'(\mathbf{w}^*, m^*)$ and $c^* = H(\mathbf{w}^*, m^*)$ imply that the adversary $\mathcal{A}$ has previously queried the signing oracle on m^* , and the tuple $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$ obtained by computing **ZKSim** in that query satisfies $\mathbf{w}_1 = \mathbf{w}^*$ and $c = c^*$. By Lemma 3 and the correctness of **Aigis.id**, the tuple $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$ generated by **ZKSim** must pass the verification of **Aigis.id**, i.e., $\text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}, \mathbf{h})) = 1$. Furthermore, since $(m^*, (c^*, (\mathbf{z}^*, \mathbf{h}^*))) \notin Q$, the tuple $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$ generated by **ZKSim** must satisfy $(\mathbf{z}, \mathbf{h}) \neq (\mathbf{z}^*, \mathbf{h}^*)$. We thus obtain $(\mathbf{z}, \mathbf{h}) \neq (\mathbf{z}^*, \mathbf{h}^*)$ and

$$\text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}, \mathbf{h})) = 1 \wedge \text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}^*, \mathbf{h}^*)) = 1.$$

In fact, $(\mathbf{w}^*, c^*, (\mathbf{z}, \mathbf{h}))$ and $(\mathbf{w}^*, c^*, (\mathbf{z}^*, \mathbf{h}^*))$ satisfying the above conditions break the CUR property of **Aigis.id** (Definition 6). Therefore, we can turn the adversary $\mathcal{A}$ into an adversary $\mathcal{C}$ (that observes $\mathcal{A}$ and outputs the two transcripts) against the CUR property of **Aigis.id**. Here $\mathcal{C}$ has roughly the same runtime as $\mathcal{A}$ and

$$\Pr[\text{Bad}] \leq \text{Adv}_{\text{Aigis.id}}^{\text{CUR}}(\mathcal{C}). \quad (16)$$

As for the probability $\Pr[1 \leftarrow \mathbf{G}_7]$, we observe that if game **G₇** outputs 1, the forgery $(m^*, c^*, (\mathbf{z}^*, \mathbf{h}^*))$ produced by $\mathcal{A}$ must satisfy $c^* = H'(\mathbf{w}^*, m^*)$ and $\text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}^*, \mathbf{h}^*)) = 1$, where $\mathbf{w}^* = \text{UseHint}(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c^* \mathbf{t}_1 \cdot 2^d, 2\gamma_2)$. In other words, $(m^*, c^*, (\mathbf{z}^*, \mathbf{h}^*))$ can pass the verification even with respect to the non-reprogrammed random oracle H' . Therefore, $(m^*, c^*, (\mathbf{z}^*, \mathbf{h}^*))$ is also a valid signature in the UF-NMA game based on the random oracle H' . Following this fact, we construct an UF-NMA adversary $\mathcal{B}$ against **Aigis-Sig⁺** as follows:

- $\mathcal{B}$ runs the adversary $\mathcal{A}$ in game **G₇** and outputs whatever $\mathcal{A}$ outputs at the end. For queries to H , $\mathcal{B}$ responds by querying its own H' oracle.

We thus obtain

$$\Pr[1 \leftarrow \mathbf{G}_7] \leq \text{Adv}_{\text{Aigis-Sig}^+}^{\text{UF-NMA}}(\mathcal{B}). \quad (17)$$

Here $\mathcal{B}$ has roughly the same runtime as $\mathcal{A}$ and makes q_H queries to H' .

Finally, by combining Eq. (8) to Eq. (17), and folding $\mathcal{D}_1$ and $\mathcal{D}_2$ into one AMLWE adversary $\mathcal{B}$, for any $k \in \mathbb{Z}_{>0}$ we obtain

$$\begin{aligned} \text{Adv}_{\text{Aigis-Sig}^+}^{\text{SUF-CMA}}(\mathcal{A}) &\leq \text{Adv}_{\text{Aigis-Sig}^+}^{\text{UF-NMA}}(\mathcal{B}) + \text{Adv}_{\text{Aigis.id}}^{\text{CUR}}(\mathcal{C}) + \frac{4q_S}{(1 - \bar{p})^2} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{D}) \\ &\quad + \frac{q_S \cdot (2q_H + q_S)}{(1 - \bar{p})^2} \cdot \epsilon + \frac{3q_S}{1 - \bar{p}} \cdot \Pr[sk \notin \Gamma_p]. \end{aligned}$$

This completes the proof of Theorem 3. $\square$

A.3 Prove the UF-NMA Security and CUR Property

In this section, we first prove the UF-NMA security of Aigis-Sig⁺ in the ROM, and then prove the CUR property of Aigis.id in the standard model.

Theorem 4. *Let $\mathcal{A}$ be a UF-NMA adversary against Aigis-Sig⁺, making q_H classical queries to the random oracle H . Then we can construct an adversary $\mathcal{B}$ against the AMLWE assumption and an adversary $\mathcal{C}$ against the SelfTargetAMIS assumption such that*

$$\text{Adv}_{\text{Aigis-Sig}^+}^{\text{UF-NMA}}(\mathcal{A}) \leq \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}) + \text{Adv}_{n,q,k,\ell,\gamma_3+\tau \cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMIS}}(\mathcal{C}).$$

Here $\mathcal{C}$ makes at most q_H classical queries to its oracle, and the running times of $\mathcal{B}$ and $\mathcal{C}$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 13 to prove this theorem.

GAMES $\mathbf{G_0-G_3}$		$H(\mathbf{w}, m)$	
1 :	$\mathbf{A} \leftarrow R_q^{k \times \ell}, \mathbf{H}, \mathbf{G} \leftarrow \Omega_{\text{WSet} \times \mathcal{M} \rightarrow \text{ChSet}}$	// $\mathbf{G_0-G_3}$	1 : return $H(\mathbf{w}, m)$ // $\mathbf{G_0-G_2}$
2 :	$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_{\eta_1}^\ell \times S_{\eta_2}^k, \mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$	// $\mathbf{G_0}$	2 : return $G(\mathbf{w} \cdot 2^{\gamma_2}, m)$ // $\mathbf{G_3}$
3 :	$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_{\eta_1}^\ell \times S_{\eta_2}^k, \mathbf{t} \leftarrow R_q^k$	// $\mathbf{G_1-G_3}$	
4 :	$pk = (\mathbf{A}, \mathbf{t}), (\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}, d)$	// $\mathbf{G_0-G_1}$	
5 :	$pk = (\mathbf{A}, \mathbf{t}' = -\mathbf{t}), (\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}', d)$	// $\mathbf{G_2-G_3}$	
6 :	$(m^*, c^*, (\mathbf{z}^*, \mathbf{h}^*)) \leftarrow \mathcal{A}^H(pk)$	// $\mathbf{G_0-G_3}$	
7 :	$\mathbf{w}^* = \text{UseHint}(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c^* \mathbf{t}_1 \cdot 2^d, 2^{\gamma_2})$	// $\mathbf{G_0-G_3}$	
8 :	return $\llbracket c^* = H(\mathbf{w}^*, m^*) \rrbracket$	// $\mathbf{G_0-G_3}$	
9 :	$\wedge \llbracket \text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}^*, \mathbf{h}^*)) = 1 \rrbracket$		

Fig. 13: The games $\mathbf{G_0}$ to $\mathbf{G_3}$ used in the proof of Theorem 4.

Game $\mathbf{G_0}$: This game is the UF-NMA security game of Aigis-Sig⁺ with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{Aigis-Sig}^+}^{\text{UF-NMA}}(\mathcal{A}) = \Pr[1 \leftarrow \mathbf{G_0}]. \quad (18)$$

Game $\mathbf{G_1}$: This game is the same as $\mathbf{G_0}$, except that the vector $\mathbf{t}$ is uniformly sampled from R_q^k , rather than computed as $\mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. Obviously, we can construct an AMLWE adversary $\mathcal{B}$ (with roughly the same runtime as $\mathcal{A}$) such that

$$|\Pr[1 \leftarrow \mathbf{G_0}] - \Pr[1 \leftarrow \mathbf{G_1}]| \leq \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}). \quad (19)$$

Game $\mathbf{G_2}$: This game is the same as $\mathbf{G_1}$, except that the vector $\mathbf{t}$ is replaced by $\mathbf{t}' = -\mathbf{t}$. Since $-\mathbf{t}$ is still a uniformly random element over R_q^k , this is only a conceptual change and thus

$$\Pr[1 \leftarrow \mathbf{G_1}] = \Pr[1 \leftarrow \mathbf{G_2}]. \quad (20)$$

Game $\mathbf{G}_3$: This game is the same as $\mathbf{G}_2$, except that the query $(\mathbf{w}, m)$ to $\mathbf{H}$ is replied by $\mathbf{G}(\mathbf{w} \cdot 2\gamma_2 \bmod q, m)$, where $\mathbf{G}$ is the new random oracle involved in the SelfTargetAMISIS problem. Since $\mathbf{w}_1 \cdot 2\gamma_2 \bmod q \neq \mathbf{w}_2 \cdot 2\gamma_2 \bmod q$ for any $\mathbf{w}_1 \neq \mathbf{w}_2$, this is also a conceptual change and thus

$$\Pr[1 \leftarrow \mathbf{G}_2] = \Pr[1 \leftarrow \mathbf{G}_3]. \quad (21)$$

Next we upper bound the probability $\Pr[1 \leftarrow \mathbf{G}_3]$. We first construct an adversary $\mathcal{C}^{\mathbf{G}}([\mathbf{A}|\mathbf{t}])$ against the SelfTargetAMISIS assumption as follows:

- $\mathcal{C}^{\mathbf{G}}([\mathbf{A}|\mathbf{t}])$ runs $\mathcal{A}^{\mathbf{H}}(\mathbf{A}, \mathbf{t}' = -\mathbf{t})$ in game $\mathbf{G}_3$ and gets its output $(m^*, c^*, (\mathbf{z}^*, \mathbf{h}^*))$. For queries to $\mathbf{G}$, $\mathcal{C}^{\mathbf{G}}([\mathbf{A}|\mathbf{t}])$ answers by querying its own oracle $\mathbf{G}$.
- $\mathcal{C}^{\mathbf{G}}([\mathbf{A}|\mathbf{t}])$ computes $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}', d)$ and

$$\mathbf{y}' = \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 \cdot 2^d - \text{UseHints}_q(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 \cdot 2^d, 2\gamma_2) \cdot 2\gamma_2 - c^*\mathbf{t}_0,$$

and then outputs $([-\mathbf{y}', \mathbf{z}^*, c^*]^T, m^*)$

Here $\mathcal{C}$ queries $\mathbf{G}$ $q_{\mathbf{H}}$ times and has roughly the same running time as $\mathcal{A}$.

We now prove that if $\mathbf{G}_3$ returns 1, i.e., the $(m^*, c^*, (\mathbf{z}^*, \mathbf{h}^*))$ generated by $\mathcal{A}$ satisfies $\text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}^*, \mathbf{h}^*)) = 1$ and $\mathbf{H}(\mathbf{w}^*, m^*) = c^*$, where $\mathbf{w}^* = \text{UseHint}(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$, then the $([-\mathbf{y}', \mathbf{z}^*, c^*]^T, m^*)$ outputs by $\mathcal{C}$ must satisfy $\|[-\mathbf{y}', \mathbf{z}^*, c^*]^T\|_{\infty} \leq \max\{\gamma_1 - \beta_1, 2\gamma_3\}$ and $\mathbf{G}([\mathbf{I}_k|\mathbf{A}|\mathbf{t}] \cdot [-\mathbf{y}', \mathbf{z}^*, c^*]^T, m^*) = c^*$, which means that $\mathcal{C}$ solves the SelfTargetAMISIS problem. In fact, under the condition that $\text{Aigis.id.V}(pk, \mathbf{w}^*, c^*, (\mathbf{z}^*, \mathbf{h}^*)) = 1$ and $\mathbf{H}(\mathbf{w}^*, m^*) = c^*$, where $\mathbf{w}^* = \text{UseHint}(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$, we have $\|c^*\|_{\infty} = 1$, $\mathbf{z}^* \in \bar{S}_{\gamma_1 - \beta_1}^{\ell}$, and

$$\mathbf{y}^* := \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 \cdot 2^d - \text{UseHints}_q(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 \cdot 2^d, 2\gamma_2) \cdot 2\gamma_2 \in \bar{S}_{\gamma_3}^{\ell}.$$

Hence we can compute

$$\|-\mathbf{y}'\|_{\infty} = \|\mathbf{y}'\|_{\infty} = \|\mathbf{y}^* - c^*\mathbf{t}_0\|_{\infty} \leq \|\mathbf{y}^*\|_{\infty} + \|c^*\mathbf{t}_0\|_{\infty} \leq \gamma_3 + \tau \cdot 2^{d-1}.$$

As for the value $\mathbf{G}([\mathbf{I}_k|\mathbf{A}|\mathbf{t}] \cdot [-\mathbf{y}', \mathbf{z}^*, c^*]^T, m^*)$, we can compute

$$\begin{aligned} & \mathbf{G}([\mathbf{I}_k|\mathbf{A}|\mathbf{t}] \cdot [-\mathbf{y}', \mathbf{z}^*, c^*]^T, m^*) \\ &= \mathbf{G}(\mathbf{w}^* \cdot 2\gamma_2 - \mathbf{A}\mathbf{z}^* + c^*\mathbf{t}_1 \cdot 2^d + c^*\mathbf{t}_0 + \mathbf{A}\mathbf{z}^* + c^*\mathbf{t}, m^*) \\ &\stackrel{(a)}{=} \mathbf{G}(\mathbf{w}^* \cdot 2\gamma_2 - \mathbf{A}\mathbf{z}^* + c^*\mathbf{t}' + \mathbf{A}\mathbf{z}^* + c^*\mathbf{t}, m^*) \\ &= \mathbf{G}(\mathbf{w}^* \cdot 2\gamma_2, m^*) = \mathbf{H}(\mathbf{w}^*, m^*) = c^*. \end{aligned}$$

Here (a) uses $\mathbf{t}_1 \cdot 2^d + \mathbf{t}_0 = \mathbf{t}'$, which is obvious since $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t}', d)$. That is to say, we have

$$\Pr[1 \leftarrow \mathbf{G}_3] \leq \text{Adv}_{n,q,k,\ell,\gamma_3+\tau \cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMISIS}}(\mathcal{C}). \quad (22)$$

Finally, by combining Eq. (18) to Eq. (22), we obtain

$$\text{Adv}_{\text{Aigis-Sig}^+}^{\text{UF-NMA}}(\mathcal{A}) \leq \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}) + \text{Adv}_{n,q,k,\ell,\gamma_3+\tau \cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMISIS}}(\mathcal{C})$$

This completes the proof of Theorem 4. $\square$

Theorem 5. *Let $\mathcal{A}$ be an adversary against the CUR property of Aigis.id. Then we can construct an adversary $\mathcal{B}$ against the AMSIS assumption such that*

$$\text{Adv}_{\text{Aigis.id}}^{\text{CUR}}(\mathcal{A}) \leq \text{Adv}_{n,q,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMISIS}}(\mathcal{B}).$$

The running time of $\mathcal{B}$ is roughly the same as those of $\mathcal{A}$.

Proof. Here we prove that if the $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}), (\mathbf{z}', \mathbf{h}'))$ generated by $\mathcal{A}$ satisfies $(\mathbf{z}, \mathbf{h}) \neq (\mathbf{z}', \mathbf{h}')$ and

$$\text{Aigis.id.V}(pk, \mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})) = 1 \wedge \text{Aigis.id.V}(pk, \mathbf{w}_1, c, (\mathbf{z}', \mathbf{h}')) = 1, \quad (23)$$

that is, $\mathcal{A}$ breaks the CUR property of Aigis.id, then we can construct an adversary $\mathcal{B}$ solving the AMSIS problem.

Based on Eq. (23) and the definition of Aigis.id.V given in Fig. 10, we have the following equation:

$$\text{UseHint}_q(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2) = \text{UseHint}_q(\mathbf{h}', \mathbf{A}\mathbf{z}' - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2).$$

If $\mathbf{z} = \mathbf{z}'$, then by Lemma 2 we must have $\mathbf{h} = \mathbf{h}'$, which is in contradiction with that $(\mathbf{z}, \mathbf{h}) \neq (\mathbf{z}', \mathbf{h}')$. Thus, we must have that $\mathbf{z} \neq \mathbf{z}'$. By the definition of Aigis.id.V again, we have the following relations:

$$\begin{aligned} \|\mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty &\leq \gamma_3, \\ \|\mathbf{A}\mathbf{z}' - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty &\leq \gamma_3. \end{aligned}$$

By the triangular inequality, this implies that

$$\|\mathbf{A}(\mathbf{z} - \mathbf{z}')\|_\infty \leq 2\gamma_3.$$

Denote $\mathbf{u}_1 = \mathbf{z}' - \mathbf{z} \in R_q^\ell$ and $\mathbf{u}_2 = \mathbf{A}(\mathbf{z} - \mathbf{z}') \in R_q^k$. Now we have

$$\|\mathbf{u}_1\|_\infty \leq \|\mathbf{z}\|_\infty + \|\mathbf{z}'\|_\infty \leq 2\gamma_1 - 2\beta_1 \leq 2\gamma_1,$$

$\|\mathbf{u}_2\|_\infty \leq 2\gamma_3$, and $\mathbf{A}\mathbf{u}_1 + \mathbf{u}_2 = \mathbf{0}$, where $\mathbf{u}_1 = \mathbf{z}' - \mathbf{z} \neq \mathbf{0}$. This means that $(\mathbf{u}_1, \mathbf{u}_2)$ is a solution to the AMSIS problem. Therefore, we can turn the adversary $\mathcal{A}$ into an adversary $\mathcal{B}$ (that observes $\mathcal{A}$ and outputs $(\mathbf{u}_1, \mathbf{u}_2)$) against the AMSIS assumption. Here $\mathcal{B}$ has roughly the same runtime as $\mathcal{A}$ and

$$\text{Adv}_{\text{Aigis.id}}^{\text{CUR}}(\mathcal{A}) \leq \text{Adv}_{n,q,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMISIS}}(\mathcal{B}).$$

This completes the proof of Theorem 5. □

A.4 Putting Everything Together

By combining Theorem 3, Theorem 4, and Theorem 5, we can prove Theorem 1, which we restate below.

Theorem 1. *Let $\bar{p}$ and Γ_p be such that $0 < \bar{p} < 1$ and $p_{sk} \leq \bar{p}$ for all $sk \in \Gamma_p$, where p_{sk} is defined in Eq. (7). Let $\mathcal{A}$ be a SUF-CMA adversary against Aigis-Sig⁺, making q_S classical queries to the signing oracle and q_H classical queries to the random oracle H . Then we can construct two adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ against the AMLWE assumption, an adversary $\mathcal{C}$ against the SelfTargetAMSIS assumption, and an adversary $\mathcal{D}$ against the AMSIS assumption such that*

$$\begin{aligned} \text{Adv}_{\text{Aigis-Sig}^+}^{\text{SUF-CMA}}(\mathcal{A}) &\leq \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1) + \text{Adv}_{n,q,k,\ell,\gamma_3+\tau \cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMSIS}}(\mathcal{C}) \\ &\quad + \text{Adv}_{n,q,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMSIS}}(\mathcal{D}) + \frac{4q_S}{(1-\bar{p})^2} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2) \\ &\quad + \frac{q_S \cdot (2q_H + q_S)}{(1-\bar{p})^2} \cdot \epsilon + \frac{3q_S}{1-\bar{p}} \cdot \Pr[sk \notin \Gamma_p]. \end{aligned}$$

Here $\mathcal{C}$ makes at most q_H classical queries to H , ϵ is defined in Eq. (7), the probability $\Pr[sk \notin \Gamma_p]$ is taken over $(pk, sk) \leftarrow \text{Aigis.id.Gen}$, and the running times of $\mathcal{B}_1$, $\mathcal{B}_2$, $\mathcal{C}$, and $\mathcal{D}$ are roughly the same as those of $\mathcal{A}$.

A.5 Proof of Lemma 4

Proof. Denote the signing oracle queried in game $\mathbf{G}_1$ as Sign_1 , then we give the detailed computational process of Sign and Sign_1 as shown in Fig. 14.

$\text{Sign}(sk, m)$	$\text{Sign}_1(sk, m)$
<pre> 1 : while $(z, h) = \perp$ do 2 : $y \leftarrow \bar{S}_{\gamma_1}^\ell$ 3 : $w = Ay$ 4 : $w_1 = \text{HighBits}(w, 2\gamma_2)$ 5 : $c = H(w_1, m)$ 6 : $z = y + cs_1$ 7 : $r = \text{LowBits}(Ay, 2\gamma_2) - cs_2$ 8 : if $z \notin \bar{S}_{\gamma_1-\beta_1}^\ell \vee r \notin \bar{S}_{\gamma_2-\beta_2-\eta_1}^k$ 9 : $(z, h) = \perp$ 10 : else $v = r + ct_0, h = \text{MakeHint}(v, w_1, 2\gamma_2)$ 11 : if $v \notin \bar{S}_{\gamma_3}^k \vee h \notin B_{64,15,\omega}$ 12 : $(z, h) = \perp$ 13 : end 14 : end 15 : return $(c, (z, h))$ </pre>	<pre> 1 : while $(z, h) = \perp$ do 2 : $y \leftarrow \bar{S}_{\gamma_1}^\ell, e \leftarrow S_{\eta_1}^k$ 3 : $w = Ay + e$ 4 : $w_1 = \text{HighBits}(w, 2\gamma_2)$ 5 : $c = H(w_1, m)$ 6 : $z = y + cs_1$ 7 : $r = \text{LowBits}(Ay, 2\gamma_2) - cs_2$ 8 : if $z \notin \bar{S}_{\gamma_1-\beta_1}^\ell \vee r \notin \bar{S}_{\gamma_2-\beta_2-\eta_1}^k$ 9 : $(z, h) = \perp$ 10 : else $v = r + ct_0, h = \text{MakeHint}(v, w_1, 2\gamma_2)$ 11 : if $v \notin \bar{S}_{\gamma_3}^k \vee h \notin B_{64,15,\omega}$ 12 : $(z, h) = \perp$ 13 : end 14 : end 15 : return $(c, (z, h))$ </pre>
$\text{NSign}(sk, m)$	$\text{NSign}_1(sk, m)$
<pre> 1 : $y \leftarrow \bar{S}_{\gamma_1}^\ell$ 2 : $w = Ay$ 3 : $w_1 = \text{HighBits}(w, 2\gamma_2)$ 4 : $c = H(w_1, m)$ 5 : if $r = \text{LowBits}(Ay, 2\gamma_2) - cs_2 \notin \bar{S}_{\gamma_2-\beta_2-\eta_1}^k$ 6 : restart from step 1 7 : $z = y + cs_1$ 8 : $v = r + ct_0, h = \text{MakeHint}(v, w_1, 2\gamma_2)$ 9 : if $z \notin \bar{S}_{\gamma_1-\beta_1}^\ell \vee v \notin \bar{S}_{\gamma_3}^k \vee h \notin B_{64,15,\omega}$ 10 : restart from step 1 11 : return $(c, (z, h))$ </pre>	<pre> 1 : $y \leftarrow \bar{S}_{\gamma_1}^\ell, e \leftarrow S_{\eta_1}^k$ 2 : $w = Ay + e$ 3 : $w_1 = \text{HighBits}(w, 2\gamma_2)$ 4 : $c = H(w_1, m)$ 5 : if $r = \text{LowBits}(Ay, 2\gamma_2) - cs_2 \notin \bar{S}_{\gamma_2-\beta_2-\eta_1}^k$ 6 : restart from step 1 7 : $z = y + cs_1$ 8 : $v = r + ct_0, h = \text{MakeHint}(v, w_1, 2\gamma_2)$ 9 : if $z \notin \bar{S}_{\gamma_1-\beta_1}^\ell \vee v \notin \bar{S}_{\gamma_3}^k \vee h \notin B_{64,15,\omega}$ 10 : restart from step 1 11 : return $(c, (z, h))$ </pre>

Fig. 14: Four signing oracles Sign , Sign_1 , NSign , and NSign_1 used in the proof of Lemma 4. In this figure, we omit the operation " $Q = Q \cup (m, c, (z, h))$ " for simplicity.

In Fig. 14, we also present two intermediate oracles NSign and NSign_1 . Next, Based on NSign and NSign_1 , we explain the equivalence between Sign and Sign_1 .

- **Sign and NSign are equivalent.** It can be observed that NSign is an equivalent rewrite of Sign , which no longer uses a while loop; instead, it restarts directly from the first step whenever the corresponding check holds, and moves the check for $r \in \bar{S}_{\gamma_2-\beta_2-\eta_1}^k$ to the very beginning.

- **NSign and NSign₁ are equivalent.** The only difference between NSign and NSign₁ is that $\mathbf{w}_1$ in NSign₁ is the high-order bits of $\mathbf{A}\mathbf{y} + \mathbf{e}$, whereas $\mathbf{w}_1$ in NSign is the high-order bits of $\mathbf{A}\mathbf{y}$. Nevertheless, once we prove the following property, NSign and NSign₁ essentially perform the same rejection sampling on $\mathbf{r}$, which implies that NSign and NSign₁ are equivalent.

For any $\mathbf{A}, \mathbf{H}, \mathbf{s}_2, m, \mathbf{y}$, and $\mathbf{e}$, $\text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c_{\mathbf{y}} \cdot \mathbf{s}_2 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$ if and only if $\text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c_{\mathbf{y}, \mathbf{e}} \cdot \mathbf{s}_2 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, where $c_{\mathbf{y}} := \mathbf{H}(\text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2), m)$ and $c_{\mathbf{y}, \mathbf{e}} := \mathbf{H}(\text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2), m)$.

We first prove the forward direction. If $\text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c_{\mathbf{y}} \cdot \mathbf{s}_2 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, by Lemma 1 we have $\text{HighBits}(\mathbf{A}\mathbf{y} - c_{\mathbf{y}} \cdot \mathbf{s}_2, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$ and $\text{LowBits}(\mathbf{A}\mathbf{y} - c_{\mathbf{y}} \cdot \mathbf{s}_2, 2\gamma_2) \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$. Since the later implies

$$\text{LowBits}(\mathbf{A}\mathbf{y} + \mathbf{e} - c_{\mathbf{y}} \cdot \mathbf{s}_2 - \mathbf{e}, 2\gamma_2) \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k,$$

applying Lemma 1 again with the above fact yields¹⁰ $\text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y} - c_{\mathbf{y}} \cdot \mathbf{s}_2, 2\gamma_2)$. Combining this equality with $\text{HighBits}(\mathbf{A}\mathbf{y} - c_{\mathbf{y}} \cdot \mathbf{s}_2, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$ gives $\text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$, which further implies $c_{\mathbf{y}} = c_{\mathbf{y}, \mathbf{e}}$ and thus $\text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c_{\mathbf{y}, \mathbf{e}} \cdot \mathbf{s}_2 \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$. For the reverse direction, the proof is analogous and can be completed by applying Lemma 1 twice. This establishes the equivalence of NSign and NSign₁.

- **NSign₁ and Sign₁ are equivalent.** It can be observed that Sign₁ is an equivalent reformulation of NSign₁ that uses a while loop.

Combining the above three equivalences, we prove that Sign and Sign₁ are equivalent, and thus the signatures generated by them have the same distribution. This completes the proof of Lemma 4. $\square$

¹⁰ In this application, we use the property $\| -c_{\mathbf{y}} \cdot \mathbf{s}_2 - \mathbf{e} \|_{\infty} \leq \beta_2 + \eta_1$, which is satisfied in our setting.

A.6 Proof of Lemma 5

Proof. In this proof, we first consider a fixed (pk, sk) , and we use $\Pr[1 \leftarrow \mathbf{G} : (pk, sk)]$ to denote the probability that game $\mathbf{G}$ returns 1 under a fixed (pk, sk) . By the definitions of games $\mathbf{G}_2^i$ and $\mathbf{G}_2^{i+1}$, the only difference between these two games lies in the $(i+1)$ -th signing query: in $\mathbf{G}_2^i$ the adversary queries **Sign**, whereas in $\mathbf{G}_2^{i+1}$ it queries **Sign**₂. The structures of **Sign** and **Sign**₂ are shown in Fig. 15, and these two oracles use different loop bodies.

Sign ($sk, m \in \mathcal{M}$)	Sign ₂ ($sk, m \in \mathcal{M}$)
<pre> 1 : while ($\mathbf{z}, \mathbf{h}$) = $\perp$ do 2 : ($\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})$) $\leftarrow$ Query(sk, m) 3 : end 4 : $\mathbf{Q} = \mathbf{Q} \cup (m, c, (\mathbf{z}, \mathbf{h}))$ 5 : return ($c, (\mathbf{z}, \mathbf{h})$) </pre>	<pre> 1 : while ($\mathbf{z}, \mathbf{h}$) = $\perp$ do 2 : ($\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})$) $\leftarrow$ Prog(sk, m) 3 : end 4 : $\mathbf{Q} = \mathbf{Q} \cup (m, c, (\mathbf{z}, \mathbf{h}))$ 5 : return ($c, (\mathbf{z}, \mathbf{h})$) </pre>
Query ($sk, m \in \mathcal{M}$)	Prog ($sk, m \in \mathcal{M}$)
<pre> 1 : $sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$ 2 : $\mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e} \leftarrow S_{\eta_1}^k$ 3 : $\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}$ 4 : $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$ 5 : $c = \text{H}(\mathbf{w}_1, m)$ 6 : $(\mathbf{z}, \mathbf{h}) \leftarrow \text{Aigis.id.P}_2(sk, \mathbf{w}_1, c, (\mathbf{A}\mathbf{y}, \mathbf{y}))$ 7 : return ($\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})$) </pre>	<pre> 1 : $sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0)$ 2 : $\mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e} \leftarrow S_{\eta_1}^k$ 3 : $\mathbf{w} = \mathbf{A}\mathbf{y} + \mathbf{e}$ 4 : $\mathbf{w}_1 = \text{HighBits}(\mathbf{w}, 2\gamma_2)$ 5 : $c \leftarrow \text{ChSet}$ 6 : $(\mathbf{z}, \mathbf{h}) \leftarrow \text{Aigis.id.P}_2(sk, \mathbf{w}_1, c, (\mathbf{A}\mathbf{y}, \mathbf{y}))$ 7 : $\mathbf{H} \leftarrow \text{H}[(\mathbf{w}_1, m) \rightarrow c]$ 8 : return ($\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})$) </pre>

Fig. 15: The oracles **Sign** and **Sign**₂ used in the proof of Lemma 5.

We use the hybrid argument with respect to the loop bodies **Query** and **Prog** to analyze the game hop $\mathbf{G}_2^i$ to $\mathbf{G}_2^{i+1}$. First, we define a new game for $j \in \mathbb{Z}_{\geq 0}$ as:

NewG_{*j*}: A game identical to $\mathbf{G}_2^i$, except that the first *j* loop bodies during the simulation of $(i+1)$ -th signing query (answered by **Sign**) are all set as **Prog**.

The above definition indicates that game **NewG**₀ (resp. **NewG**_∞) is exactly the game $\mathbf{G}_2^i$ (resp. $\mathbf{G}_2^{i+1}$). Next, for any $k \in \mathbb{Z}_{>0}$, we can compute

$$\begin{aligned}
& \left| \Pr[1 \leftarrow \mathbf{G}_2^i : (pk, sk)] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1} : (pk, sk)] \right| \\
&= \left| \Pr[1 \leftarrow \mathbf{NewG}_0] - \Pr[1 \leftarrow \mathbf{NewG}_\infty] \right| \\
&\leq \sum_{j=0}^{k-1} \left| \Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}] \right| \\
&\quad + \left| \Pr[1 \leftarrow \mathbf{NewG}_k] - \Pr[1 \leftarrow \mathbf{NewG}_\infty] \right| \\
&\stackrel{(a)}{\leq} \sum_{j=0}^{k-1} \left| \Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}] \right| + (p'_{sk})^k,
\end{aligned} \tag{24}$$

where

$$p'_{sk} = \Pr \left[(\mathbf{z}, \mathbf{h}) = \perp : \begin{array}{l} \mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e} \leftarrow S_{\eta_1}^k, \mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2) \\ c \leftarrow \text{ChSet}, (\mathbf{z}, \mathbf{h}) \leftarrow \text{Aigis.id.P}_2(sk, \mathbf{w}_1, c, \mathbf{A}\mathbf{y}, \mathbf{y}) \end{array} \right].$$

The derivation of (a) is as follows. By the definitions of NewG_k and NewG_∞ , this two games have exactly the same computation process as long as not all of the first k loops of **Prog** in the $(i+1)$ -th signing query result in $(\mathbf{z}, \mathbf{h}) = \perp$. Note that the probability that k loops of **Prog** all result in $(\mathbf{z}, \mathbf{h}) = \perp$ is exactly $(p'_{sk})^k$, thus we have $|\Pr[1 \leftarrow \text{NewG}_k] - \Pr[1 \leftarrow \text{NewG}_\infty]| \leq (p'_{sk})^k$.

Here we prove that $p'_{sk} = p_{sk}$, where p_{sk} (defined in Eq. (7)) does not sample $\mathbf{e}$ and sets $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$. First, by the definition of Aigis.id.P_2 given in Fig. 10, whether its output is $\perp$ depends on the checks on $\mathbf{z}, \mathbf{r}, \mathbf{v}$, and $\mathbf{h}$, among which the check on $\mathbf{r}$ is

$$\mathbf{r} = \text{LowBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) - c\mathbf{s}_2 \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k.$$

If $\mathbf{y}$ makes $\mathbf{r} \notin \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, then clearly $\text{Aigis.id.P}_2(sk, \mathbf{w}_1, c, \mathbf{A}\mathbf{y}, \mathbf{y})$ returns $\perp$ for both $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$ and $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2)$. If $\mathbf{y}$ makes $\mathbf{r} \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, by a similar argument to the “NSign and NSign₁ are equivalent” in the proof Lemma 4, we have $\text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$ for any $\mathbf{e} \in S_{\eta_1}^k$. This implies that if $\mathbf{y}$ makes $\mathbf{r} \in \bar{S}_{\gamma_2 - \beta_2 - \eta_1}^k$, then $\text{Aigis.id.P}_2(sk, \mathbf{w}_1, c, \mathbf{A}\mathbf{y}, \mathbf{y})$ has the same output for both $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2)$ and $\mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2)$. Therefore, we have

$$\begin{aligned} p'_{sk} &= \Pr \left[(\mathbf{z}, \mathbf{h}) = \perp : \begin{array}{l} \mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e} \leftarrow S_{\eta_1}^k, \mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2) \\ c \leftarrow \text{ChSet}, (\mathbf{z}, \mathbf{h}) \leftarrow \text{Aigis.id.P}_2(sk, \mathbf{w}_1, c, \mathbf{A}\mathbf{y}, \mathbf{y}) \end{array} \right] \\ &= \Pr \left[(\mathbf{z}, \mathbf{h}) = \perp : \begin{array}{l} \mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{w}_1 = \text{HighBits}(\mathbf{A}\mathbf{y}, 2\gamma_2) \\ c \leftarrow \text{ChSet}, (\mathbf{z}, \mathbf{h}) \leftarrow \text{Aigis.id.P}_2(sk, \mathbf{w}_1, c, \mathbf{A}\mathbf{y}, \mathbf{y}) \end{array} \right] \quad (25) \\ &= p_{sk}. \end{aligned}$$

It can be observed that NewG_j and NewG_{j+1} differ only in the $(j+1)$ -th loop body of the $(i+1)$ -th signing query, in which NewG_j uses **Query**, while NewG_{j+1} uses **Prog**. Here we perform a conceptual change on games NewG_j and NewG_{j+1} : for the $(i+1)$ -th signing query, we assume that the randomness used in the first j **Prog** loops (denoted by $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$) and the randomness used in the $(j+1)$ -th **Query** or **Prog** loop (denoted by $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1})$) are sampled at the very beginning of the game¹¹, even if they are not all necessarily used¹². Then, to compute $|\Pr[1 \leftarrow \text{NewG}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1}]|$, we introduce four classical events as follows:

- **Bad_j**: The randomness $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes the first j **Prog** loops all produce $(\mathbf{z}, \mathbf{h}) = \perp$.
- **Bad_{j+1}**: The $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ and $\mathbf{w}_{j+1} := \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ ensure that $\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2) = \text{HighBits}(\mathbf{w}_l, 2\gamma_2)$ for some $l \in \{1, \dots, j\}$.

¹¹ The $\mathbf{y}_1, \dots, \mathbf{y}_{j+1}$ are uniformly sampled from $\bar{S}_{\gamma_1}^\ell$, $\mathbf{e}_1, \dots, \mathbf{e}_{j+1}$ are uniformly sampled from $S_{\eta_1}^k$, and $c_1, \dots, c_j$ are uniformly sampled from ChSet .

¹² For example, if a $(\mathbf{z}, \mathbf{h}) \neq \perp$ is produced in the t -th **Prog** loop for a $t < j$, the signing oracle aborts right after that, and the randomness $(\mathbf{y}_{t+1}, \mathbf{e}_{t+1}, c_{t+1}), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ and $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1})$ are never used.

- Bad_{j+1}^H : Before performing the $(i+1)$ -th signing query, the adversary $\mathcal{A}$ has ever queried H on an input $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$. Here $\mathbf{w}_{j+1} := \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$.
- Bad_{j+1}^S : Before performing the $(i+1)$ -th signing query, the H has ever been reprogrammed¹³ on $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$, where $\mathbf{w}_{j+1} := \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$.

Since the samplings of $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ and $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1})$ are independent, events Bad_j and $\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S$ are independent. Conditioned on $\text{Bad}_j \wedge (\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S \vee \text{Bad}_{j+1})$ not occurring, the simulation of the $(i+1)$ -th signing query in NewG_{j+1} either does not execute the $(j+1)$ -th loop body **Prog**, or executes it but from $\mathcal{A}$'s view, the reprogramming performed in Steps 5 and 7 of **Prog** (Fig. 15) is equivalent to directly querying $H(\mathbf{w}_1, m)$. In other words, NewG_j and NewG_{j+1} are equivalent when $\text{Bad}_j \wedge (\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S \vee \text{Bad}_{j+1})$ does not occur. Thus,

$$\begin{aligned}
& |\Pr[1 \leftarrow \text{NewG}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1}]| \\
& \leq \Pr[\text{Bad}_j \wedge (\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S \vee \text{Bad}_{j+1})] \\
& \leq \Pr[\text{Bad}_j \wedge (\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S)] + \Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}] \\
& = \Pr[\text{Bad}_j] \cdot \Pr[\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S] + \Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}] \\
& = (p'_{sk})^j \cdot \Pr[\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S] + \Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}] \\
& \stackrel{(a)}{=} (p_{sk})^j \cdot \Pr[\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S] + \Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}] \\
& = (p_{sk})^j \cdot \Pr[1 \leftarrow \text{BadG}_0] + \Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}].
\end{aligned} \tag{26}$$

Here (a) follows from Eq. (25). BadG_0 is a new game defined in Fig. 16, where BadG_0 finally returns 1 if and only if $\text{Bad}_{j+1}^H \vee \text{Bad}_{j+1}^S$ occurs.

BadG₀
1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ 3 : run $\mathcal{A}^{H, \text{Sign}_2}(pk)$ and abort just before the $(i+1)$ -th signing query 4 : if $\mathcal{A}$ ever queried H on $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$ or H was ever reprogrammed 5 : at $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$, return 1, else return 0
BadG'₀
1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), \mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e} \leftarrow S_{\eta_1}^k, c \leftarrow \text{ChSet}$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ 3 : if $\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2)$, return 1, else return 0

Fig. 16: Games BadG_0 and BadG'_0 used in the proof of Lemma 5. Here H is simulated on-the-fly.

As for the probability $\Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}]$, denote Bad_j^k ($k \in \{1, \dots, j\}$) as the event that $(\mathbf{y}_k, \mathbf{e}_k, c_k)$ causes the **Prog** loop produce $(\mathbf{z}, \mathbf{h}) = \perp$, and denote Bad_{j+1}^l

¹³ These reprogrammings occur during the simulation of the first i Sign_2 queries.

($l \in \{1, \dots, j\}$) as the event that $\text{HighBits}(\mathbf{w}_l, 2\gamma_2) = \text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2)$, where $\mathbf{w}_l := \mathbf{A}\mathbf{y}_l + \mathbf{e}_l$ and $\mathbf{w}_{j+1} := \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$. Now we have $\text{Bad}_j = \bigwedge_{k=1}^j \text{Bad}_j^k$ and $\text{Bad}_{j+1} = \bigvee_{l=1}^j \text{Bad}_{j+1}^l$, and we can compute

$$\begin{aligned}
\Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}] &\leq \sum_{l=1}^j \Pr[\text{Bad}_j \wedge \text{Bad}_{j+1}^l] \\
&= \sum_{l=1}^j \Pr[(\bigwedge_{k=1}^j \text{Bad}_j^k) \wedge \text{Bad}_{j+1}^l] \\
&\stackrel{(a)}{=} \sum_{l=1}^j (p'_{sk})^{j-1} \Pr[\text{Bad}_j^l \wedge \text{Bad}_{j+1}^l] \\
&\stackrel{(b)}{\leq} \sum_{l=1}^j (p'_{sk})^{j-1} \cdot \Pr[1 \leftarrow \mathbf{BadG}'_0] \\
&= j(p'_{sk})^{j-1} \cdot \Pr[1 \leftarrow \mathbf{BadG}'_0] \\
&\stackrel{(c)}{=} j(p_{sk})^{j-1} \cdot \Pr[1 \leftarrow \mathbf{BadG}'_0].
\end{aligned} \tag{27}$$

Here (a) uses the fact that $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ and $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1})$ are independently sampled, and (b) follows the definition of $\mathbf{BadG}'_0$ given in Fig. 16, which is a new game s.t. $\Pr[1 \leftarrow \mathbf{BadG}'_0] = \Pr[\text{Bad}_{j+1}^l]$ for any $l \in \{1, \dots, j\}$. (c) follows from Eq. (25).

We point out that although $\mathbf{BadG}_0$ and $\mathbf{BadG}'_0$ use $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, \mathbf{w}_{j+1})$ with subscript $j+1$, this is merely a notation inherited from the previous proof, and $\mathbf{BadG}_0$ is in fact independent of j . Hence, by combining Eq. (24) to Eq. (27), for any $k \in \mathbb{Z}_{>0}$,

$$\begin{aligned}
&|\Pr[1 \leftarrow \mathbf{G}_2^i : (pk, sk)] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1} : (pk, sk)]| \\
&\leq \sum_{j=0}^{k-1} (p_{sk})^j \cdot \Pr[1 \leftarrow \mathbf{BadG}_0] + \sum_{j=0}^{k-1} j(p_{sk})^{j-1} \cdot \Pr[1 \leftarrow \mathbf{BadG}'_0] + (p_{sk})^k
\end{aligned}$$

By letting $k \rightarrow \infty$, the $(p_{sk})^k$ term vanishes and thus we obtain

$$\begin{aligned}
&|\Pr[1 \leftarrow \mathbf{G}_2^i : (pk, sk)] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1} : (pk, sk)]| \\
&\leq \frac{1}{1 - p_{sk}} \cdot \Pr[1 \leftarrow \mathbf{BadG}_0] + \frac{1}{(1 - p_{sk})^2} \Pr[1 \leftarrow \mathbf{BadG}'_0].
\end{aligned}$$

Then, by using the $(\bar{p}, \Gamma_p)$ defined in Theorem 3, we have

$$\begin{aligned}
&|\Pr[1 \leftarrow \mathbf{G}_2^i] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1}]| \\
&\leq \frac{\mathbb{E}[\Pr[1 \leftarrow \mathbf{BadG}_0]]}{1 - \bar{p}} + \frac{\mathbb{E}[\Pr[1 \leftarrow \mathbf{BadG}'_0]]}{(1 - \bar{p})^2} + \Pr[sk \notin \Gamma_p] \\
&\stackrel{(a)}{=} \frac{1}{1 - \bar{p}} \cdot \Pr[1 \leftarrow \mathbf{BadG}_1] + \frac{1}{(1 - \bar{p})^2} \cdot \Pr[1 \leftarrow \mathbf{BadG}'_1] + \Pr[sk \notin \Gamma_p].
\end{aligned} \tag{28}$$

Here the expectation and the probability $\Pr[sk \notin \Gamma_p]$ are both taken over $(pk, sk) \leftarrow \text{Aigis.id.Gen.}$ (a) follows from the definitions of $\mathbf{BadG}_1$ and $\mathbf{BadG}'_1$ given in Fig. 17, where $\mathbf{BadG}_1$ (resp. $\mathbf{BadG}'_1$) is the same as $\mathbf{BadG}_0$ (resp. $\mathbf{BadG}'_0$) except that the (pk, sk) is sampled by Aigis.id.Gen.

BadG₁
1 : $(pk, sk) \leftarrow \text{Aigis.id.Gen}, pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ 3 : run $\mathcal{A}^{\text{H, Sign}_2}(pk)$ and abort just before the $(i+1)$ -th signing query 4 : if $\mathcal{A}$ ever queried H on $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$ or H was ever reprogrammed 5 : at $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$, return 1, else return 0
BadG'₁
1 : $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ 2 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), \mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e} \leftarrow S_{\eta_1}^k, c \leftarrow \text{ChSet}$ 3 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ 4 : if $\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2)$, return 1, else return 0

Fig. 17: Games $\mathbf{BadG}_1$ and $\mathbf{BadG}'_1$ used in the proof of Lemma 5. Here H is simulated on-the-fly.

In Fig. 18 we also define game $\mathbf{BadG}_2$ (resp. $\mathbf{BadG}'_2$), which is the same as $\mathbf{BadG}_1$ (resp. $\mathbf{BadG}'_1$) except that the $\mathbf{w}_{j+1}$ is uniformly sampled from R_q^k . Then, we construct AMLWE adversaries $\mathcal{B}, \mathcal{B}'$ as follows:

- $\mathcal{B}$ runs $\mathbf{BadG}_1$, replaces the $(\mathbf{A}, \mathbf{w}_{j+1})$ by its AMLWE challenge $(\mathbf{A}, \mathbf{w})$, and outputs the final return of $\mathbf{BadG}_1$.
- $\mathcal{B}'$ runs $\mathbf{BadG}'_1$, replaces the $(\mathbf{A}, \mathbf{w}_{j+1})$ by its AMLWE challenge $(\mathbf{A}, \mathbf{w})$, and outputs the final return of $\mathbf{BadG}'_1$.

Notice that the $\mathbf{w}$ of the AMLWE challenge $(\mathbf{A}, \mathbf{w})$ either equals to $\mathbf{A}\mathbf{y} + \mathbf{e}$ with $\mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell$ and $\mathbf{e} \leftarrow S_{\eta_1}^k$, or is uniformly sampled from R_q^k . At this point, the running time of $\mathcal{B}$ and $\mathcal{B}'$ are roughly the same as those of $\mathcal{A}$ and we can conclude that

$$\begin{aligned}
|\Pr[1 \leftarrow \mathbf{BadG}_1] - \Pr[1 \leftarrow \mathbf{BadG}_2]| &= \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}), \\
|\Pr[1 \leftarrow \mathbf{BadG}'_1] - \Pr[1 \leftarrow \mathbf{BadG}'_2]| &= \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}').
\end{aligned} \tag{29}$$

As for the probability $\Pr[1 \leftarrow \mathbf{BadG}_2]$, by using the parameter ϵ defined in Eq. (7) and the fact that $\mathcal{A}$ queries H at most q_H times, we have

$$\begin{aligned}
\Pr[1 \leftarrow \mathbf{BadG}_2] &\leq \Pr[1 \leftarrow \mathbf{BadG}_2 : sk \in \Gamma_p] + \Pr[sk \notin \Gamma_p] \\
&\leq \left(q_H + \frac{i}{1 - \bar{p}} \right) \cdot \epsilon + \Pr[sk \notin \Gamma_p].
\end{aligned} \tag{30}$$

Here $i/(1 - \bar{p})$ denotes the upper bound of the expected number of reprogramming times during the simulation of the first i Sign_2 queries. Similarly, we have

$$\Pr[1 \leftarrow \mathbf{BadG}'_2] \leq \epsilon. \tag{31}$$

BadG₂
1 : $(pk, sk) \leftarrow \text{Aigis.id.Gen}, pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), \mathbf{Q} = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} \leftarrow R_q^k$ 3 : run $\mathcal{A}^{\text{H, Sign}_2}(pk)$ and abort just before the $(i+1)$ -th signing query 4 : if $\mathcal{A}$ ever queried H on $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$ or H was ever reprogrammed 5 : at $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$, return 1, else return 0
BadG'₂
1 : $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ 2 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), \mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e} \leftarrow S_{\eta_1}^k, c \leftarrow \text{ChSet}$ 3 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} \leftarrow R_q^k$ 4 : if $\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2) = \text{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{e}, 2\gamma_2)$, return 1, else return 0

Fig. 18: Games **BadG₂** and **BadG'₂** used in the proof of Lemma 5. Here H is simulated on-the-fly.

Define $\mathcal{A}_i$ as an AMLWE adversary that runs a uniformly sampled adversary from $\{\mathcal{B}, \mathcal{B}'\}$. Obviously, we have

$$2 \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i) = \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}) + \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}'). \quad (32)$$

Finally, by combining Eq. (28) to Eq. (32) with the fact $i < q_S$, for any $k \in \mathbb{Z}_{>0}$

$$\begin{aligned}
|\Pr[1 \leftarrow \mathbf{G}_2^i] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1}]| &\leq \frac{2}{(1 - \bar{p})^2} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i) + \frac{q_H + q_S}{(1 - \bar{p})^2} \cdot \epsilon \\
&\quad + \frac{2}{1 - \bar{p}} \cdot \Pr[sk \notin \Gamma_p].
\end{aligned}$$

This completes the proof of Lemma 5. $\square$

A.7 Proof of Lemma 6

Proof. The proof idea of this lemma is similar to that of Lemma 5; namely, we first consider a fixed (pk, sk) , and then analyze the difference between $\mathbf{G}_3^i$ and $\mathbf{G}_3^{i+1}$ using a hybrid argument based on the loop bodies **Prog** and **Trans**.

$\text{Sign}_2(sk, m \in \mathcal{M})$	$\text{Sign}_3(sk, m \in \mathcal{M})$
<pre> 1 : while (z, h) = ⊥ do 2 : (w₁, c, (z, h)) ← Prog(sk) 3 : end 4 : Q = Q ∪ (m, c, (z, h)) 5 : return (c, (z, h)) </pre>	<pre> 1 : while (z, h) = ⊥ do 2 : (w₁, c, (z, h)) ← Trans(sk) 3 : end 4 : Q = Q ∪ (m, c, (z, h)) 5 : return (c, (z, h)) </pre>
$\text{Prog}(sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), m \in \mathcal{M})$	$\text{Trans}(sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), m \in \mathcal{M})$
<pre> 1 : y ← $\bar{S}_{\gamma_1}^\ell$, e ← $S_{\eta_1}^k$, w = Ay + e 2 : w₁ = HighBits(w, 2γ₂), c ← ChSet 3 : (z, h) ← Aigis.id.P₂(sk, w₁, c, (Ay, y)) 4 : H ← H[(w₁, m) → c] 5 : return (w₁, c, (z, h)) </pre>	<pre> 1 : y ← $\bar{S}_{\gamma_1}^\ell$, e ← $S_{\eta_1}^k$, w = Ay + e 2 : w₁ = HighBits(w, 2γ₂), c ← ChSet 3 : (z, h) ← Aigis.id.P₂(sk, w₁, c, (Ay, y)) 4 : if (z, h) = ⊥, do nothing 5 : else H ← H[(w₁, m) → c] 6 : return (w₁, c, (z, h)) </pre>

Fig. 19: The oracles Sign_2 and Sign_3 used in the proof of Lemma 6.

By the definitions of games $\mathbf{G}_3^i$ and $\mathbf{G}_3^{i+1}$, the only difference between these two games lies in the $(i+1)$ -th signing query: in $\mathbf{G}_3^i$ the adversary queries Sign_2 , whereas in $\mathbf{G}_3^{i+1}$ it queries Sign_3 . The structures of Sign_2 and Sign_3 are shown in Fig. 19, and these two oracles use different loop bodies **Prog** and **Trans**. In the following, we use the hybrid argument with respect to **Prog** and **Trans** to analyze the game hop $\mathbf{G}_3^i$ to $\mathbf{G}_3^{i+1}$. First, we define a new game for $j \in \mathbb{Z}_{\geq 0}$ as:

NewG_j: A game identical to $\mathbf{G}_3^i$, except that the first j loop bodies during the simulation of $(i+1)$ -th signing query (answered by Sign_2) are all set as **Trans**.

The above definition indicates that game **NewG₀** (resp. **NewG_∞**) is exactly the game $\mathbf{G}_3^i$ (resp. $\mathbf{G}_3^{i+1}$). Next, for any $k \in \mathbb{Z}_{>0}$, we can compute

$$\begin{aligned}
& \left| \Pr[1 \leftarrow \mathbf{G}_3^i : (pk, sk)] - \Pr[1 \leftarrow \mathbf{G}_3^{i+1} : (pk, sk)] \right| \\
&= \left| \Pr[1 \leftarrow \mathbf{NewG}_0] - \Pr[1 \leftarrow \mathbf{NewG}_\infty] \right| \\
&\leq \sum_{j=0}^{k-1} \left| \Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}] \right| \\
&\quad + \left| \Pr[1 \leftarrow \mathbf{NewG}_k] - \Pr[1 \leftarrow \mathbf{NewG}_\infty] \right| \\
&\stackrel{(a)}{\leq} \sum_{j=0}^{k-1} \left| \Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}] \right| + (p'_{sk})^k \\
&\stackrel{(b)}{=} \sum_{j=0}^{k-1} \left| \Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}] \right| + (p_{sk})^k.
\end{aligned} \tag{33}$$

The derivation of (a) is similar to that in Eq. (24), and (b) follows from Eq. (25).

It can be observed that $\mathbf{NewG}_j$ and $\mathbf{NewG}_{j+1}$ differ only in the $(j+1)$ -th loop body of the $(i+1)$ -th signing query, in which $\mathbf{NewG}_j$ uses **Prog**, while $\mathbf{NewG}_{j+1}$ uses **Trans**. Here we perform a conceptual change on games $\mathbf{NewG}_j$ and $\mathbf{NewG}_{j+1}$: for the $(i+1)$ -th signing query, we assume that the randomness used in the first j **Trans** loops (denoted by $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$) and the randomness used in the $(j+1)$ -th **Prog** or **Trans** loop (denoted by $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$) are sampled at the very beginning of the game¹⁴, even if they are not all necessarily used¹⁵. Then, to compute $|\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]|$, we introduce three classical events as follows:

- $\mathbf{Bad}_j$: The randomness $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes the first j **Trans** loops all produce $(\mathbf{z}, \mathbf{h}) = \perp$.
- $\mathbf{Bad}_{j+1}$: The randomness $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the $(j+1)$ -th **Prog/Trans** loop produces $(\mathbf{z}, \mathbf{h}) = \perp$.
- $\mathbf{Bad}_{j+1}^{\mathbf{H},*}$: During the execution, the adversary $\mathcal{A}$ has ever queried $\mathbf{H}$ on an input $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$, where $\mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$.

We stress that the event $\mathbf{Bad}_{j+1}^{\mathbf{H},*}$ concerns all queries to $\mathbf{H}$ made by $\mathcal{A}$, rather than only those queries made before the $(i+1)$ -th signing query as in the proof of Lemma 5. Hence, we cannot directly say that events $\mathbf{Bad}_j$ and $\mathbf{Bad}_{j+1}^{\mathbf{H},*}$ are independent.

Conditioned on $\mathbf{Bad}_j \wedge \mathbf{Bad}_{j+1} \wedge \mathbf{Bad}_{j+1}^{\mathbf{H},*}$ not occurring, the simulation of the $(i+1)$ -th signing query either does not execute the $(j+1)$ -th loop, or executes it with the reprogramming performed in **Prog** and **Trans** being identical, or executes it with the reprogramming in **Prog** being unnoticed by $\mathcal{A}$, and thus **Prog** is equivalent to **Trans**. In other words, $\mathbf{NewG}_j$ and $\mathbf{NewG}_{j+1}$ are equivalent when $\mathbf{Bad}_j \wedge \mathbf{Bad}_{j+1} \wedge \mathbf{Bad}_{j+1}^{\mathbf{H},*}$ does not occur. We therefore have

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]| \\
& \leq \Pr[\mathbf{Bad}_j \wedge \mathbf{Bad}_{j+1} \wedge \mathbf{Bad}_{j+1}^{\mathbf{H},*}] \\
& = \Pr[\mathbf{Bad}_j] \cdot \Pr[\mathbf{Bad}_{j+1} \wedge \mathbf{Bad}_{j+1}^{\mathbf{H},*} | \mathbf{Bad}_j] \\
& = (p'_{sk})^j \cdot \Pr[\mathbf{Bad}_{j+1} \wedge \mathbf{Bad}_{j+1}^{\mathbf{H},*} | \mathbf{Bad}_j] \\
& \stackrel{(a)}{=} (p_{sk})^j \cdot \Pr[\mathbf{Bad}_{j+1} \wedge \mathbf{Bad}_{j+1}^{\mathbf{H},*} | \mathbf{Bad}_j] \\
& = (p_{sk})^j \cdot \Pr[1 \leftarrow \mathbf{BadG}_0].
\end{aligned} \tag{34}$$

Here (a) follows from Eq. (25). $\mathbf{BadG}_0$ is a new game defined in Fig. 20, which returns 1 if and only if $\mathbf{Bad}_j \wedge \mathbf{Bad}_{j+1}^{\mathbf{H},*}$ occurs under the condition that $\mathbf{Bad}_j$ occurs.

We point out that since $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ cause the event $\mathbf{Bad}_j$ to occur, the $(i+1)$ -th signing query in game $\mathbf{BadG}_0$ is guaranteed to proceed to the $(j+1)$ -th **Trans** loop. Next, to compute $\Pr[1 \leftarrow \mathbf{BadG}_0]$, we introduce a sequence of new games as follows based on $\mathbf{BadG}_0$.

¹⁴ The $\mathbf{y}_1, \dots, \mathbf{y}_{j+1}$ are uniformly sampled from $\tilde{S}_{\gamma_1}^\ell$, $\mathbf{e}_1, \dots, \mathbf{e}_{j+1}$ are uniformly sampled from $S_{\eta_1}^k$, and $c_1, \dots, c_{j+1}$ are uniformly sampled from ChSet .

¹⁵ For example, if a $(\mathbf{z}, \mathbf{h}) \neq \perp$ is produced in the t -th **Trans** loop for a $t < j$, the signing oracle aborts right after that, and the randomness $(\mathbf{y}_{t+1}, \mathbf{e}_{t+1}, c_{t+1}), \dots, (\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ are never used.

BadG₀	
1 :	$pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$
2 :	sample $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes Bad_j occurs
3 :	$\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}, c_{j+1} \leftarrow \text{ChSet}$
4 :	run $\mathcal{A}^{\text{H}, \text{Sign}}(pk)$, for the first i signing queries, answered by Sign₃
5 :	for the simulation of the $(i+1)$ -th signing query, the first j loop bodies are Trans , using
6 :	$(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ as the randomness
7 :	the $(j+1)$ -th loop bodies is Trans , using $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ as the randomness, and all
8 :	subsequent loop bodies are Prog
9 :	for the $(i+2)$ -th and all subsequent signing queries, respond with Sign₂
10 :	if the $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the Prog/Trans loop produces $(\mathbf{z}, \mathbf{h}) = \perp$
11 :	and there $\exists x$ s.t. $\text{L}_H(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) \neq \perp$
12 :	return 1, else 0

Fig. 20: The game **BadG₀** used in the proof of Lemma 6. In this game, the second step can be done via reject sampling, and the random oracle **H** is simulated on-the-fly based on the record list L_H .

Game BadG₁: This game is the same as **BadG₀**, except that in the $(j+1)$ -th **Trans** loop of the $(i+1)$ -th signing query, it no longer computes $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}))$ using $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$, but directly outputs $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})) = (\perp, \perp, \perp)$.

BadG₁	
1 :	$pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$
2 :	sample $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes Bad_j occurs
3 :	$\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}, c_{j+1} \leftarrow \text{ChSet}$
4 :	run $\mathcal{A}^{\text{H}, \text{Sign}}(pk)$, for the first i signing queries, answered by Sign₃
5 :	for the simulation of the $(i+1)$ -th signing query, the first j loop bodies are Trans , using
6 :	$(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ as the randomness
7 :	the $(j+1)$ -th loop bodies is Trans but directly outputs $(\perp, \perp, \perp)$, and all
8 :	subsequent loop bodies are Prog
9 :	for the $(i+2)$ -th and all subsequent signing queries, respond with Sign₂
10 :	if the $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the Prog/Trans loop produces $(\mathbf{z}, \mathbf{h}) = \perp$
11 :	and there $\exists x$ s.t. $\text{L}_H(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) \neq \perp$
12 :	return 1, else 0

Fig. 21: The game **BadG₁** used in the proof of Lemma 6.

Note that game **BadG₀** outputting 1 implies that $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the $(j+1)$ -th **Trans** loop to produce $(\mathbf{z}, \mathbf{h}) = \perp$. By the definition of **Trans** in Fig 19, it does nothing when $(\mathbf{z}, \mathbf{h}) = \perp$, and directly outputs $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h}) = \perp)$, which causes the $(i+1)$ -th signing query to proceed to the next loop. Therefore, conditioned on $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causing the $(j+1)$ -th **Trans** loop to produce $(\mathbf{z}, \mathbf{h}) = \perp$,

we can equivalently output $(\perp, \perp, \perp)$ in the $(j+1)$ -th **Trans** loop, which also causes the $(i+1)$ -th signing query to proceed to the next loop but without executing the concrete computation of **Trans**. This implies that

$$\Pr[1 \leftarrow \mathbf{BadG}_0] = \Pr[1 \leftarrow \mathbf{BadG}_1]. \quad (35)$$

Game $\mathbf{BadG}_2$: This game is the same as $\mathbf{BadG}_1$, except that it no longer samples $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$, and directly answers the the $(i+1)$ -th and all subsequent signing queries of $\mathcal{A}$ by $\mathbf{Sign}_2$.

$\mathbf{BadG}_2$
1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \tilde{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}, c_{j+1} \leftarrow \text{ChSet}$ 3 : run $\mathcal{A}^{\mathbf{H}, \mathbf{Sign}}(pk)$, the first i signing query are answered by $\mathbf{Sign}_3$ 4 : and the remaining signing query are replied by $\mathbf{Sign}_2$ 5 : if the $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the Prog/Trans loop produces $(\mathbf{z}, \mathbf{h}) = \perp$ 6 : and there $\exists x$ s.t. $\mathbf{L}_H(\mathbf{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) \neq \perp$ 7 : return 1, else 0

Fig. 22: The game $\mathbf{BadG}_2$ used in the proof of Lemma 6.

In game $\mathbf{BadG}_1$, the randomness $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes $\mathbf{Bad}_j$ to occur, and the $(j+1)$ -th **Trans** loop of the $(i+1)$ -th signing query directly outputs $(\perp, \perp, \perp)$. Thus, the first $j+1$ **Trans** loops of the $(i+1)$ -th signing query must all yield $(\mathbf{z}, \mathbf{h}) = \perp$ and produce no signature. Since the **Trans** loop does nothing when $(\mathbf{z}, \mathbf{h}) = \perp$ (unlike **Prog**, which requires reprogramming $\mathbf{H}$), we can directly omit these $j+1$ **Trans** loops and instead expect the subsequent **Prog** loops to successfully generate a signature. In this case, there is no need to sample $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$, and the $(i+1)$ -th signing query is in fact answered by $\mathbf{Sign}_2$ (whose loop bodies are all **Prog**), which is exactly our game $\mathbf{BadG}_2$. Hence, $\mathbf{BadG}_2$ is an equivalent rewrite of $\mathbf{BadG}_1$ and

$$\Pr[1 \leftarrow \mathbf{BadG}_1] = \Pr[1 \leftarrow \mathbf{BadG}_2]. \quad (36)$$

Game $\mathbf{BadG}_3$: This game is the same as $\mathbf{BadG}_2$, except that it no longer checks whether the $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the **Trans** loop produces $(\mathbf{z}, \mathbf{h}) = \perp$. Obviously, we have

$$\Pr[1 \leftarrow \mathbf{BadG}_2] \leq \Pr[1 \leftarrow \mathbf{BadG}_3]. \quad (37)$$

We stress that although $\mathbf{BadG}_3$ uses $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, \mathbf{w}_{j+1}, c_{j+1})$ with subscript $j+1$, this is merely a notation inherited from the previous proof, and $\mathbf{BadG}_3$ is in fact independent of j . At this point, by combining Eq. (33) to Eq. (37), for any

BadG₃
1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}, c_{j+1} \leftarrow \text{ChSet}$ 3 : run $\mathcal{A}^{\text{H, Sign}}(pk)$, the first i signing query are replied by Sign_3 4 : and the remaining signing query are replied by Sign_2 5 : if $\exists x$ s.t. $\text{L}_H(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) \neq \perp$ 6 : return 1, else 0

Fig. 23: The game **BadG₃** used in the proof of Lemma 6.

$k \in \mathbb{Z}_{>0}$, we get

$$\begin{aligned} & \left| \Pr [1 \leftarrow \mathbf{G}_3^i : (pk, sk)] - \Pr [1 \leftarrow \mathbf{G}_3^{i+1} : (pk, sk)] \right| \\ & \leq \sum_{j=0}^{k-1} (p_{sk})^j \cdot \Pr [1 \leftarrow \mathbf{BadG}_3] + (p_{sk})^k. \end{aligned}$$

By letting $k \rightarrow \infty$, we get

$$\left| \Pr [1 \leftarrow \mathbf{G}_3^i : (pk, sk)] - \Pr [1 \leftarrow \mathbf{G}_3^{i+1} : (pk, sk)] \right| \leq \frac{1}{1 - p_{sk}} \cdot \Pr [1 \leftarrow \mathbf{BadG}_3].$$

Then, by averaging over $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ and using the $(\bar{p}, \Gamma_p)$ defined in Theorem 3, we can compute

$$\begin{aligned} & \left| \Pr [1 \leftarrow \mathbf{G}_3^i] - \Pr [1 \leftarrow \mathbf{G}_3^{i+1}] \right| \\ & \leq \frac{1}{1 - \bar{p}} \cdot \mathbb{E}_{(pk, sk)} [\Pr [1 \leftarrow \mathbf{BadG}_3]] + \Pr_{(pk, sk)} [sk \notin \Gamma_p] \\ & \stackrel{(a)}{=} \frac{1}{1 - \bar{p}} \cdot \Pr [1 \leftarrow \mathbf{BadG}_4] + \Pr_{(pk, sk)} [sk \notin \Gamma_p]. \end{aligned} \tag{38}$$

Here (a) follows from the definition of **BadG₄** given in Fig. 24, which is a game the same as **BadG₃**, except that the (pk, sk) is sampled by **Aigis.id.Gen**.

In Fig. 24 we also define a new game **BadG₅**, which is the same as **BadG₄**, except that the $\mathbf{w}_{j+1}$ is uniformly sampled from R_q^k . Then, based on game **BadG₄**, we construct an AMLWE adversary $\mathcal{A}_i$ as follows:

- $\mathcal{A}_i$ runs **BadG₄**, repalces the $(\mathbf{A}, \mathbf{w}_{j+1})$ by its AMLWE challenge $(\mathbf{A}, \mathbf{w})$, and outputs the final return of **BadG₄**. Here $\mathbf{w}$ either equals to $\mathbf{A}\mathbf{y} + \mathbf{e}$ with $\mathbf{y} \leftarrow \bar{S}_{\gamma_1}^\ell$ and $\mathbf{e} \leftarrow S_{\eta_1}^k$, or is uniformly sampled from R_q^k .

At this point, the running time of $\mathcal{A}_i$ is roughly the same as that of $\mathcal{A}$ and we can conclude that

$$|\Pr [1 \leftarrow \mathbf{BadG}_4] - \Pr [1 \leftarrow \mathbf{BadG}_5]| = \text{Adv}_{n, q, k, \ell, \gamma_1, \eta_1}^{\text{AMLWE}}(\mathcal{A}_i). \tag{39}$$

As for the probability $\Pr [1 \leftarrow \mathbf{BadG}_5]$, by using the parameter ϵ defined in Eq. (7) and the fact that $\mathcal{A}$ queries H at most q_H times, we have

$$\Pr [1 \leftarrow \mathbf{BadG}_5] \leq q_H \cdot \epsilon. \tag{40}$$

BadG₄
1 : $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ 2 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 3 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}, c_{j+1} \leftarrow \text{ChSet}$ 4 : run $\mathcal{A}^{\text{H,Sign}}(pk)$, the first i signing query are replied by Sign_3 5 : and the remaining signing query are replied by Sign_2 6 : if $\exists x$ s.t. $\text{L}_H(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) \neq \perp$ 7 : return 1, else 0
BadG₅
1 : $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ 2 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 3 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} \leftarrow R_q^k, c_{j+1} \leftarrow \text{ChSet}$ 4 : run $\mathcal{A}^{\text{H,Sign}}(pk)$, the first i signing query are replied by Sign_3 5 : and the remaining signing query are replied by Sign_2 6 : if $\exists x$ s.t. $\text{L}_H(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) \neq \perp$ 7 : return 1, else 0

Fig. 24: The games **BadG₄** and **BadG₅** used in the proof of Lemma 6.

Finally, by combining Eq. (38) to Eq. (40), we obtain

$$\begin{aligned}
|\Pr[1 \leftarrow \mathbf{G}_3^i] - \Pr[1 \leftarrow \mathbf{G}_3^{i+1}]| &\leq \frac{1}{1 - \bar{p}} \cdot \text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i) + \frac{q_H}{1 - \bar{p}} \cdot \epsilon \\
&\quad + \Pr_{(pk,sk)}[sk \notin \Gamma_p].
\end{aligned}$$

This completes the proof of Lemma 6. □

Appendix B Proof of Theorem 2

The QROM proof of Aigis-Sig^+ has a similar idea with its ROM proof. However, since the adversary can make quantum queries to the random oracle in the QROM, the ROM proof of Aigis-Sig^+ cannot be directly transferred to the QROM, and it is exactly the following two parts of proof prevent such transfer.

1. The upper bound on $|\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]|$ in the proof of Lemma 5 (i.e., Eq. (26)) relies on the classical event Bad_{j+1}^H , which is no longer well-defined when H is quantum-accessible.
2. The upper bound on $|\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]|$ in the proof of Lemma 6 (i.e., Eq. (34)) relies on the classical event $\text{Bad}_{j+1}^{H,*}$, which is likewise no longer well-defined when H is quantum-accessible.

In Appendix B.1, we present new proofs of Eq. (26) and Lemma 5 in the QROM. In Appendix B.2, we present new proofs of Eq. (34) and Lemma 6 in the QROM. Combining these proofs with the remaining parts of the ROM proof of Aigis-Sig^+ , we establish Theorem 2 in Appendix B.3, proving the SUF-CMA security of Aigis-Sig^+ in the QROM.

Remark 2. Notice that although Theorem 4 is proven in the ROM, its proof can be directly transferred to the QROM. The key reason is that the random oracle H simulated in the proof of Theorem 4 is stateless: the proof does not record queries to H and their corresponding responses. Thus, if we allow the SelfTargetAMSiS adversary $\mathcal{C}$ to make quantum queries its oracle G , the proof of Theorem 4 transfers directly to the QROM.

Before giving the proof, we first introduce the semi-classical oracle and some related lemmas.

Definition 7 (Semi-Classical Oracle [5]). *For a subset $S \subseteq \mathcal{X}$, the indicator function f_S from $\mathcal{X}$ to $\{0, 1\}$ equals 1 iff its input $x \in S$. The semi-classical oracle, $\mathcal{O}_S^{\text{SC}}$, is a quantum accessible oracle that performs the following two operations on its query state $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle$:*

1. *Initialize a single qubit register L with the state $|0\rangle_L$.*
2. *Transform $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |0\rangle_L$ into $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |f_S(x)\rangle_L$ and then measure L .*

During the execution of a quantum oracle algorithm $\mathcal{A}^{\mathcal{O}_S^{\text{SC}}}$, let Find be the classical event that the measurement of $\mathcal{O}_S^{\text{SC}}$ (on L) ever has outcome 1.

Lemma 7. *Let $S \subseteq \mathcal{X}$ be random. Let z be a random bitstring. (S, z may have arbitrary joint distribution $\mathfrak{D}$.) Let $\mathcal{A}$ be a quantum oracle algorithm with at most q queries (not necessarily unitary). Let*

$$P_{\text{left}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^\emptyset(z), (S, z) \leftarrow \mathfrak{D}],$$

$$P_{\text{right}} := \Pr[b = 1 \wedge \neg \text{Find} : b \leftarrow \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}].$$

Here $\emptyset$ denotes an oracle that does nothing on the query state. Let $P_{\text{find}} := \Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{\text{SC}}}(z), (S, z) \leftarrow \mathfrak{D}]$. Then

$$|P_{\text{left}} - P_{\text{right}}| \leq \sqrt{(q+1) \cdot P_{\text{find}}} \text{ and } \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq \sqrt{(q+1) \cdot P_{\text{find}}}.$$

Proof. This is a direct corollary of [5, Theorem 1] by setting the function H used by it as a constant zero function.

Lemma 8 (Search in Semi-Classical Oracle [5, Theorem 2]). *Let $S \subseteq \mathcal{X}$ be random and z be a random bitstring. (S, z) may have an arbitrary joint distribution $\mathfrak{D}$. Let $\mathcal{A}$ be any quantum oracle algorithm making at most q queries to a semi-classical oracle with domain $\mathcal{X}$. Let $\mathcal{B}$ be an algorithm that on input z , chooses $i \leftarrow_{\$} [q]$; runs $\mathcal{A}^{\mathcal{O}_{\mathfrak{D}}^{SC}}(z)$ until (just before) the i -th query, measures the input register and outputs the measurement outcome x . Then*

$$\Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_{\mathfrak{D}}^{SC}}(z), (S, z) \leftarrow \mathfrak{D}] \leq 4q \cdot \Pr[x \in S : x \leftarrow \mathcal{B}(z), (S, z) \leftarrow \mathfrak{D}].$$

B.1 New Proof of Eq. (26) and Lemma 5 in the QROM

In this section, we compute the upper bound on $|\Pr[1 \leftarrow \text{NewG}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1}]|$ in the QROM. Similar to its ROM bound computed in Eq. (26), we also need to use the following three classical events:

- Bad_j : The randomness $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes the first j Prog loops all produce $(\mathbf{z}, \mathbf{h}) = \perp$.
- Bad_{j+1} : The $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ and $\mathbf{w}_{j+1} := \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ ensure that $\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2) = \text{HighBits}(\mathbf{w}_l, 2\gamma_2)$ for some $l \in \{1, \dots, j\}$.
- Bad_{j+1}^S : Before performing the $(i+1)$ -th signing query, the H has ever been reprogrammed¹⁶ on $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$, where $\mathbf{w}_{j+1} := \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$.

If Bad_j does not occur, the $(i+1)$ -th signing query does not run the $(j+1)$ -th loop, and thus games NewG_j and NewG_{j+1} are equivalent. Therefore, we have

$$\begin{aligned} & |\Pr[1 \leftarrow \text{NewG}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1}]| \\ &= |\Pr[1 \leftarrow \text{NewG}_j \wedge \text{Bad}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1} \wedge \text{Bad}_j]| \\ &= \Pr[\text{Bad}_j] \cdot |\Pr[1 \leftarrow \text{NewG}_j | \text{Bad}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1} | \text{Bad}_j]| \\ &= (p'_{sk})^j \cdot |\Pr[1 \leftarrow \text{NewG}_j | \text{Bad}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1} | \text{Bad}_j]| \quad (41) \\ &\stackrel{(a)}{=} (p_{sk})^j \cdot |\Pr[1 \leftarrow \text{NewG}_j | \text{Bad}_j] - \Pr[1 \leftarrow \text{NewG}_{j+1} | \text{Bad}_j]| \\ &\stackrel{(b)}{=} (p_{sk})^j \cdot |\Pr[1 \leftarrow \text{NewGB}_j] - \Pr[1 \leftarrow \text{NewGB}_{j+1}]|. \end{aligned}$$

Here (a) uses Eq. (25), and (b) follows from the definition of games NewGB_j and NewGB_{j+1} given in Fig. 25, i.e., NewGB_j (resp. NewGB_{j+1}) returns 1 if and only if NewG_j (resp. NewG_{j+1}) returns 1 under the condition that Bad_j occurs. Next, based on NewGB_j , we introduce a sequence of games to compute the upper bound on $|\Pr[1 \leftarrow \text{NewGB}_j] - \Pr[1 \leftarrow \text{NewGB}_{j+1}]|$:

Game NewGB_j^a : This game is the same as NewGB_j , except that it finally checks whether the event $\text{Bad}_{j+1}^S \vee \text{Bad}_{j+1}$ occurs, and returns 0 if so. Obviously, we have

$$|\Pr[1 \leftarrow \text{NewGB}_j] - \Pr[1 \leftarrow \text{NewGB}_j^a]| \leq \Pr[\text{Bad}_{j+1}^S \vee \text{Bad}_{j+1} | \text{Bad}_j]. \quad (42)$$

¹⁶ These reprogrammings occur during the simulation of the first i Sign_2 queries of $\text{NewG}_j/\text{NewG}_{j+1}$. Since the signing oracle can be only classically accessed, these reprogramming points are all classical, and thus Bad_{j+1}^S is a well-defined classical event.

NewGB_j and NewGB_{j+1}	
1 :	$pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$
2 :	sample $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes Bad_j occurs
3 :	$\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k$
4 :	run $\mathcal{A}^{\text{H, Sign}}(pk)$, for the first i signing queries, answered by Sign₂
5 :	for the simulation of the $(i+1)$ -th signing query, the first j loop bodies are Prog
6 :	using $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ as the randomness
7 :	the $(j+1)$ -th loop bodies is Query , using $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1})$ as the randomness // NewGB_j
8 :	the $(j+1)$ -th loop bodies is Prog , using $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1})$ and a freshly sampled c_{j+1} // NewGB_{j+1}
9 :	as the randomness // NewGB_{j+1}
10 :	and all subsequent loop bodies are Query
11 :	for the $(i+2)$ -th and all subsequent signing queries, respond with Sign
12 :	if the forgery of $\mathcal{A}$ passes the verification of the SUF-CMA game of Aigis-Sig⁺
13 :	return 1, else 0

Fig. 25: Games **NewGB_j** and **NewGB_{j+1}** used in Appendix B.1. Here, the second step can be done via reject sampling, and the random oracle $\mathbf{H}$ is simulated by using a $2q_{\mathbf{H}}$ -wise independent function.

The conditioning on **Bad_j** in the probability on the right-hand side arises because Step 2 of **NewGB_j** (Fig. 25) requires that **Bad_j** occurs.

Game NewGB_j^b: Compared with **NewGB_j^a**, this new game has two changes:

1. Before the $(i+1)$ -th signing query, the $\mathbf{H}$ is replaced with $\mathbf{H} \circ \mathcal{O}_S^{SC}$, which behaves the same as $\mathbf{H}$, only with $\mathcal{O}_S^{SC}$ first queried on the input register. Here $S := \{(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) : x \in \mathcal{M}\}$.
2. **NewGB_j^b** finally checks whether the event **Find** occurs, and returns 0 if so.

By using Lemma 7, we have

$$|\Pr[\leftarrow \text{NewGB}_j^a] - \Pr[1 \leftarrow \text{NewGB}_j^b]| \leq \sqrt{(q_{\mathbf{H}} + 1) \cdot \Pr[\text{Find} : \text{NewGB}_j^b]}. \quad (43)$$

Game NewGB_j^c: This game is the same as **NewGB_j^b**, except that the $(j+1)$ -th loop of the $(i+1)$ -th signing query is changed to **Prog**, and this loop uses $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1})$ and a freshly sampled $c_{j+1} \leftarrow \text{ChSet}$ as the randomness.

Since **NewGB_j^b** returning 1 implies that neither **Find** nor **Bad_{j+1}^S** $\vee$ **Bad_{j+1}** occurs, from the adversary $\mathcal{A}$'s view, querying $\mathbf{H}(\mathbf{w}_1, m)$ is equivalent to the re-programming performed in Steps 5 and 7 of **Prog** (Fig. 15). Thus, **NewGB_j^c** is a conceptual change of **NewGB_j^b**.

$$\Pr[\leftarrow \text{NewGB}_j^b] = \Pr[1 \leftarrow \text{NewGB}_j^c]. \quad (44)$$

Game NewGB_j^d: This game is the same as **NewGB_j^c**, except that the $\mathbf{H} \circ \mathcal{O}_S^{SC}$ queried before the $(i+1)$ -th signing query is changed back to $\mathbf{H}$, and this game no

longer checks whether the event **Find** occurs. By using Lemma 7 again, we have

$$\begin{aligned} |\Pr[\leftarrow \mathbf{NewGB}_j^c] - \Pr[1 \leftarrow \mathbf{NewGB}_j^d]| &\leq \sqrt{(q_H + 1) \cdot \Pr[\mathbf{Find} : \mathbf{NewGB}_j^c]} \\ &\stackrel{(a)}{=} \sqrt{(q_H + 1) \cdot \Pr[\mathbf{Find} : \mathbf{NewGB}_j^b]}. \end{aligned} \quad (45)$$

Here (a) follows from the fact that the running process of $\mathbf{NewGB}_j^c$ is the same as that of $\mathbf{NewGB}_j^b$ before the $(i + 1)$ -th signing query.

Then, by using Lemma 8, the game $\mathbf{BadG}_0^H$ defined in Fig. 26 satisfies

$$\Pr[\mathbf{Find} : \mathbf{NewGB}_j^b] \leq 4q_H \cdot \Pr[1 \leftarrow \mathbf{BadG}_0^H]. \quad (46)$$

The parameter q used in game $\mathbf{BadG}_0^H$ denotes the number of queries to H before the $(i + 1)$ -th signing query, and satisfies $q \leq q_H$.

BadG₀^H
1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ 3 : $l \leftarrow \{1, \dots, q\}$, run $\mathcal{A}^{H, \text{Sign}_2}(pk)$ and abort just before the l -th H query 4 : measure the input register and denote the outcome as x 5 : if $x \in \{(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) : x \in \mathcal{M}\}$, return 1, else return 0
BadG₀^S
1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}$ 3 : run $\mathcal{A}^{H, \text{Sign}_2}(pk)$ and abort just before the $(i + 1)$ -th signing query 4 : if H was ever reprogrammed at $(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), *)$, return 1, else return 0

Fig. 26: Games $\mathbf{BadG}_0^H$ and $\mathbf{BadG}_0^S$ used in Appendix B.1. Here, the random oracle H is simulated by using a $2q_H$ -wise independent function.

As for the $\Pr[1 \leftarrow \mathbf{NewGB}_j^d]$, it is not hard to see that $\mathbf{NewGB}_j^d$ is identical to $\mathbf{NewGB}_{j+1}$, except that $\mathbf{NewGB}_j^d$ additionally checks whether event $\mathbf{Bad}_{j+1}^S \vee \mathbf{Bad}_{j+1}$ occurs at the end. Hence, similar to Eq. (42), we have

$$|\Pr[\leftarrow \mathbf{NewGB}_j^d] - \Pr[1 \leftarrow \mathbf{NewGB}_{j+1}]| \leq \Pr[\mathbf{Bad}_{j+1}^S \vee \mathbf{Bad}_{j+1} | \mathbf{Bad}_j] \quad (47)$$

and we can compute

$$\begin{aligned} \Pr[\mathbf{Bad}_{j+1}^S \vee \mathbf{Bad}_{j+1} | \mathbf{Bad}_j] &\leq \Pr[\mathbf{Bad}_{j+1}^S | \mathbf{Bad}_j] + \Pr[\mathbf{Bad}_{j+1} | \mathbf{Bad}_j] \\ &\stackrel{(a)}{=} \Pr[\mathbf{Bad}_{j+1}^S] + \Pr[\mathbf{Bad}_{j+1} | \mathbf{Bad}_j] \\ &\stackrel{(b)}{=} \Pr[1 \leftarrow \mathbf{BadG}_0^S] + \Pr[\mathbf{Bad}_{j+1} | \mathbf{Bad}_j]. \end{aligned} \quad (48)$$

Here (a) uses the fact that $\mathbf{Bad}_{j+1}^S$ and $\mathbf{Bad}_j$ are independent, and (b) follows from the definition of games $\mathbf{BadG}_0^S$ given in Fig. 26, i.e., $\mathbf{BadG}_0^S$ returns 1 if and only if $\mathbf{Bad}_{j+1}^S$ returns 1. Finally, by combining Eq. (41) to Eq. (48), we obtain

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]| \\
&= (p_{sk})^j \cdot |\Pr[1 \leftarrow \mathbf{NewGB}_j] - \Pr[1 \leftarrow \mathbf{NewGB}_{j+1}]| \\
&\leq (p_{sk})^j \cdot \left(4(q_H + 1) \cdot \sqrt{\Pr[1 \leftarrow \mathbf{BadG}_0^H]} + 2 \Pr[1 \leftarrow \mathbf{BadG}_0^S] \right) \\
&\quad + 2(p_{sk})^j \cdot \Pr[\mathbf{Bad}_{j+1} | \mathbf{Bad}_j] \\
&= (p_{sk})^j \cdot \left(4(q_H + 1) \cdot \sqrt{\Pr[1 \leftarrow \mathbf{BadG}_0^H]} + 2 \Pr[1 \leftarrow \mathbf{BadG}_0^S] \right) \\
&\quad + 2 \cdot \Pr[\mathbf{Bad}_{j+1} \wedge \mathbf{Bad}_j].
\end{aligned} \tag{49}$$

In fact, Eq. (49) is exactly the QROM counterpart of Eq. (26). Roughly speaking, to handle the adversary's quantum queries to H , our proof above splits the game $\mathbf{BadG}_0$ used in Eq. (26) into two games $\mathbf{BadG}_0^H$ and $\mathbf{BadG}_0^S$, which are used to handle the quantum queries to H and the reprogramming of H before the $(i+1)$ -th signing query, respectively. Since $\mathbf{BadG}_0^H$ is derived using the semi-classical oracle technique, the probability $\Pr[1 \leftarrow \mathbf{BadG}_0^H]$ at the end of Eq. (49) requires a square-root factor, which incurs a larger security loss compared to Eq. (26), where no square root is needed. At this point, by using Eq. (49) in place of Eq. (26) and following the proof strategy of Lemma 5 after Eq. (26), we can construct an AMLWE adversary $\mathcal{A}_i$ such that

$$\begin{aligned}
|\Pr[1 \leftarrow \mathbf{G}_2^i] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1}]| &\leq \frac{(12q_H + 24)}{(1 - \bar{p})^2} \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i)} \\
&\quad + \frac{4q_H + 2q_S + 6}{(1 - \bar{p})^2} \cdot \sqrt{\epsilon} + \frac{3}{1 - \bar{p}} \cdot \Pr[sk \notin \Gamma_p].
\end{aligned}$$

Here $\mathcal{A}_i$ has roughly the same running time as $\mathcal{A}$. Based on the above formula, the QROM counterpart of Lemma 5 can be restated as follows:

Lemma 5. *For any $i \in \{0, 1, \dots, q_S - 1\}$, we can construct an AMLWE adversary $\mathcal{A}_i$ such that $|\Pr[1 \leftarrow \mathbf{G}_2^i] - \Pr[1 \leftarrow \mathbf{G}_2^{i+1}]| \leq \frac{(12q_H + 24)}{(1 - \bar{p})^2} \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i)} + \frac{4q_H + 2q_S + 6}{(1 - \bar{p})^2} \cdot \sqrt{\epsilon} + \frac{3}{1 - \bar{p}} \cdot \Pr[sk \notin \Gamma_p]$, and the running time of $\mathcal{A}_i$ is roughly the same as that of $\mathcal{A}$.*

B.2 New Proof of Eq. (34) and Lemma 6 in the QROM

In this section, we compute the upper bound on $|\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]|$ in the QROM. Similar to its ROM bound computed in Eq. (34), we also need to use the following two classical events:

- $\mathbf{Bad}_j$: The randomness $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes the first j **Prog** loops all produce $(\mathbf{z}, \mathbf{h}) = \perp$.

- **Bad**_{*j*+1}: The randomness $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the $(j+1)$ -th **Prog**/**Trans** loop produces $(\mathbf{z}, \mathbf{h}) = \perp$.

Conditioned on **Bad**_{*j*} $\wedge$ **Bad**_{*j*+1} not occurring, the simulation of the $(i+1)$ -th signing query either does not execute the $(j+1)$ -th loop, or executes it with the reprogramming performed in **Prog** and **Trans** being identical. Thus, **NewG**_{*j*} and **NewG**_{*j*+1} are equivalent when **Bad**_{*j*} $\wedge$ **Bad**_{*j*+1} does not occur. We hence have

$$\begin{aligned}
& |\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]| \\
&= |\Pr[1 \leftarrow \mathbf{NewG}_j \wedge \mathbf{Bad}_j \wedge \mathbf{Bad}_{j+1}] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1} \wedge \mathbf{Bad}_j \wedge \mathbf{Bad}_{j+1}]| \\
&= \Pr[\mathbf{Bad}_j] \cdot \left| \frac{\Pr[1 \leftarrow \mathbf{NewG}_j \wedge \mathbf{Bad}_{j+1} | \mathbf{Bad}_j]}{\Pr[1 \leftarrow \mathbf{NewG}_{j+1} \wedge \mathbf{Bad}_{j+1} | \mathbf{Bad}_j]} \right| \\
&= (p'_{sk})^j \cdot \left| \frac{\Pr[1 \leftarrow \mathbf{NewG}_j \wedge \mathbf{Bad}_{j+1} | \mathbf{Bad}_j]}{\Pr[1 \leftarrow \mathbf{NewG}_{j+1} \wedge \mathbf{Bad}_{j+1} | \mathbf{Bad}_j]} \right| \tag{50} \\
&\stackrel{(a)}{=} (p_{sk})^j \cdot \left| \frac{\Pr[1 \leftarrow \mathbf{NewG}_j \wedge \mathbf{Bad}_{j+1} | \mathbf{Bad}_j]}{\Pr[1 \leftarrow \mathbf{NewG}_{j+1} \wedge \mathbf{Bad}_{j+1} | \mathbf{Bad}_j]} \right| \\
&\stackrel{(b)}{=} (p_{sk})^j \cdot |\Pr[1 \leftarrow \mathbf{NewGB}_j] - \Pr[1 \leftarrow \mathbf{NewGB}_{j+1}]|.
\end{aligned}$$

Here (a) uses Eq. (25), and (b) follows from the definition of games **NewGB**_{*j*} and **NewGB**_{*j*+1} given in Fig. 27, i.e., **NewGB**_{*j*} (resp. **NewGB**_{*j*+1}) returns 1 if and only if **NewG**_{*j*} (resp. **NewG**_{*j*+1}) returns 1 and **Bad**_{*j*+1} occurs under the condition that **Bad**_{*j*} $\wedge$ **Bad**_{*j*+1} occurs. Next, based on **NewGB**_{*j*}, we introduce a sequence of games to compute the upper bound on $|\Pr[1 \leftarrow \mathbf{NewGB}_j] - \Pr[1 \leftarrow \mathbf{NewGB}_{j+1}]|$:

NewGB_{*j*} and **NewGB**_{*j*+1}

- 1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), \mathbf{Q} = \emptyset$
- 2 : sample $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ causes **Bad**_{*j*} occurs
- 3 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}, c_{j+1} \leftarrow \text{ChSet}$
- 4 : run $\mathcal{A}^{\text{H,Sign}}(pk)$, for the first i signing queries, answered by **Sign**₃
- 5 : for the simulation of the $(i+1)$ -th signing query, the first j loop bodies are
- 6 : **Trans**, using $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ as the randomness
- 7 : the $(j+1)$ -th loop bodies is **Prog**, using $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ as the randomness // **NewGB**_{*j*}
- 8 : the $(j+1)$ -th loop bodies is **Trans**, using $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ as the randomness // **NewGB**_{*j*+1}
- 9 : and all subsequent loop bodies are **Prog**
- 10 : for the $(i+2)$ -th and all subsequent signing queries, respond with **Sign**₂
- 11 : if the $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes the **Prog**/**Trans** loop produces $(\mathbf{z}, \mathbf{h}) = \perp$
- 12 : (i.e., **Bad**_{*j*+1} occurs) and the forgery of $\mathcal{A}$ passes the
- 13 : verification of the SUF-CMA game of Aigis-Sig⁺, return 1, else 0

Fig. 27: Games **NewGB**_{*j*} and **NewGB**_{*j*+1} used in Appendix B.2. Here, the second step can be done via reject sampling, and the random oracle **H** is simulated by using a $2q_{\text{H}}$ -wise independent function.

Game NewGB_j^a : This game is the same as NewGB_j , except that in the $(j+1)$ -th **Prog** loop of the $(i+1)$ -th signing query, it first reprograms $\mathbf{H}$ to

$$\mathbf{H}[(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), m) \rightarrow c_{j+1}]$$

and then directly sets $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})) = (\perp, \perp, \perp)$, where m denotes the $(i+1)$ -th signing query of $\mathcal{A}$.

For game NewGB_j , by the definition of **Prog** given in Fig. 15, if $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes event Bad_{j+1} to occur, then the operations performed in the $(j+1)$ -th **Prog** loop of the $(i+1)$ -th signing query are equivalent to those in game NewGB_j^a . Notice that NewGB_j returning 1 implies that Bad_{j+1} occurs, and hence games NewGB_j^a and NewGB_j are equivalent.

$$\Pr[\leftarrow \text{NewGB}_j^a] = \Pr[1 \leftarrow \text{NewGB}_j]. \quad (51)$$

Game NewGB_j^b : In this game, before the $(i+1)$ -th signing query, the $\mathbf{H}$ is replaced by $\mathbf{H} \circ \mathcal{O}_S^{SC}$, which is the same as $\mathbf{H}$ except that the $\mathcal{O}_S^{SC}$ is first queried on the input registers. Here $S := \{(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) : x \in \mathcal{M}\}$. In addition, NewGB_j^b finally checks whether the event **Find** occurs, and returns 0 if so.

By using Lemma 7, we have

$$|\Pr[\leftarrow \text{NewGB}_j^b] - \Pr[1 \leftarrow \text{NewGB}_j^a]| \leq \sqrt{(q_H + 1) \cdot \Pr[\text{Find} : \text{NewGB}_j^b]}. \quad (52)$$

Game NewGB_j^c : This game is the same as NewGB_j^b , except that in the $(j+1)$ -th **Prog** loop of the $(i+1)$ -th signing query, it no longer reprograms $\mathbf{H}$ and only sets $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})) = (\perp, \perp, \perp)$.

Since NewGB_j^b returning 1 implies that **Find** does not occur, the reprogramming of $\mathbf{H}$ performed in the $(j+1)$ -th loop of **Prog** cannot be noticed by $\mathcal{A}$. Thus, NewGB_j^c is a conceptual change of NewGB_j^b .

$$\begin{aligned} \Pr[\leftarrow \text{NewGB}_j^c] &= \Pr[1 \leftarrow \text{NewGB}_j^b], \\ \Pr[\text{Find} : \text{NewGB}_j^c] &= \Pr[\text{Find} : \text{NewGB}_j^b]. \end{aligned} \quad (53)$$

Game NewGB_j^d : This game is the same as NewGB_j^c , except that the $\mathbf{H} \circ \mathcal{O}_S^{SC}$ queried before the $(i+1)$ -th signing query is changed back to $\mathbf{H}$, and this game no longer checks whether the event **Find** occurs.

By using Lemma 7 again, we have

$$|\Pr[\leftarrow \text{NewGB}_j^c] - \Pr[1 \leftarrow \text{NewGB}_j^d]| \leq \sqrt{(q_H + 1) \cdot \Pr[\text{Find} : \text{NewGB}_j^c]}. \quad (54)$$

Then, by using Lemma 8, the game BadG_0 defined in Fig. 26 satisfies

$$\Pr[\text{Find} : \text{NewGB}_j^c] \leq 4q_H \cdot \Pr[1 \leftarrow \text{BadG}_0]. \quad (55)$$

As for game $\mathbf{NewGB}_j^d$, by an analysis similar to Eq. (51) but with the perspective switched from **Prog** to **Trans**, $\mathbf{NewGB}_j^d$ is equivalent to $\mathbf{NewGB}_{j+1}$ as defined in Fig. 27. Thus

$$\Pr[1 \leftarrow \mathbf{NewGB}_j^d] = \Pr[1 \leftarrow \mathbf{NewGB}_{j+1}]. \quad (56)$$

BadG₀
1 : $pk = (\mathbf{A}, \mathbf{t}_1), sk = (\mathbf{A}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_1, \mathbf{t}_0), Q = \emptyset$ 2 : $\mathbf{y}_{j+1} \leftarrow \bar{S}_{\gamma_1}^\ell, \mathbf{e}_{j+1} \leftarrow S_{\eta_1}^k, \mathbf{w}_{j+1} = \mathbf{A}\mathbf{y}_{j+1} + \mathbf{e}_{j+1}, c_{j+1} \leftarrow \text{ChSet}$ 3 : $l \leftarrow \{1, \dots, q_H\}$, run $\mathcal{A}^{\mathbf{H}, \text{Sign}}(pk)$, the first i signing query are replied by Sign_3 4 : and the remaining signing query are replied by Sign_2 5 : abort just before the l -th H query, measure the input register and denote the outcome as x 6 : if $x \in \{(\text{HighBits}(\mathbf{w}_{j+1}, 2\gamma_2), x) : x \in \mathcal{M}\}$, return 1, else return 0

Fig. 28: Game $\mathbf{BadG}_0$ used in Appendix B.2. Here, the random oracle **H** is simulated by using a $2q_H$ -wise independent function.

In game $\mathbf{BadG}_0$, unlike game $\mathbf{NewGB}_j^c$, $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ are no longer sampled, and the $(i+1)$ -th signing query is directly answered by Sign_2 , without employing distinct loop bodies across different loops. This is an equivalent modification, for the following reason:

- Since the $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$ sampled in $\mathbf{NewGB}_j^c$ ensures that $\mathbf{Bad}_j$ occurs and the $(j+1)$ -th **Prog** loop only sets $(\mathbf{w}_1, c, (\mathbf{z}, \mathbf{h})) = (\perp, \perp, \perp)$, the first j **Trans** loops and the $(j+1)$ -th **Prog** of the $(i+1)$ -th signing query must all yield $(\mathbf{z}, \mathbf{h}) = \perp$ and produce no signature. Since the **Trans** loop does nothing when $(\mathbf{z}, \mathbf{h}) = \perp$ (unlike **Prog**, which requires reprogramming **H**), we can directly omit these $j+1$ loops and instead expect the subsequent **Prog** loops to successfully generate a signature. In this case, there is no need to sample $(\mathbf{y}_1, \mathbf{e}_1, c_1), \dots, (\mathbf{y}_j, \mathbf{e}_j, c_j)$, and the $(i+1)$ -th signing query is in fact answered by Sign_2 (whose loop bodies are all **Prog**).

In addition, note that unlike $\mathbf{NewGB}_j^c$, $\mathbf{BadG}_0$ does not check at the end whether $(\mathbf{y}_{j+1}, \mathbf{e}_{j+1}, c_{j+1})$ causes $\mathbf{Bad}_{j+1}$ to occur. This is reasonable because there is no need to consider whether $\mathbf{Bad}_{j+1}$ occurs when computing the probability of **Find**.

Finally, by combining Eq. (50) to Eq. (56), we have

$$\begin{aligned} & |\Pr[1 \leftarrow \mathbf{NewG}_j] - \Pr[1 \leftarrow \mathbf{NewG}_{j+1}]| \\ & \leq (p_{sk})^j \cdot 4(q_H + 1) \cdot \sqrt{\Pr[\Pr[1 \leftarrow \mathbf{BadG}_0]]}. \end{aligned} \quad (57)$$

In fact, Eq. (57) is exactly the QROM counterpart of Eq. (34). Now, by using Eq. (57) in place of Eq. (34) and following the proof strategy of Lemma 6 after Eq. (37), we can construct an AMLWE adversary $\mathcal{A}_i$ such that

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_3^i] - \Pr[1 \leftarrow \mathbf{G}_3^{i+1}]| & \leq \frac{4q_H + 4}{1 - \bar{p}} \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i)} + \frac{4q_H + 4}{1 - \bar{p}} \cdot \sqrt{\epsilon} \\ & \quad + \Pr[sk \notin \Gamma_p]. \end{aligned}$$

Here $\mathcal{A}_i$ has roughly the same running time as $\mathcal{A}$. Based on the above formula, the QROM counterpart of Lemma 6 can be restated as follows:

Lemma 6. *For any $i \in \{0, 1, \dots, q_S - 1\}$, we can construct an AMLWE adversary $\mathcal{A}_i$ such that $|\Pr[1 \leftarrow \mathbf{G}_3^i] - \Pr[1 \leftarrow \mathbf{G}_3^{i+1}]| \leq \frac{4q_H+4}{1-\bar{p}} \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{A}_i)} + \frac{4q_H+4}{1-\bar{p}} \cdot \sqrt{\epsilon} + \Pr[sk \notin \Gamma_p]$, and the running time of $\mathcal{A}_i$ is roughly the same as that of $\mathcal{A}$.*

B.3 Putting Everything Together

By replacing Lemma 5 and Lemma 6 used in Appendix A with those reproved in Appendix B.1 and Appendix B.2, respectively, we can prove Theorem 2, which we restate below.

Theorem 2. *Let $\bar{p}$ and Γ_p be such that $0 < \bar{p} < 1$ and $p_{sk} \leq \bar{p}$ for all $sk \in \Gamma_p$, where p_{sk} is defined in Eq. (7). Let $\mathcal{A}$ be a SUF-CMA adversary against Aigis-Sig⁺, making q_S classical queries to the signing oracle and q_H quantum queries to the random oracle $\mathbf{H}$. Then we can construct two adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ against the AMLWE assumption, an adversary $\mathcal{C}$ against the SelfTargetAMSIS assumption, and an adversary $\mathcal{D}$ against the AMSIS assumption such that*

$$\begin{aligned} \text{Adv}_{\text{Aigis-Sig}^+}^{\text{SUF-CMA}}(\mathcal{A}) &\leq \text{Adv}_{n,q,k,\ell,\eta_1,\eta_2}^{\text{AMLWE}}(\mathcal{B}_1) + \text{Adv}_{n,q,k,k,\ell,\gamma_3+\tau \cdot 2^{d-1},\gamma_1-\beta_1}^{\text{SelfTargetAMSIS}}(\mathcal{C}) \\ &\quad + \text{Adv}_{n,q,k,k,\ell,2\gamma_3,2\gamma_1}^{\text{AMSIS}}(\mathcal{D}) \\ &\quad + \frac{32q_S(q_H+2)}{(1-\bar{p})^2} \cdot \sqrt{\text{Adv}_{n,q,k,\ell,\gamma_1,\eta_1}^{\text{AMLWE}}(\mathcal{B}_2)} \\ &\quad + \frac{2q_S(4q_H+q_S+5)}{(1-\bar{p})^2} \cdot \sqrt{\epsilon} + \frac{4q_S}{1-\bar{p}} \cdot \Pr[sk \notin \Gamma_p]. \end{aligned}$$

Here $\mathcal{C}$ makes at most q_H quantum queries to its oracle, ϵ is defined in Eq. (7), the probability $\Pr[sk \notin \Gamma_p]$ is taken over $(pk, sk) \leftarrow \text{Aigis.id.Gen}$ (Fig. 10), and the running times of $\mathcal{B}_1$, $\mathcal{B}_2$, $\mathcal{C}$, and $\mathcal{D}$ are roughly the same as those of $\mathcal{A}$.